Reinforcement Learning

Comprehensive Lecture Notes

Vittorio Cava

A.Y. 2026/27

Contents

I Lecture 2 **8**

1 Recap: Multi-Armed Bandits *(slides 2–18)* **8**

1.1 Exploration: the Big Pain of RL *(slides 3)* . . . 8
1.2 Multi-Armed Bandit *(slides 4–5)* . . . 8
1.3 Regret *(slides 6)* . . . 9
1.4 Greedy Algorithm *(slides 7)* . . . 9
1.5 Explore & Commit Algorithm *(slides 8)* . . . 10
1.6 Hoeffding Inequality *(slides 9–10)* . . . 10
1.7 Regret Calculation *(slides 11–12)* . . . 11
1.8 Regret Decaying *(slides 13)* . . . 11
1.9 Statistics to Maintain & Confidence *(slides 14)* . . . 12
1.10 Optimism in the Face of Uncertainty *(slides 15)* . . . 12
1.11 UCB Algorithm *(slides 16)* . . . 12
1.12 UCB Algorithm: Regret *(slides 17)* . . . 13

2 From Multi-Armed to Contextual Bandits *(slides 19–24)* **14**

2.1 From Multi-Armed to Contextual Bandits *(slides 19–20)* . . . 14
2.2 Contextual Bandits: Interaction *(slides 21–23)* . . . 14
2.3 Contextual Bandits: Example *(slides 24)* . . . 15

3 Policy *(slides 25)* **15**

4 Contextual Bandits: Regret *(slides 26)* **15**

5 Explore & Commit Algorithm *(slides 27–31)* **16**

5.1 The algorithm *(slides 27)* . . . 16
5.2 Unbiased estimates: inverse propensity scoring *(slides 27–29)* . . . 16
5.3 Fitting a policy *(slides 27)* . . . 17
5.4 Simulation *(slides 30)* . . . 17

5.5 Explore & Commit Algorithm: Regret *(slides 31)* 17

6 ϵ-Greedy *(slides 32–35)* 18

7 Real World Example *(slides 36)* 20

8 Bayes Theorem Recall *(slides 37)* 20

9 Bayesian Bandits *(slides 38)* 21

10 Gaussian Bayesian Bandits Example *(slides 39–40)* 21

10.1 Setting *(slides 39)* 21

10.2 Posterior update *(slides 40)* 21

11 Gaussian Bayesian Bandits: UCB *(slides 41–43)* 23

12 Gaussian Bayesian Bandits: Thompson Sampling *(slides 44–48)* 24

12.1 Probability matching *(slides 44–45)* 24

12.2 The Gaussian algorithm *(slides 46–47)* 25

12.3 Simulation *(slides 48)* 26

13 Bayesian Bandits: Thompson Sampling *(slides 49)* 26

14 Summary 27

A Supplementary material (in the companion note, not on the slides) 28

B Notation map and slide/note discrepancies 29

II Lecture 2 – Part 2 30

1 Recap *(slides 2–11)* 30

1.1 From Multi-Armed to Contextual Bandits *(slides 3)* 30

1.2 Contextual Bandits: Interaction *(slides 4)* 30

1.3 Contextual Bandits: Regret *(slides 5)* 30

1.4 Explore & Commit Algorithm *(slides 6)* 31

1.5 ϵ-Greedy *(slides 7)* 31

1.6 Bayesian Bandits *(slides 8)* 32

1.7 Gaussian Bayesian Bandits: UCB *(slides 9)* 32

1.8 Gaussian Bayesian Bandits: Thompson Sampling *(slides 10)* 32

2 Sequential Decision Making *(slides 12–14)* 32

3 Markov Assumption *(slides 15–17)* 33

4 Markov Decision Process (MDP) *(slides 18–19)* **34**

5 Reward *(slides 20)* **35**

6 Deterministic MDP Example *(slides 21)* **35**

7 Stochastic MDP Example *(slides 22)* **36**

8 Policy *(slides 23)* **37**

9 Trajectory Probability *(slides 24)* **37**

10 State Visitation Probability *(slides 25–27)* **37**

10.1 Definition *(slides 25)* 37

10.2 Examples *(slides 26–27)* 37

11 Another Example MDP *(slides 28)* **39**

12 Infinite Horizon Discounted Setting *(slides 29)* **39**

13 Value Function *(slides 30–31)* **40**

14 Back to Discount Factor *(slides 32)* **40**

15 Bellman Equation *(slides 33–36)* **41**

16 Discounted State-Action Distribution *(slides 37–38)* **42**

17 Optimal Policy *(slides 39–40)* **42**

18 Bellman Optimality *(slides 41–42)* **43**

19 Bellman Optimality Example *(slides 43–45)* **43**

20 Bellman Optimality (Theorem 1) *(slides 46–49)* **44**

21 Bellman Optimality (Theorem 2) *(slides 50–55)* **45**

22 Summary **47**

A Appendix: supplementary material (in the companion PDF, not on the slides) **48**

A.1 Discounted sums as expectations under $d^{\pi}_{s_0}$ 48

A.2 Bellman operators (notation used in the PDF's proofs) 48

III Lecture 3 **49**

1 Recap: Sequential Decision Making and MDPs *(slides 2–12)* **50**

1.1 Sequential Decision Making and the MDP *(slides 3–4)* . . . 50
1.2 Policies, Value Functions and Q-Functions *(slides 5–6)* . . . 50
1.3 The Discount Factor 7 . . . 51
1.4 Bellman Expectation Equation 8 . . . 51
1.5 Bellman Optimality *(slides 9–12)* . . . 52

2 Policy Evaluation *(slides 13–15)* 52

3 Exact Policy Evaluation *(slides 16–22)* 53

4 Fixed-Point Iteration *(slides 23–24)* 54

5 Contractions *(slides 25–28)* 55
5.1 Contraction Operator *(slides 27–28)* . . . 56

6 Iterative Policy Evaluation *(slides 29–32)* 57

7 Iterative Policy Evaluation Theorem *(slides 33–37)* 58

8 Iterative Policy Evaluation: Iterations 38 59

9 How to Find the Optimal Policy? *(slides 39–43)* 60

10 The Bellman Optimality Backup is a Contraction *(slides 44–47)* 60

11 Bellman Operator for Q 48 61

12 Value Iteration *(slides 49–53)* 61

13 Policy from Value Iteration *(slides 54–65)* 62

14 Number of Iterations of Value Iteration *(slides 66–74)* 64

15 Another Note on Value Iteration 75 65

16 Summary 66
16.1 Comparison of the methods and results . . . 66
16.2 Cost scaling . . . 66
16.3 Key take-aways . . . 66

A Supplementary: Banach's Theorem, Existence and Rate 67

B Supplementary: Self-Check List from the Companion Note 68

IV Lecture 3 – Part 2 69

Setting, notation and imported results 69

1 Recap2–11 **69**
1.1 Bellman Equation3 . . . 69
1.2 Bellman Optimality Example4 . . . 70
1.3 Exact Policy Evaluation5 . . . 70
1.4 Fixed-Point Iteration & Contractions6 . . . 71
1.5 Iterative Policy Evaluation7 . . . 72
1.6 How to Find the Optimal Policy?8 . . . 72
1.7 Value Iteration & Optimal Policy9 . . . 72
1.8 Another Note on Value Iteration10 . . . 72

2 Policy Iteration12–20 **72**

3 Properties of Policy Iteration21–35 **73**
3.1 Monotonic Improvement23–30 . . . 74
3.2 Convergence31 . . . 75
3.3 Complexity32 . . . 76
3.4 Is there a max number of iterations of policy iteration?33–34 . . . 76
3.5 When do we stop?35 . . . 76

4 Advantage Function36–38 **76**
4.1 Policy Iteration with Advantage Function38 . . . 77

5 Performance Difference Lemma39–59 **77**

6 We Did Dynamic Programming!60–62 **79**

7 Primal Linear Program63–66 **80**

8 Summary **81**

A Supplementary material (not on the slides) **82**

V Lecture 4 **84**

1 Recap *(slides 2–10)* **85**
1.1 Policy Iteration *(slides 3–6)* . . . 85
1.2 We Did Dynamic Programming! *(slides 7–8)* . . . 85
1.3 Primal Linear Program *(slides 9)* . . . 86

2 Finite-Horizon MDPs *(slides 11–25)* **86**
2.1 Definition and time-dependent policies *(slides 11–15)* . . . 86
2.2 Finite-Horizon MDPs — Example *(slides 16)* . . . 86
2.3 Finite-Horizon MDP & Policy Interaction *(slides 17–21)* . . . 87
2.4 Finite-Horizon MDP: V & Q *(slides 22–24)* . . . 87

2.5 Finite-Horizon MDP: Bellman Equation *(slides 25)* 87

3 Finding the Optimal Policy *(slides 26–32)* **87**

4 Control Problems and Optimal Control *(slides 33–39)* **88**

4.1 Control Problems *(slides 33–36)* 88

4.2 Optimal Control *(slides 37–39)* 89

5 Discretization and Bellman's Curse of Dimensionality *(slides 40–42)* (SKIPPED IN CLASS) **89**

6 Linear Systems and Quadratic Cost *(slides 43–46)* **90**

6.1 Linear Systems *(slides 43–44)* 90

6.2 Quadratic Cost Function *(slides 45–46)* 90

7 Q Functions for Linear Systems and Quadratic Cost *(slides 47–54)* **91**

8 Finding the Best Action and the Inductive Step *(slides 55–63)* **91**

8.1 Finding the Best Action *(slides 55–57)* 91

8.2 Inductive Step: Proof *(slides 58–63)* 92

9 The LQR Algorithm *(slides 64–70)* **92**

9.1 Cost at H *(slides 64–65)* 92

9.2 LQR algorithm *(slides 66–70)* 93

10 LQR Extensions *(slides 71–76)* **94**

10.1 LQR for Linear Time Varying (LTV) Systems *(slides 72–73)* 94

10.2 LQR for Affine Systems *(slides 74)* 94

10.3 LQR for Stochastic Systems *(slides 75)* 94

10.4 Penalize for Change in Controls *(slides 76)* 95

11 LQR for Non-Linear Systems *(slides 77–82)* **95**

12 LQR for Trajectory Following in Non-Linear Systems *(slides 83–85)* **96**

13 Iterative LQR (iLQR) *(slides 86–92)* **97**

13.1 Iterative LQR (iLQR) *(slides 86)* 97

13.2 Iterative LQR — Algorithm *(slides 87)* 97

13.3 Iterative LQR — LTV *(slides 88–89)* 97

13.4 Iterative LQR — Convergence *(slides 90–91)* 98

13.5 Iterative LQR — Tricks *(slides 92)* 99

14 Receding Horizon Control, i.e. Model Predictive Control (MPC) *(slides 93)* **99**

15 Summary **100**

15.1 Comparison of the methods 100
15.2 Key take-aways 100

A Supplementary material (in the companion PDFs, not on the slides) 101

Part I

Lecture 2

How to read these notes

Structure. Sections and subsections follow the *slide deck's own flow*, and each carries its slide range. The formal content of the companion note (definitions, lemmas, theorems, proofs) is merged at the point where the slides introduce the topic. Material that is in the companion note but has *no* corresponding slide is collected in Appendix A and clearly marked as supplementary.
Boxes. Definitions, theorems, ... are framed in colour; every non-obvious result is followed by a blue **Intuition** box. All figures are original TikZ/pgfplots drawings (no external images). Figures labelled "toy simulation" were produced by small seeded simulations whose data are embedded in the source.
Notation. The slides call the context distribution μ, which clashes with the arm means μ_i; here it is written ρ (as in the companion note, which also writes z_t, $\mathcal{Z}$ for x_t, $\mathcal{X}$). The full notation map and a list of small slide/note discrepancies are in Appendix B.

1 Recap: Multi-Armed Bandits *(slides 2–18)*

Slides 2–18 recall Lecture 1 (bandits and regret). The formal results used later are Hoeffding's inequality, the explore-then-commit regret bound and the UCB regret bound; they are cited in the companion note as Theorem 5, Theorem 8 and Corollary 9 of Lecture 1. Short derivations are included here so the notes are self-contained.

1.1 Exploration: the Big Pain of RL *(slides 3)*

An RL agent must *carefully and systematically explore*: remember which states it has visited and try to reach unexplored regions. The core tension is the **exploration–exploitation trade-off**: either make the best decision given the current information (*exploit*) or collect more information (*explore*), i.e. sacrifice something now to gain more later. The slide calls it a chicken-and-egg problem: to know which option is best you must try it, but trying it costs you if it is not the best.

Intuition

Going to your favourite restaurant is the safe choice (exploit); trying the new place round the corner may disappoint tonight but could reveal a new favourite for the next hundred dinners (explore). The longer the horizon, the more an early experiment is worth; with one dinner left, you should simply exploit. Bandits are the simplest model in which this trade-off can be analysed exactly.

1.2 Multi-Armed Bandit *(slides 4–5)*

Definition 1.1 (Multi-armed bandit)**.** A *multi-armed bandit* (MAB) is a simplified MDP with a **single state** and K actions ("arms") $a_1, \dots, a_K$. Arm i has an unknown reward distribution ν_i with mean $\mu_i = \mathbb{E}_{r\sim\nu_i}[r]$. For $t = 0, \dots, T-1$ the learner (1) pulls an arm $I_t \in \{1, \dots, K\}$ based on the history and (2) observes an i.i.d. reward $r_t \sim \nu_{I_t}$; the rewards of untried arms are *not* observed.

Intuition

Think of K slot machines. Each pull gives you a payoff *and* information about that one machine only ("bandit feedback"). Because information only comes from the arms you choose, *what you do determines what you learn*; this is why exploration is needed at all. The question left open by the slide is: "what are we trying to optimise exactly?" The answer is the regret.

1.3 Regret *(slides 6)*

Definition 1.2 (Regret)**.** With $\mu^\star = \max_{i\in[K]} \mu_i$,

$$\text{Regret}_T \;=\; T\mu^\star - \sum_{t=0}^{T-1} \mu_{I_t}.$$

It is the total expected reward obtained by pulling the best arm for T rounds minus the total expected reward of the arms actually pulled (the *opportunity loss*).

Intuition

Writing $\Delta_i = \mu^\star - \mu_i$ and $N_T(i)$ for the number of pulls of arm i, regret equals $\sum_i \Delta_i N_T(i)$: every pull of a suboptimal arm costs its gap. With means 0.6 and 0.4 and $T = 100$, pulling the worse arm 10 times costs $10 \times 0.2 = 2$ reward units, whereas pulling it 50 times costs 10. A good algorithm therefore wants *few* pulls of bad arms, but it can only know an arm is bad by pulling it.

1.4 Greedy Algorithm *(slides 7)*

Algorithm 1.3 (Greedy)**.** Try each arm once, then commit to the arm with the highest observed reward.

The problem is that a bad arm (low μ_i) may produce a high reward by chance, since $r \sim \nu_i$ is random.

Example 1.4 (Two Bernoulli arms)**.** Arm a_1 pays 1 with probability 0.6 (else 0); arm a_2 pays 1 with probability 0.4. Arm a_1 is clearly better, yet with probability $0.4 \times 0.4 = 16\%$ we observe $(r_1, r_2) = (0, 1)$ and the greedy rule commits to a_2.

	$r(a_2) = 0$	$r(a_2) = 1$
$r(a_1) = 1$ prob. 0.6	0.36 picks a_1 ✓	0.24 tie
$r(a_1) = 0$ prob. 0.4	0.24 tie	0.16 picks a_2 ×

Figure 1: Outcomes of the first two pulls in the greedy example (probabilities multiply because the arms are independent).

Intuition

Greedy treats one noisy sample as the truth. If ties are broken uniformly, the probability of locking onto the worse arm is $0.16 + 0.24 = 0.40$ (not just the 16% highlighted on the

slide), and once committed it loses 0.2 per round forever, so the expected regret is about $0.4 \times 0.2\,T = 0.08\,T$: *linear* in T. Any sensible algorithm must keep sampling long enough to be confident.

1.5 Explore & Commit Algorithm *(slides 8)*

Algorithm 1.5 (Explore-then-commit, MAB)**.** Choose an exploration length N per arm (the slide writes $N = T/K$ as a placeholder; it must satisfy $NK \ll T$, and the optimised value is derived on slide 12).

1. **Explore.** For $k = 1, \dots, K$: pull arm k exactly N times and compute the empirical mean $\hat{\mu}_k = \frac{1}{N}\sum_{i=1}^{N} r_i$.
2. **Commit.** For $t = NK, \dots, T-1$: pull $\hat{I} = \arg\max_{i \in [K]} \hat{\mu}_i$.

1.6 Hoeffding Inequality *(slides 9–10)*

To say how wrong $\hat{\mu}$ can be, we need a confidence interval.

Theorem 1.6 (Hoeffding inequality (Lecture 1, Thm. 5))**.** *Let $X_1, \dots, X_N \in [0,1]$ be i.i.d. with mean μ. For every $\delta \in (0,1)$, with probability at least $1-\delta$,*

$$\left|\tfrac{1}{N}\textstyle\sum_{i=1}^{N} X_i - \mu\right| \;\le\; \sqrt{\tfrac{\ln(2/\delta)}{2N}} \;=\; O\!\left(\sqrt{\tfrac{\ln(1/\delta)}{N}}\right).$$

Applying it to all K arms at once with a union bound (replace δ by δ/K) gives, with probability at least $1-\delta$, $|\hat{\mu}_i - \mu_i| \le \sqrt{\ln(2K/\delta)/(2N)}$ simultaneously *for every arm i.*

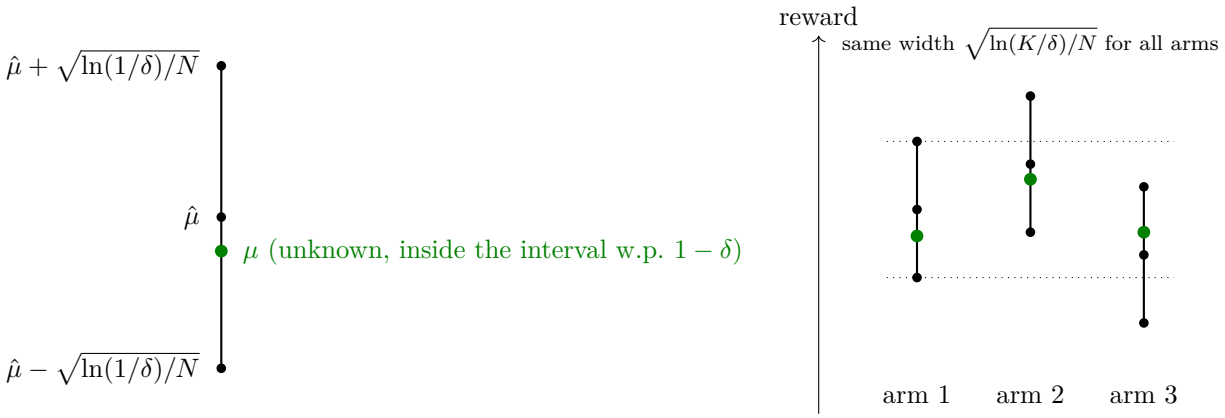


Figure 2: Left (slide 9): the confidence interval around one empirical mean; the true mean (green) lies inside with probability $1-\delta$. Right (slide 10): during commitment, all K intervals hold simultaneously, so the width contains $\ln(K/\delta)$. Black dots: $\hat{\mu}_i$ and interval ends.

Intuition

The interval shrinks like $1/\sqrt{N}$: to halve the uncertainty you need four times the data. With $\delta = 0.01$ ("the bound holds with probability 99%") and $N = 100$ samples, the half-width is $\sqrt{\ln(200)/200} \approx 0.163$; with $N = 1000$ it is ≈ 0.051. For $K = 10$ arms and $N = 100$, the union bound makes it $\sqrt{\ln(2000)/200} \approx 0.195$: the price of "all arms at once" is only logarithmic in K.

1.7 Regret Calculation *(slides 11–12)*

Let $\hat{I} = \arg\max_i \hat{\mu}_i$ (empirical best arm) and $I^\star = \arg\max_i \mu_i$ (best arm).

Theorem 1.7 (Regret of explore-then-commit, MAB)**.** *Let $L = \ln(2K/\delta)$ (the slides write $\ln(K/\delta)$, absorbing constants into the $O(\cdot)$) and $1 \le NK \le T$. With probability at least $1-\delta$,*

$$\mathrm{Regret}_T = \mathrm{Regret}_{\text{explore}} + \mathrm{Regret}_{\text{exploit}} \;\le\; NK + 2T\sqrt{\frac{L}{N}}.$$

Choosing $N^\star = \left(T\sqrt{L}/K\right)^{2/3}$ gives $\mathrm{Regret}_T \le 3\,T^{2/3}K^{1/3}L^{1/3} = O\big(T^{2/3}K^{1/3}\ln^{1/3}(K/\delta)\big)$.

Proof. Let $\varepsilon = \sqrt{L/(2N)}$. By Theorem 1.6 all $|\hat{\mu}_i - \mu_i| \le \varepsilon$ on an event of probability $\ge 1-\delta$. On it,

$$\mu_{I^\star} \le \hat{\mu}_{I^\star} + \varepsilon \le \hat{\mu}_{\hat{I}} + \varepsilon \le \mu_{\hat{I}} + 2\varepsilon,$$

so every committed round costs at most 2ε. Exploration lasts NK rounds with per-round regret ≤ 1, so $\mathrm{Regret}_{\text{explore}} \le NK$, and $\mathrm{Regret}_{\text{exploit}} \le (T-NK)\,2\varepsilon \le 2T\sqrt{L/N}$. For the second claim let $f(N) = NK + 2T\sqrt{L}\,N^{-1/2}$. Then $f'(N) = K - T\sqrt{L}\,N^{-3/2}$ vanishes at $N^\star = (T\sqrt{L}/K)^{2/3}$ and f is convex, so this is the minimiser. From $N^{\star 3/2} = T\sqrt{L}/K$ we get $2T\sqrt{L}\,N^{\star -1/2} = 2KN^\star$, hence $f(N^\star) = 3KN^\star = 3K^{1/3}T^{2/3}L^{1/3}$ (ignoring rounding of $N^\star$). □

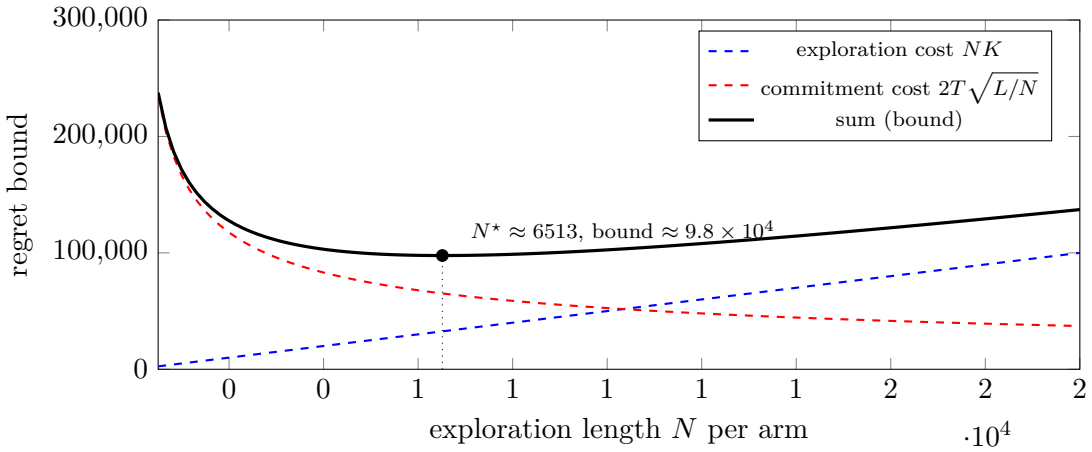


Figure 3: The explore/exploit trade-off inside Theorem 1.7 for $T = 10^6$, $K = 5$, $\delta = 0.01$ ($L \approx 6.91$): more exploration costs linearly, but shrinks the commitment error like $1/\sqrt{N}$.

Intuition

Too little exploration makes you commit to the wrong arm; too much wastes rounds on bad arms. The optimum balances the two curves (at $N^\star$ the commitment term is exactly twice the exploration term). Numerically, for $T = 10^6$, $K = 5$, $\delta = 0.01$: $N^\star \approx 6{,}513$ pulls per arm, i.e. $\approx 32{,}600$ exploration rounds (3.3% of T), and a regret bound of $\approx 9.8 \times 10^4$, about 9.8% of the maximum possible. Dividing by T, the *average* regret per round decays like $T^{-1/3}$, i.e. it "approaches 0 as $T \to \infty$" (slide 12), but slowly.

1.8 Regret Decaying *(slides 13)*

The $T^{2/3}$ rate of explore-then-commit is "kind of slow". The slide asks for something like $O(\sqrt{T})$ and states that $\sqrt{T}$ is the best possible: it is a *lower bound*, no algorithm can do better in the

worst case (the standard minimax bound is $\Omega(\sqrt{KT})$; it is quoted, not proved, in the lecture). So we look for a new algorithm.

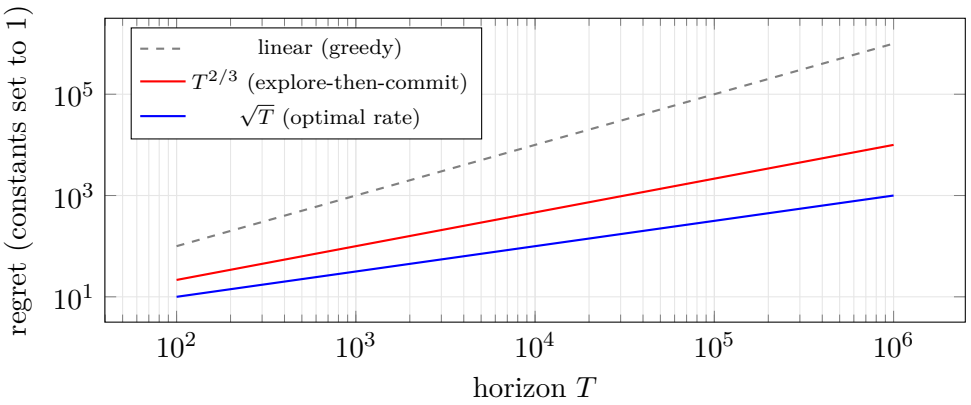


Figure 4: Growth rates compared (slide 13 describes them in words). On a log–log plot a rate T^α is a line of slope α.

Intuition

For $T = 10^6$: $\sqrt{T} = 10^3$ versus $T^{2/3} = 10^4$. A tenfold gap, i.e. per-round regret 0.1% versus 1% (with all constants set to 1).

1.9 Statistics to Maintain & Confidence *(slides 14)*

To compute confidence bounds *while* learning we keep, for each arm i at time t:

$$N_t(i) = \sum_{\tau=0}^{t-1} \mathbf{1}\{I_\tau = i\}, \qquad \hat{\mu}_t(i) = \frac{1}{N_t(i)} \sum_{\tau=0}^{t-1} \mathbf{1}\{I_\tau = i\}\, r_\tau,$$

and, with probability $1 - \delta$ (simultaneously for all arms and all times),

$$|\hat{\mu}_t(i) - \mu_i| \le \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}}.$$

Intuition

Unlike explore-then-commit, arms now have *different* sample sizes, so each arm gets its *own* interval width. The KT inside the logarithm is a union bound over every (arm, time) pair, because the algorithm will consult these bounds at every round.

1.10 Optimism in the Face of Uncertainty *(slides 15)*

Pick the arm with the highest *upper* confidence bound (the top of its interval).

1.11 UCB Algorithm *(slides 16)*

Algorithm 1.8 (UCB)**.** Pull each arm once (first K rounds). For $t = K, \dots, T-1$:

1. pick $I_t = \arg\max_{i \in [K]} \Big(\hat{\mu}_t(i) + \sqrt{\frac{\ln(KT/\delta)}{N_t(i)}} \Big)$;
2. observe the reward and update $N_t, \hat{\mu}_t$.

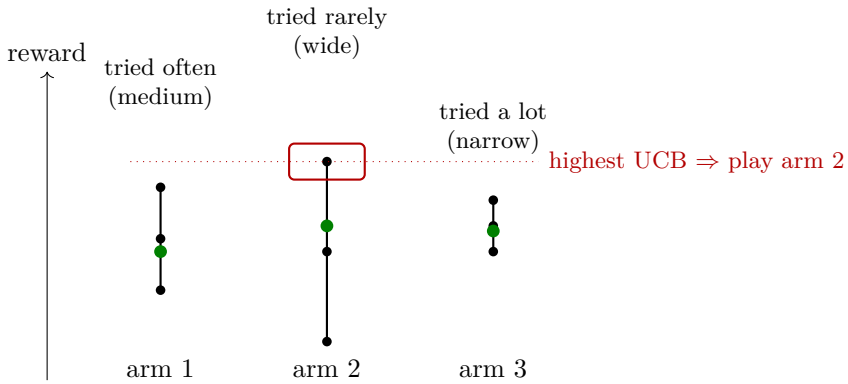


Figure 5: Optimism (slide 15). Arm 2 has a *lower* empirical mean than arm 3, but its wide interval (few pulls) gives the highest upper bound, so UCB explores it. Green dots: true means.

The square-root *reward bonus* is large for arms not tried many times, which drives exploration.

Intuition

UCB acts as if each arm were as good as it *could plausibly be*. Under-sampled arms get an inflated score and are tried; if the optimism was unjustified, the empirical mean drops and $N_t(i)$ grows, so the bonus shrinks and the arm is abandoned. Nothing is hard-coded about *when* to stop exploring: the schedule emerges from the data. Exploration and exploitation are fused into one rule.

1.12 UCB Algorithm: Regret *(slides 17)*

Theorem 1.9 (Regret of UCB (Lecture 1))**.** *Let* $L' = \ln(2KT/\delta)$*. With probability at least* $1-\delta$*, UCB satisfies* $\text{Regret}_T \le K + 4\sqrt{KT\,L'} = \widetilde{O}(\sqrt{KT})$*.*

Proof. On the event that all intervals hold, write $w_t = \sqrt{L'/N_t(I_t)}$. Then

$$\mu^\star \le \hat{\mu}_t(I^\star) + \sqrt{\tfrac{L'}{N_t(I^\star)}} \le \hat{\mu}_t(I_t) + w_t \le \mu_{I_t} + 2w_t,$$

so the per-round regret is $\mu^\star - \mu_{I_t} \le 2w_t$ (the first inequality is validity of the interval for $I^\star$, the second is the UCB rule, the third is validity for I_t). Summing over $t \ge K$,

$$\sum_{t=K}^{T-1} \frac{1}{\sqrt{N_t(I_t)}} = \sum_i \sum_{n=1}^{N_T(i)-1} \frac{1}{\sqrt{n}} \le \sum_i 2\sqrt{N_T(i)} \le 2\sqrt{K \textstyle\sum_i N_T(i)} \le 2\sqrt{KT},$$

by $\sum_{n\le m} n^{-1/2} \le 2\sqrt{m}$ and Cauchy–Schwarz. The first K rounds cost at most K. Hence $\text{Regret}_T \le K + 2\sqrt{L'} \cdot 2\sqrt{KT}$. □

Intuition

Each arm contributes $\sum_{n\le N} 1/\sqrt{n} \approx 2\sqrt{N}$ to the regret sum, and the total number of pulls is capped at T; the worst case is spreading pulls evenly, which yields $\sqrt{KT}$. For $K = 5$, $T = 10^6$ the *rate* is $\approx \sqrt{5 \cdot 10^6} \approx 2.2 \times 10^3$ (times $\sqrt{\ln}$ and a constant), versus the $T^{2/3}$ rate of explore-then-commit which is of order 10^4. (The slide cites an external note for details, https://wensun.github.io/CS4789_data/UCB_note_new.pdf.)

2 From Multi-Armed to Contextual Bandits *(slides 19–24)*

2.1 From Multi-Armed to Contextual Bandits *(slides 19–20)*

A multi-armed bandit is *stateless*: an action leads to a reward. A **contextual bandit** adds a context (state) that is observed before acting and that influences which action is best.

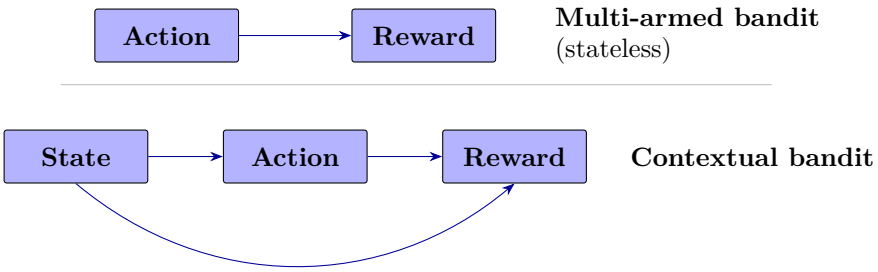


Figure 6: Slides 19–20 redrawn. In a contextual bandit the reward depends on the *state/context and the action* (the curved arrow).

2.2 Contextual Bandits: Interaction *(slides 21–23)*

Definition 2.1 (Contextual bandit)**.** A contextual bandit is a distribution ρ over a context set $\mathcal{X}$ together with a mean-reward function $r : \mathcal{X} \times \mathcal{A} \to [0,1]$, where $|\mathcal{A}| = K$. For $t = 0, 1, \ldots, T-1$:

1. a context $x_t \sim \rho$ is drawn i.i.d. and *revealed*;
2. the learner picks $a_t \in \mathcal{A}$, possibly at random, using x_t and the history;
3. a reward r_t with $\mathbb{E}[r_t \mid x_t, a_t] = r(x_t, a_t)$ is revealed, and nothing is revealed about the other $K-1$ actions.

Remark. For simplicity the slides (slide 22) assume *deterministic* rewards $r_t = r(x_t, a_t)$, "as the context is the challenge here"; the companion note stresses that every result below holds with noisy rewards too (the proof of Lemma 5.3 conditions on the noise).

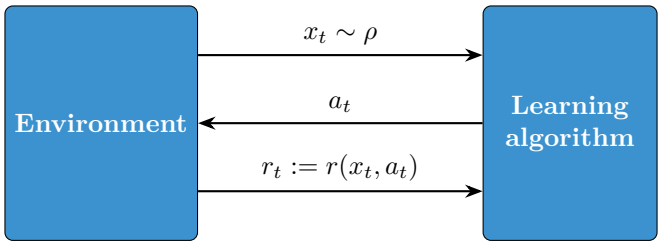


Figure 7: The interaction protocol (slide 23). After the reward, a new round starts: contexts are *not* influenced by past actions, they are sampled i.i.d.

Intuition

The i.i.d. context is what separates a contextual bandit from a full MDP: your action affects your *reward* today but not the *state* you will see tomorrow, so there are no long-term consequences to plan around. It is a bandit with side information. The difficulty that remains is that the best action depends on the context, and you only see the reward of the action you actually took.

2.3 Contextual Bandits: Example *(slides 24)*

A major application is **recommendation systems**: the *context* is user information (age, height, weight, job, ...), the *arms* are items to recommend (news, movies, ...), and each arm has a click-through rate (0/1 reward) that we want to maximise. The question: how do we decide which item to propose next *in a personalised way*?

Example 2.2. Two equally frequent user types and two items, with click-through rates

	news	movies
younger users	0.10	0.30
older users	0.35	0.15

A context-blind bandit sees an average CTR of 0.225 for both items and earns 0.225 per round at best, whereas the personalised rule (movies for younger, news for older) earns $\frac{1}{2}(0.30 + 0.35) = 0.325$.

Intuition

Ignoring context averages away exactly the structure that matters. But learning one separate bandit per context is hopeless when contexts are many or continuous (every user is unique). The remedy, developed next, is to learn a *policy* from a restricted class Π that generalises across contexts.

3 Policy *(slides 25)*

Definition 3.1 (Policy)**.** A policy π is a mapping from (all) states/contexts to actions that determines how the agent selects actions. It can be *deterministic*, $a = \pi(x)$, or *stochastic*, $\pi(a \mid x)$ (equivalently $a \sim \pi(\cdot \mid x)$). Given a class Π of policies, the *value* of π is its expected reward, $V(\pi) = \mathbb{E}_{x\sim\rho}[r(x, \pi(x))]$.

Intuition

A policy is a rulebook: "if the user looks like this, show that". For a finite context set $\mathcal{X}$ with K actions there are $K^{|\mathcal{X}|}$ deterministic policies: with $|\mathcal{X}| = 10$ and $K = 3$ that is $3^{10} = 59{,}049$, huge, but its *logarithm* is only $10 \ln 3 \approx 10.99$. It is this logarithm, $\ln|\Pi|$, that will govern the regret.

4 Contextual Bandits: Regret *(slides 26)*

The optimal policy in the class is $\pi^\star = \arg\max_{\pi\in\Pi} \mathbb{E}_{x\sim\rho}\, r(x, \pi(x)) = \arg\max_{\pi\in\Pi} V(\pi)$. At every round $a_t = \pi_t(x_t)$ is selected and a reward received.

Definition 4.1 (Contextual regret)**.**

$$\mathrm{Reg}(T) \;=\; T\,V(\pi^\star) - \sum_{t=0}^{T-1} V(\pi_t) \;=\; T\,\mathbb{E}_{x\sim\rho}[r(x, \pi^\star(x))] - \sum_{t=0}^{T-1} \mathbb{E}_{x\sim\rho}[r(x, \pi_t(x))],$$

where π_t is the policy the learner is using at round t (note: *it changes at every iteration*).

Intuition

The first term is the total expected reward if we always used the best policy; the second is what our sequence of learned policies earns. If $\mathcal{X}$ is a single context, $V(\pi) = r(a)$ and this is exactly the multi-armed regret of Section 1. In the recommendation example the best policy earns 0.325 per round, so a context-blind learner has regret at least $(0.325 - 0.225)T = 0.1\,T$, linear in T.

5 Explore & Commit Algorithm *(slides 27–31)*

5.1 The algorithm *(slides 27)*

Algorithm 5.1 (Explore-then-commit for contextual bandits)**.** Input: horizon T, policy class Π, exploration budget N.

1. **Explore.** For $t = 0, \dots, N-1$: observe $x_t \sim \rho$, draw $a_t \sim \mathrm{Unif}(\mathcal{A})$, observe $r_t = r(x_t, a_t)$, and build for x_t an *unbiased estimate* $\hat{\mathbf{r}}_t$ of the reward vector $\big(r(x_t, a)\big)_{a \in \mathcal{A}}$.
2. **Fit.** Compute the policy $\hat{\pi} \in \arg\max_{\pi \in \Pi} \sum_{t=0}^{N-1} \hat{\mathbf{r}}_t[\pi(x_t)]$.
3. **Commit.** For $t = N, \dots, T-1$: observe $x_t \sim \rho$ and play $a_t = \hat{\pi}(x_t)$.

The missing ingredient is the unbiased estimate $\hat{\mathbf{r}}_t$: we only observe the reward of *one* action.

5.2 Unbiased estimates: inverse propensity scoring *(slides 27–29)*

An estimator $\hat{\theta}$ is *unbiased* if $\mathbb{E}[\hat{\theta}] = \theta$, i.e. $\mathrm{bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta = 0$ (it is centred on the true value). The goal here: $\mathbb{E}_{a_t \sim p}[\hat{\mathbf{r}}_t[a]] = r(x_t, a)$ for all a.

Definition 5.2 (Inverse propensity scoring (IPS))**.** Suppose that at round t the action was drawn from a *known* distribution p_t with $p_t(a) > 0$ for all a. The IPS reward vector is

$$\hat{\mathbf{r}}_t[a] = \frac{r_t\, \mathbf{1}[a_t = a]}{p_t(a)}, \qquad a \in \mathcal{A},$$

i.e. the vector with a single non-zero entry $r_t / p_t(a_t)$ at the played action. Under uniform exploration, $p_t(a) = 1/K$ and $\hat{\mathbf{r}}_t[a] = \{K r_t \text{ if } a = a_t,\ 0 \text{ otherwise}\}$ (slide 29: "$r_t/(1/|\mathcal{A}|)$").

Lemma 5.3 (IPS is unbiased)**.** *For every context x_t and every action a, $\mathbb{E}_{a_t \sim p_t}[\hat{\mathbf{r}}_t[a] \mid x_t] = r(x_t, a)$. Consequently $\mathbb{E}[\hat{\mathbf{r}}_t[\pi(x_t)]] = V(\pi)$ for every policy π.*

Proof. Condition on x_t and take the expectation over $a_t \sim p_t$ and the reward noise:

$$\mathbb{E}\left[\frac{r_t\, \mathbf{1}[a_t = a]}{p_t(a)}\right] = \sum_{a' \in \mathcal{A}} p_t(a')\, \frac{\mathbb{E}[r_t \mid x_t, a']\, \mathbf{1}[a' = a]}{p_t(a)} = p_t(a)\, \frac{r(x_t, a)}{p_t(a)} = r(x_t, a).$$

Averaging over $x_t \sim \rho$ gives $\mathbb{E}[\hat{\mathbf{r}}_t[\pi(x_t)]] = V(\pi)$. □

Intuition

The division by $p_t(a)$ "removes the asymmetry due to sampling" (slide 28): an action played half as often has its rewards counted twice as heavily, exactly like survey weights that up-weight under-sampled groups. A concrete check with $K = 2$, uniform exploration and $r(x, 2) = 0.8$:

whenever arm 2 is played (half the time) we record $\hat{\mathbf{r}} = (0,\ 1.6)$, otherwise $(\cdot,\ 0)$, so the average is $\frac{1}{2} \cdot 1.6 + \frac{1}{2} \cdot 0 = 0.8$. The estimator is unbiased but *noisy*: a single observation is either 0 or Kr_t, far from the true mean (made precise in Lemma A.1, Appendix A). Importantly, $\hat{\mathbf{r}}_t$ is defined for *all* actions even though only one is observed, which is what lets us evaluate policies that would have acted differently.

5.3 Fitting a policy *(slides 27)*

Step 2 of Algorithm 5.1 maximises the *counterfactual value estimate* $\hat{V}(\pi) = \frac{1}{N} \sum_{t<N} \hat{\mathbf{r}}_t[\pi(x_t)]$ over Π. By Lemma 5.3 each summand has mean $V(\pi)$, so $\hat{V}(\pi)$ estimates $V(\pi)$ for *every* $\pi \in \Pi$ from the *same* exploration data.

Intuition

Under uniform exploration, a round only "counts" for policy π if $\pi(x_t)$ happens to equal the randomly played a_t (probability $1/K$), and then it counts with weight Kr_t. In the toy problem below (two contexts, two arms) the four deterministic policies have values $V(a_1, a_1) = 0.525$, $V(a_1, a_2) = 0.675$, $V(a_2, a_1) = 0.425$, $V(a_2, a_2) = 0.575$ (red $\to$ first entry, blue $\to$ second), so $\pi^\star = (a_1, a_2)$. Because Π here contains *all* context-to-action maps, the maximisation splits into one independent argmax per context; for a restricted class (e.g. linear policies) it does not.

5.4 Simulation *(slides 30)*

Slide 30 shows an animation of the explore phase: a timeline of contexts (red/blue) and, per context, the running estimates of each arm's reward. Here is an original static version.

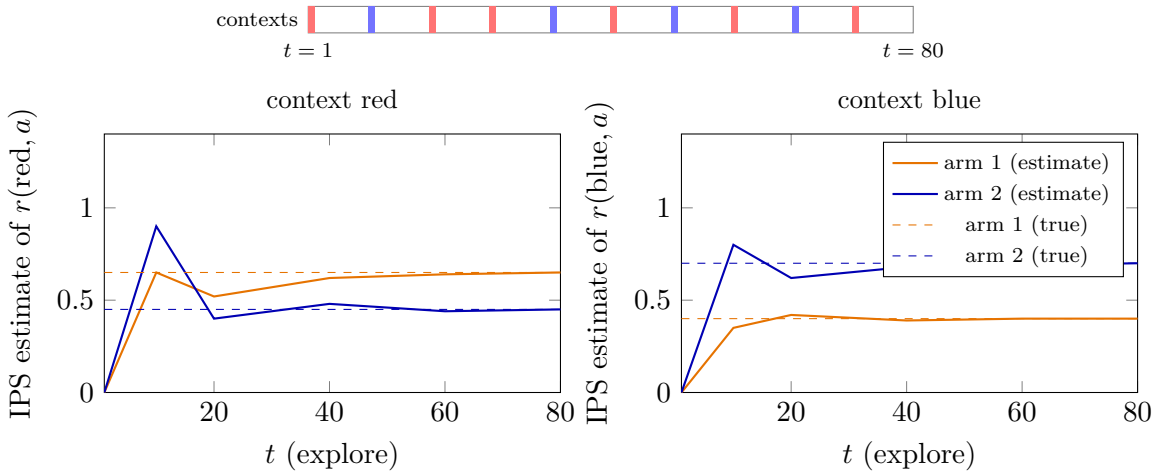


Figure 8: Toy simulation of the explore phase (seeded; $N = 80$, uniform exploration, two equally likely contexts, deterministic rewards $r(\text{red}, \cdot) = (0.65, 0.45)$, $r(\text{blue}, \cdot) = (0.40, 0.70)$). Solid: per-context IPS averages; dashed: true values. They are noisy at first but settle on the correct ordering, so $\hat{\pi} = \pi^\star$ in this run (about 81% of 2000 seeds at $N = 80$).

5.5 Explore & Commit Algorithm: Regret *(slides 31)*

The slide states that the regret "also depends on the size of the class of policies". Here is the precise statement.

Theorem 5.4 (Regret of contextual explore-then-commit)**.** *Let* Π *be finite and run Algorithm 5.1 with* $1 \le N \le T$*. With probability at least* $1-\delta$*,*

$$\mathrm{Reg}(T) \;\le\; N + TK\sqrt{\frac{2\ln(2|\Pi|/\delta)}{N}},$$

and choosing N *to minimise the right-hand side gives*

$$\mathrm{Reg}(T) \;\le\; 3\cdot 2^{-1/3}\,T^{2/3}K^{2/3}\ln^{1/3}\!\Big(\frac{2|\Pi|}{\delta}\Big).$$

Proof. Write $\hat{V}(\pi) = \frac{1}{N}\sum_{t<N}\hat{\mathbf{r}}_t[\pi(x_t)]$. By Lemma 5.3 the summands are i.i.d. with mean $V(\pi)$ and by Lemma A.1 (Appendix A) they lie in $[0,K]$. Hoeffding's inequality (Theorem 1.6, rescaled by K) gives, for one fixed π and any $\delta' \in (0,1)$,

$$\mathbb{P}\Big(|\hat{V}(\pi) - V(\pi)| > K\sqrt{\ln(2/\delta')/(2N)}\Big) \le \delta'.$$

Take $\delta' = \delta/|\Pi|$ and a union bound over the $|\Pi|$ policies: with probability at least $1-\delta$, every $\pi \in \Pi$ satisfies $|\hat{V}(\pi) - V(\pi)| \le \varepsilon$ with $\varepsilon = K\sqrt{\ln(2|\Pi|/\delta)/(2N)}$. On that event, the same three-step chain as in Theorem 1.7 applies to $\hat{\pi}$, which maximises $\hat{V}$:

$$V(\pi^\star) \le \hat{V}(\pi^\star) + \varepsilon \le \hat{V}(\hat{\pi}) + \varepsilon \le V(\hat{\pi}) + 2\varepsilon.$$

Exploration contributes at most N (each round's regret is at most 1); commitment contributes at most $(T-N)\cdot 2\varepsilon \le 2T\varepsilon$. Adding them gives the first display.

For the second, set $c = \sqrt{2\ln(2|\Pi|/\delta)}$, so the bound reads $f(N) = N + TKcN^{-1/2}$. As in the MAB case, f is convex and $f'(N) = 1 - \frac{1}{2}TKcN^{-3/2}$ vanishes at $N^\star = (TKc/2)^{2/3}$. Writing $u = (TKc/2)^{1/3}$ gives $f(N^\star) = u^2 + 2u^2 = 3(TKc/2)^{2/3}$, and substituting $c^{2/3} = 2^{1/3}\ln^{1/3}(2|\Pi|/\delta)$ gives the claim. □

Intuition

It is the same trade-off as in the MAB: exploring costs N in regret; each extra sample sharpens *all* policy estimates simultaneously, and the union bound over policies only costs a $\ln|\Pi|$ factor inside the square root. Numerically, for $|\Pi| = 10^6$, $K = 3$, $\delta = 0.05$, $T = 10^6$: $\ln(2|\Pi|/\delta) \approx 17.5$, the optimal exploration length is $N^\star \approx 4.3\times 10^4$ rounds (4.3% of T) and the bound is $\approx 1.29\times 10^5$ (12.9% of T). Making the class a million times richer, from $|\Pi| = 10^6$ to 10^{12}, multiplies the bound only by $(3.15/2.60) \approx 1.21$ (the factor $\ln^{1/3}(2|\Pi|/\delta)$ goes $2.02 \to 2.60 \to 3.15$ for $|\Pi| = 10^2, 10^6, 10^{12}$). The price of context is thus *logarithmic in the number of policies* but, because uniform exploration wastes a factor K per observation, it grows with K.

Remark (On the exponent of K). Slide 31 prints the bound as $O(T^{2/3}K^{1/3}\ln^{1/3}(|\Pi|))$, while the proof above (which only uses the *range* $[0,K]$ of the IPS terms) yields $K^{2/3}$. The $K^{1/3}$ form is the known sharper rate: using the *variance* bound of Lemma A.1 with Bernstein's inequality instead of Hoeffding's gives deviations of order $\sqrt{K\ln(|\Pi|/\delta)/N}$ and hence $T^{2/3}K^{1/3}\ln^{1/3}(|\Pi|/\delta)$ plus a lower-order term. This refinement is a standard fact but is *not* proved in the companion note, so the theorem is stated with the exponent that is actually proved.

6 ϵ-Greedy *(slides 32–35)*

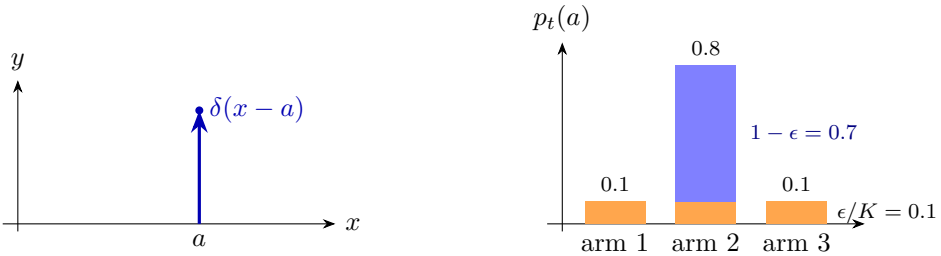
Instead of fixing a threshold for exploring and then committing, we can *interleave* exploration and exploitation.

Algorithm 6.1 (ϵ-greedy for contextual bandits)**.** For $t = 0, 1, \ldots, T-1$:

1. observe $x_t \sim \rho$;
2. draw $a_t \sim p_t = (1-\epsilon_t)\,\delta(\pi^t(x_t)) + \epsilon_t\,\mathrm{Unif}(\mathcal{A})$;
3. observe r_t and form the IPS vector $\hat{\mathbf{r}}_t$ using the propensity $p_t(a_t)$ *actually used*;
4. update the policy: $\pi^{t+1} \in \arg\max_{\pi\in\Pi} \sum_{i\le t} \hat{\mathbf{r}}_i[\pi(x_i)]$.

Gradually **decay** ϵ_t towards zero (slide 34, "Step 3").

Here $\delta(\pi^t(x_t))$ is the Dirac delta placing all the mass on the action $\pi^t(x_t)$, so explicitly $p_t(a) = (1-\epsilon_t)\mathbf{1}[a = \pi^t(x_t)] + \epsilon_t/K$. The two extremes are $\epsilon = 0$ (pure exploitation) and $\epsilon = 1$ (uniform exploration). Since $p_t(a) \ge \epsilon_t/K > 0$, IPS is well defined, and Lemma 5.3 applies at every round (it only needs p_t to be known, and p_t may change with t).



Dirac delta: all the mass on a single point

p_t for $K = 3$, $\epsilon = 0.3$, greedy action = arm 2

Figure 9: Left (slide 33): the Dirac delta used in p_t. Right: the resulting mixture: every arm gets a floor ϵ/K of exploration probability and the greedy arm additionally gets $1-\epsilon$.

Intuition

ϵ_t is a dial between "trust what I have learned" and "roll a die". The floor ϵ_t/K is what guarantees every action keeps being sampled so that IPS stays valid. The dial must be turned down *gradually*: from $r_t \le 1$ and $p_t(a) \ge \epsilon_t/K$ we get $0 \le \hat{\mathbf{r}}_t[a] \le K/\epsilon_t$, so if ϵ_t collapses, the estimates of rarely played actions become extremely noisy (e.g. $\epsilon_t = 0.01$, $K = 3$ allows values up to 300). Note that, unlike explore-then-commit, learning never stops, so the policy keeps adapting. (Neither the slides nor the companion note state a regret bound for ϵ-greedy; slide 35 only shows a simulation.)

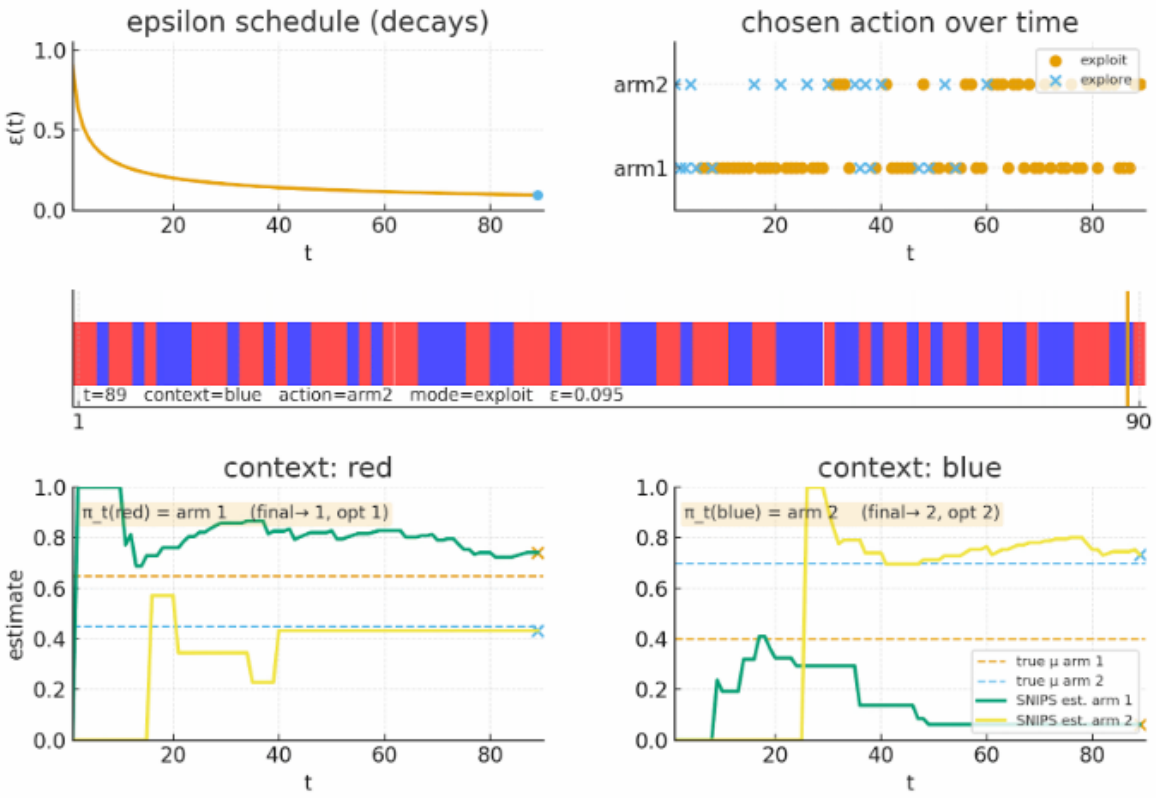


Figure 10: Toy simulation of ϵ-greedy (seeded, $T = 90$): early on most rounds are exploratory (open red circles); as ϵ_t decays the learner mostly exploits its current policy, which here becomes $\pi = (\text{red} \to \text{arm } 1,\ \text{blue} \to \text{arm } 2) = \pi^\star$ (27 of the 90 rounds were exploratory). This ends optimal in about 71% of 1000 seeds at $t = 90$, so individual runs vary; same colours and dashed true values as the earlier figure.

7 Real World Example *(slides 36)*

Slide 36 points to a production use of contextual-bandit algorithms: Microsoft's *Azure Personalizer* (https://azure.microsoft.com/en-us/products/cognitive-services/personalizer/), which the slide notes is *going out of service in October 2026* (the slide shows a screenshot of the product page, not reproduced here).

Intuition

Personalising content maps one-to-one onto the model of this lecture: the context is the user/page features, the actions are the candidate items, and the reward is a click or other feedback. Each shown item produces feedback for *that* item only, which is why IPS-style estimates and exploration are needed in practice. Whether a particular product is still available is a separate matter from the algorithms, which remain standard.

8 Bayes Theorem Recall *(slides 37)*

Theorem 8.1 (Bayes' theorem)**.** *For a hypothesis H and data D with $p(D) > 0$,*

$$p(H \mid D) = \frac{p(H)\, p(D \mid H)}{p(D)}.$$

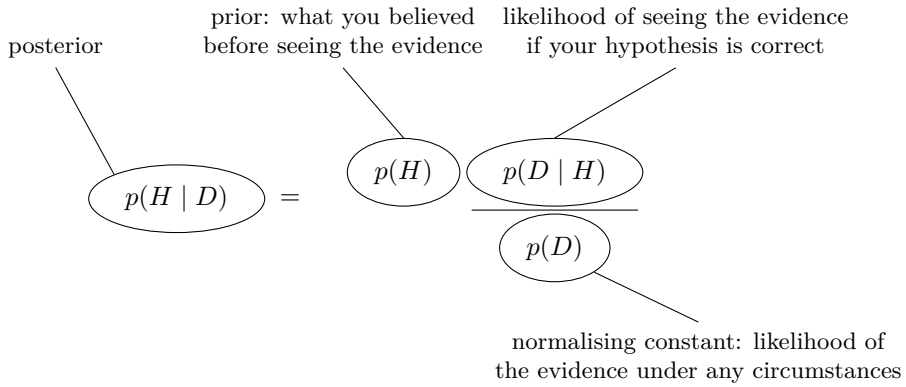


Figure 11: The annotated Bayes formula of slide 37.

Intuition

Posterior $\propto$ prior $\times$ likelihood; $p(D)$ only renormalises. Example: an ad is either *good* (CTR 0.6) or *bad* (CTR 0.4), prior 50/50. After observing one click, $p(\text{good} \mid \text{click}) = \frac{0.5 \cdot 0.6}{0.5 \cdot 0.6 + 0.5 \cdot 0.4} = 0.6$; after two clicks the odds multiply by $0.6/0.4$ again, giving odds 2.25, i.e. $p \approx 0.69$. Each observation nudges belief towards the hypotheses that explain it best. This is the engine of Bayesian bandits.

9 Bayesian Bandits *(slides 38)*

So far we made *no assumption* about the reward distributions ν_i; we only derived bounds that hold for any bounded rewards. In **Bayesian bandits**:

- we exploit *prior* knowledge of the rewards;
- we update a *posterior* distribution of rewards from the history;
- we use the posterior to guide exploration, via **upper confidence bounds** (Bayesian UCB) or **probability matching** (Thompson sampling).

Intuition

A frequentist guarantee protects you against *any* reward distribution but is therefore conservative; a Bayesian method gets a head start when the prior is sensible (e.g. CTRs are mostly small) and the posterior directly tells you "how sure am I about each arm". The price is that the guarantee is only as good as the modelling assumptions.

10 Gaussian Bayesian Bandits Example *(slides 39–40)*

10.1 Setting *(slides 39)*

Assume each reward distribution is Gaussian, $\nu_i = \mathcal{N}(\mu(i), \sigma^2(i))$. After some pulls the learner holds, for each arm, a belief (a Gaussian) about that arm's quality: the centre is the best guess $\mu(i)$ and the width reflects remaining uncertainty.

10.2 Posterior update *(slides 40)*

The prior is often $\mathcal{N}(0, 1)$. The posterior after history h_t is proportional to the prior times the product of the likelihoods of all the rewards observed when this arm was selected (slide 40:

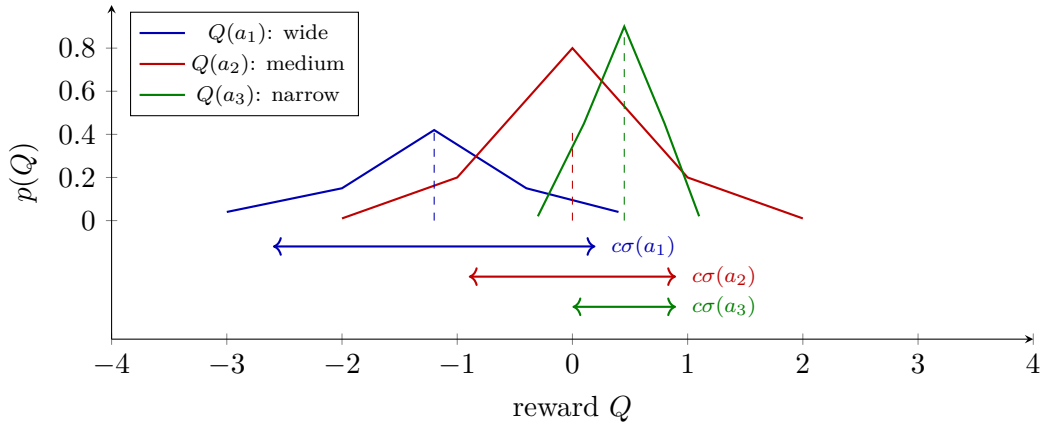


Figure 12: Beliefs over the quality of three arms (slide 39 redrawn; dashed lines mark the means $\mu(a_i)$; the double arrows are confidence widths $\pm c\sigma$ around each mean). Arm 1 is very uncertain, arm 3 is well known.

"product over all time-steps t that select action i"). For a Gaussian prior and Gaussian noise this has a closed form.

Proposition 10.1 (Gaussian conjugacy)**.** *Let the unknown mean of an arm have prior $\mu \sim \mathcal{N}(m_0, s_0^2)$, and let $r_1, \dots, r_n$ be i.i.d. $\mathcal{N}(\mu, \sigma^2)$ with σ^2 known. Then the posterior is $\mathcal{N}(m_n, s_n^2)$ with*

$$\frac{1}{s_n^2} = \frac{1}{s_0^2} + \frac{n}{\sigma^2}, \qquad m_n = s_n^2\Big(\frac{m_0}{s_0^2} + \frac{1}{\sigma^2}\sum_{i=1}^{n} r_i\Big).$$

Proof. By Bayes' rule (Theorem 8.1) the posterior density is proportional to prior times likelihood,

$$p(\mu \mid r_{1:n}) \propto \exp\Big(-\frac{(\mu - m_0)^2}{2s_0^2}\Big) \prod_{i=1}^{n} \exp\Big(-\frac{(r_i - \mu)^2}{2\sigma^2}\Big).$$

The exponent is a quadratic in μ; collecting terms,

$$-\frac{1}{2}\Big[\mu^2\Big(\frac{1}{s_0^2} + \frac{n}{\sigma^2}\Big) - 2\mu\Big(\frac{m_0}{s_0^2} + \frac{\sum_i r_i}{\sigma^2}\Big)\Big] + \text{const},$$

which is $-\frac{1}{2s_n^2}(\mu - m_n)^2 + \text{const}$ for exactly the s_n^2 and m_n of the statement. A density proportional to a Gaussian kernel is that Gaussian. □

Corollary 10.2 (One-step update)**.** *If arm i has the posterior $\mathcal{N}(\mu_{t-1}(i), \sigma_{t-1}^2(i))$ and is pulled, yielding r_t, its new posterior is $\mathcal{N}(\mu_t(i), \sigma_t^2(i))$ with*

$$\frac{1}{\sigma_t^2(i)} = \frac{1}{\sigma_{t-1}^2(i)} + \frac{1}{\sigma^2}, \qquad \mu_t(i) = \sigma_t^2(i)\Big(\frac{\mu_{t-1}(i)}{\sigma_{t-1}^2(i)} + \frac{r_t}{\sigma^2}\Big);$$

the posteriors of the other arms are unchanged.

Proof. Posterior updating can be done one observation at a time: the old posterior plays the role of the prior and Proposition 10.1 is applied with $n = 1$. Arms not pulled provide no data about their own mean, so their posteriors stay the same. □

This is the compact update shown on slide 40, $p(\mu_t(i), \sigma_t^2(i) \mid r_t) \propto p(\mu, \sigma^2)\,\mathcal{N}(r_t \mid \cdot)$: the slide writes the Gaussian likelihood of the new reward with the current posterior parameters as shorthand; the closed form above is what is actually computed.

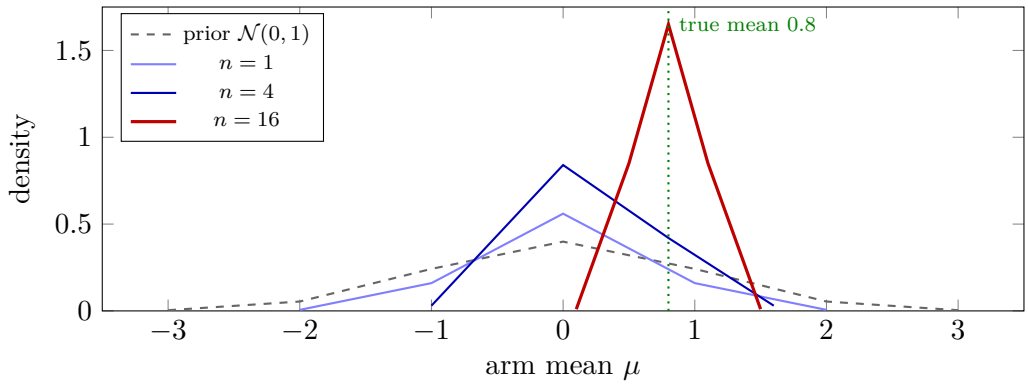


Figure 13: Toy simulation of Proposition 10.1 ($\sigma = 1$, true mean 0.8, seeded data): the posterior narrows like $1/\sqrt{n}$ (standard deviations 1, 0.71, 0.45, 0.24 for $n = 0, 1, 4, 16$) and its centre drifts from the prior mean 0 towards the truth (posterior means 0.00, 0.25, 0.66 for $n = 1, 4, 16$ in this run).

Intuition

Precisions (inverse variances) *add*: prior precision $1/s_0^2$ plus $1/\sigma^2$ per observation, and the posterior mean is a precision-weighted average of the prior mean and the data. Worked example: prior $\mathcal{N}(0,1)$, $\sigma^2 = 1$, and $n = 4$ observations summing to 3. Then $s_4^2 = 1/(1+4) = 0.2$ and $m_4 = 0.2 \cdot (0+3) = 0.6$: the sample mean 0.75 is pulled slightly toward the prior mean 0 (like having one extra "virtual" observation equal to 0). The more data an arm has, the less the prior matters, and a rarely tried arm keeps a wide posterior, which is exactly what the exploration rules below exploit.

11 Gaussian Bayesian Bandits: UCB *(slides 41–43)*

Since we are modelling a distribution, we already have a measure of confidence. What is confidence for a Gaussian? Its *standard deviation*. The slide proposes a UCB rule built from it:

Algorithm 11.1 (Bayesian UCB (Gaussian))**.** At round t play

$$a_t = \arg\max_{i \in [K]} \left(\mu_t(i) + \frac{c\,\sigma_t(i)}{\sqrt{N_t(i)}} \right),$$

where $\mu_t(i)$ is the posterior mean, $N_t(i)$ the number of pulls of arm i, $\sigma_t(i)$ the Gaussian's standard deviation parameter and $c > 0$ a constant that sets how optimistic we are.

The slide's wording "select the action with highest standard deviation" is shorthand: the score is the mean plus c times the uncertainty of the mean, $\sigma_t(i)/\sqrt{N_t(i)}$, which is the standard error of the arm's mean (with a weak prior it coincides with the posterior standard deviation from Proposition 10.1).

Intuition

Compare with Algorithm 1.8: the distribution-free width $\sqrt{\ln(KT/\delta)/N}$ is replaced by $c\sigma/\sqrt{N}$, with c playing the role of $\sqrt{\ln(KT/\delta)}$. Under a Gaussian belief, $c = 2$ means that the true mean lies below the score with probability $\Phi(2) \approx 97.7\%$. In the snapshot (means $0.55, 0.10, 0.70$; counts $25, 4, 100$; $c = 2$) the scores are 0.95, 1.10, 0.90: the well-explored best-looking arm is passed over in favour of the least-explored one, as optimism demands. Slide 43 shows an animation of the rule: as an arm is pulled its bonus shrinks, so the scores converge towards the true means and the choice settles on the best arm.

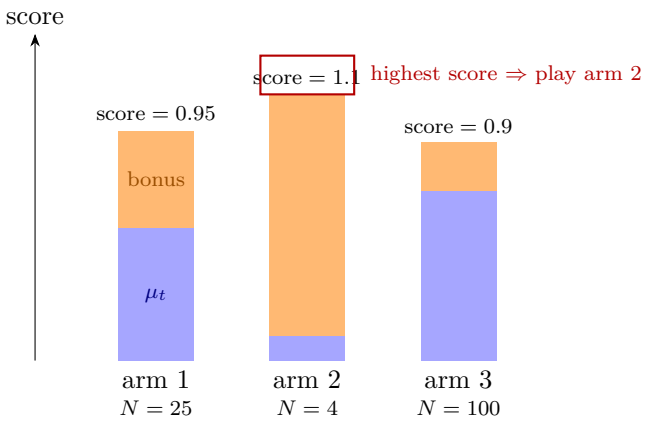


Figure 14: Snapshot of Algorithm 11.1 with $c = 2$, $\sigma = 1$ (illustrative numbers). Blue: posterior mean; orange: bonus $c\sigma/\sqrt{N}$. Arm 2 has the *lowest* mean but is almost unexplored, so its bonus (1.0) lifts it to the top.

12 Gaussian Bayesian Bandits: Thompson Sampling *(slides 44–48)*

12.1 Probability matching *(slides 44–45)*

Thompson sampling (TS) is a way to do *probability matching*: it selects an action with probability equal (or proportional) to the probability that the action is optimal given what we know, i.e. its *posterior probability of being optimal*. It is optimistic in the face of uncertainty, because uncertain actions have a higher probability of attaining the maximum, and it *uses sampling to avoid the complications of actual probability matching*.

Slide example. You are choosing between two ads. Each step you draw a plausible click-through rate for A and for B from your current beliefs, show the ad with the higher draw, observe the result and update your beliefs; the truly better ad is picked more and more over time.

Algorithm 12.1 (Thompson sampling (posterior sampling))**.** For $t = 0, 1, \ldots, T-1$:

1. for each arm a, draw $\tilde{\mu}_t(a)$ *independently* from the current posterior of $\mu(a)$;
2. play $A_t \in \arg\max_a \tilde{\mu}_t(a)$;
3. observe r_t and update the posterior of arm A_t only.

In the contextual case the posterior is over the parameters of the *reward model*: step 1 draws one model, and step 2 acts greedily with respect to that draw.

Proposition 12.2 (Thompson sampling is probability matching)**.** *Assume the arms' means are independent under the posterior given the history h_t. Then for every arm a (ties broken by the same rule on both sides)*

$$\mathbb{P}(A_t = a \mid h_t) = \mathbb{P}(a \in \arg\max_{a'} \mu(a') \mid h_t),$$

i.e. the algorithm plays each arm exactly as often as it believes that arm is optimal.

Proof. By independence, drawing $\tilde{\mu}_t(a)$ from each arm's marginal posterior independently is a draw of the whole vector $\tilde{\mu}_t$ from the joint posterior of $(\mu(1), \ldots, \mu(K))$ given h_t. The event $\{A_t = a\}$ is the event $\{a \in \arg\max_{a'} \tilde{\mu}_t(a')\}$, whose probability under the joint posterior is by definition the posterior probability that a is an optimal arm. □

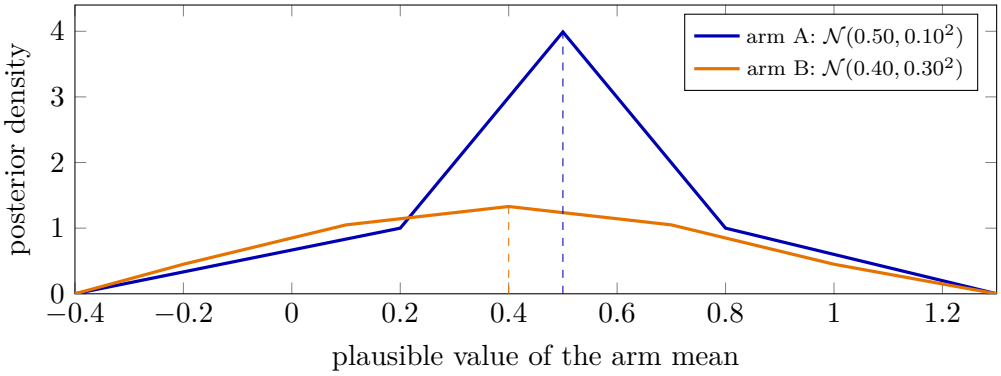


Figure 15: Two posteriors with independent arms. A has the higher mean but B is much more uncertain; B's posterior probability of being the better arm is $\Phi\big(-0.1/\sqrt{0.1^2+0.3^2}\big) \approx 0.376$, so TS plays B about 37.6% of the time.

Intuition

Why "sample, then argmax" (slide 45) is enough: you never compute $\mathbb{P}(\text{arm } a \text{ is best})$ explicitly; you just *simulate one possible world* consistent with your beliefs and act optimally in it. Over many rounds the frequency with which each arm wins its simulated world is precisely its probability of being optimal. In the figure, for independent Gaussians $\mathbb{P}(\mu_B > \mu_A) = \Phi\big((m_B - m_A)/\sqrt{s_A^2 + s_B^2}\big) = \Phi(-0.316) \approx 0.376$ (a Monte Carlo check with 10^6 pairs of draws gives 0.375). The less-known arm keeps getting tried *in proportion to how plausibly it might be best*; once its posterior narrows below the other arm's, that probability collapses to zero and exploration stops by itself.

12.2 The Gaussian algorithm *(slides 46–47)*

Algorithm 12.3 (Thompson sampling, Gaussian arms)**.** For $t = 0, \dots, T$:

1. for each arm $i = 1, \dots, K$ sample a value $\tilde{\mu}_t(i)$ *independently* from $\mathcal{N}(\mu_{t-1}(i), \sigma^2_{t-1}(i))$ (the slides denote this sampled value by $\hat{\mathbf{r}}_i$; it is *not* the IPS vector of Section 5);
2. pull the arm $I_t = \arg\max_{i \in [K]} \tilde{\mu}_t(i)$;
3. observe the reward r_t;
4. update the posterior of the pulled arm via Corollary 10.2.

Slide 47 adds two remarks. First, the sampled value is an estimate of the *reward*; in a general MDP it can be replaced by the Q-function, i.e. we estimate a *distribution over* Q and sample from it. Second, "this can be done with different distributions as well" (see Section 13).

Intuition

Compared with Bayesian UCB, nothing is added to the mean: uncertainty enters *through the randomness of the sample*. A wide posterior occasionally produces a very high draw and so gets tried; a narrow posterior almost always draws close to its mean. This "randomised optimism" needs no tuning constant c, only a sensible prior.

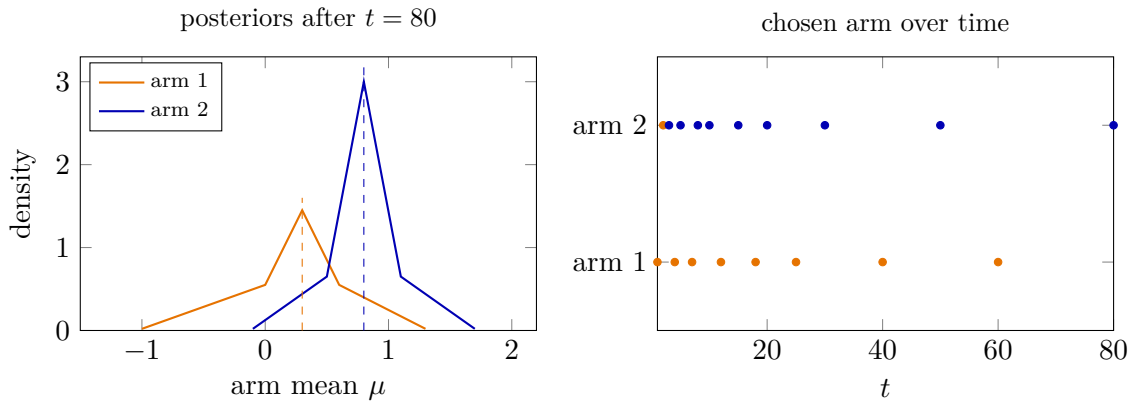


Figure 16: Toy simulation of Algorithm 12.3 (seeded): two Gaussian arms with true means 0.3 and 0.8 (dashed), noise $\sigma = 1$, prior $\mathcal{N}(0, 1)$, $T = 80$. Arm 1 is tried 13 times and then largely abandoned; arm 2 is played in 67 rounds. Final posteriors: arm 1 $\mathcal{N}(0.22, 0.27^2)$ (still wide, as it was rarely sampled), arm 2 $\mathcal{N}(0.69, 0.12^2)$ (narrow).

12.3 Simulation *(slides 48)*

13 Bayesian Bandits: Thompson Sampling *(slides 49)*

Slide 49 repeats the algorithm with *Beta posteriors* for an A/B ad test (click-through rates in $[0, 1]$). Each step: sample $\tilde\theta_A, \tilde\theta_B$ from the Beta posteriors, pick the argmax, observe a click, and update that arm only.

Proposition 13.1 (Beta–Bernoulli conjugacy)**.** *If $\mu \sim \mathrm{Beta}(\alpha, \beta)$ and $r \sim \mathrm{Bernoulli}(\mu)$, then $\mu \mid r \sim \mathrm{Beta}(\alpha + r,\ \beta + 1 - r)$. After k successes in n trials from the uniform prior $\mathrm{Beta}(1, 1)$, the posterior is $\mathrm{Beta}(1 + k,\ 1 + n - k)$, with mean $\frac{k+1}{n+2}$ and variance $\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$ (with $\alpha = 1 + k$, $\beta = 1 + n - k$).*

Proof. The Beta density is proportional to $\mu^{\alpha-1}(1-\mu)^{\beta-1}$ and the Bernoulli likelihood is $\mu^r(1-\mu)^{1-r}$. Their product is proportional to $\mu^{\alpha+r-1}(1-\mu)^{\beta+(1-r)-1}$, the kernel of $\mathrm{Beta}(\alpha+r, \beta+1-r)$. Iterating n times adds the successes to α and the failures to β; the moments are those of the Beta distribution. □

Algorithm 13.2 (Thompson sampling, Bernoulli rewards)**.** Keep $(\alpha_i, \beta_i) = (1, 1)$ for each arm i. At each round: draw $\tilde\theta_i \sim \mathrm{Beta}(\alpha_i, \beta_i)$ for all i, play $I_t = \arg\max_i \tilde\theta_i$, observe the click $c_t \in \{0, 1\}$, and set $\alpha_{I_t} \leftarrow \alpha_{I_t} + c_t$, $\beta_{I_t} \leftarrow \beta_{I_t} + 1 - c_t$.

Intuition

Beta–Bernoulli is the Bernoulli analogue of Proposition 10.1: "pseudo-counts" of successes and failures. For ad A ($k = 6$, $n = 10$): mean $7/12 \approx 0.583$, variance ≈ 0.0187. For ad B ($k = 1$, $n = 3$): mean $2/5 = 0.4$, variance 0.04. The probability that a draw for A beats a draw for B is ≈ 0.77 (Monte Carlo, 10^6 draws), so Thompson sampling would show A about 77% of the time and still B about 23% of the time, because B's few observations leave real doubt. Each click or non-click then moves the corresponding posterior and the shares drift towards the truly better ad. This is the "different distributions" remark of slides 47 and 49: the algorithm only needs a posterior one can *sample* from.

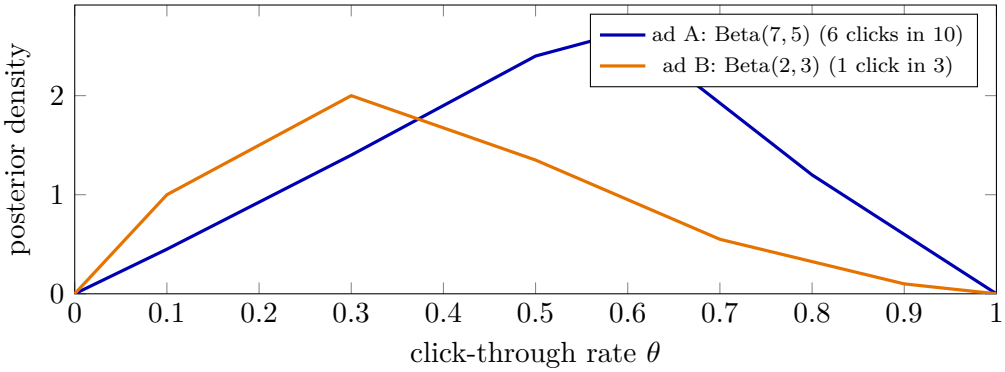


Figure 17: Beta posteriors for an A/B test from the uniform prior (illustrative counts). Ad B has little data, so its posterior is broad.

14 Summary

Method	Setting	How it explores	Assumes	Result in this lecture
Greedy	MAB	try each arm once, commit to the best sample	nothing	linear regret (about $0.08T$ in the Bernoulli example)
Explore-then-commit	MAB	N pulls per arm, then commit	rewards in $[0,1]$; needs T, K, δ to set N	$O(T^{2/3}K^{1/3}\ln^{1/3}(K/\delta))$ (Thm. 1.7)
UCB	MAB	optimism: mean $+\sqrt{\ln(KT/\delta)/N_t}$	rewards in $[0,1]$	$\widetilde{O}(\sqrt{KT})$, optimal up to logs (Thm. 1.9)
Explore-then-commit	CB	uniform actions + IPS for N rounds, then $\hat{\pi}$	finite Π	$O\big(T^{2/3}K^{2/3}\ln^{1/3}(\lvert\Pi\rvert/\delta)\big)$ as proved (Thm. 5.4); slide: $K^{1/3}$
ϵ-greedy	CB	mixture $(1-\epsilon_t)\delta_{\pi^t}+\epsilon_t\text{Unif}$ + IPS, $\epsilon_t \downarrow 0$	finite/given Π	no bound stated (simulation only)
Bayesian UCB	MAB (Gaussian)	posterior mean $+\,c\,\sigma_t/\sqrt{N_t}$	Gaussian prior and noise	none stated (rule and snapshot)
Thompson sampling	MAB / CB	sample from posterior, act greedily on the sample	sampleable posterior (Gaussian, Beta, ...)	plays arm a with prob. $\mathbb{P}(a \text{ optimal})$ (Prop. 12.2); no regret bound stated

Table 1: Comparison of the methods covered. "CB" = contextual bandit, "MAB" = multi-armed bandit. "None stated" means neither the slides nor the companion note give a bound.

Key take-aways

- **Exploration is unavoidable, and regret measures its price.** Greedy suffers linear regret; explore-then-commit balances exploration against commitment to get $T^{2/3}$; optimism (UCB) gets the optimal $\sqrt{T}$ rate up to logs, with no schedule to tune.
- **Context changes the object being learned, not the logic.** A contextual bandit asks for a *policy*, with i.i.d. contexts and bandit feedback; regret is measured against the best policy in Π, and the cost of a rich class is only $\ln|\Pi|$ (inside a cube root).

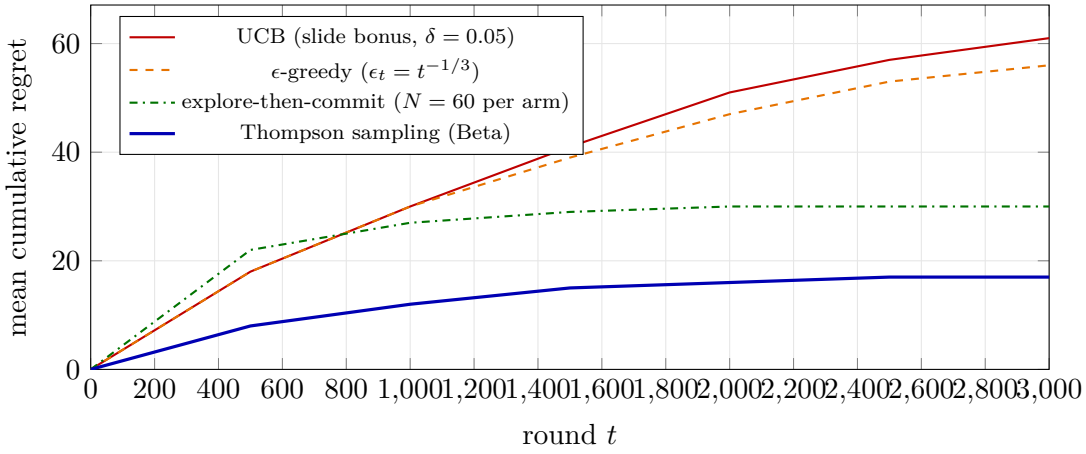


Figure 18: Toy simulation (qualitative): Bernoulli bandit with $K = 3$ arms of means $0.7, 0.5, 0.4$, $T = 3000$, regret averaged over 300 seeded runs. At $T = 3000$ the average regrets are about 17 (Thompson), 30 (explore-then-commit), 56 (ϵ-greedy) and 61 (UCB). Constants matter in a short horizon: UCB with the conservative slide bonus $\sqrt{\ln(KT/\delta)/N}$ explores a lot, and explore-then-commit works well here only because $N = 60$ was hand-picked (the theory needs the gaps, or T, K, δ and a bound, to choose it). Asymptotically the ordering of *rates* (linear $\gg T^{2/3} \gg \sqrt{T}$) is what the theorems describe.

- **IPS makes learning from partial feedback possible.** Dividing by the known propensity gives an unbiased estimate of every action's reward (Lemma 5.3), at the price of variance up to K (uniform exploration) or K/ϵ_t (ϵ-greedy), which is why exploration probabilities must stay positive.
- **Explore-then-commit separates the phases, ϵ-greedy interleaves them.** The first is easy to analyse (Theorem 5.4); the second keeps learning and adapting but needs a decay schedule for ϵ_t.
- **Bayesian bandits turn uncertainty into a distribution you can use.** Conjugate updates (Gaussian, Beta–Bernoulli) are one-line formulas; Bayesian UCB adds a multiple of the posterior width, while Thompson sampling samples from the posterior and thereby plays each arm with its probability of being optimal (Proposition 12.2).

A Supplementary material (in the companion note, not on the slides)

The following result is in the companion note but has no corresponding slide; it supplies the technical ingredient (range of the IPS terms) used in the proof of Theorem 5.4.

Lemma A.1 (What unbiasedness costs)**.** *Under uniform exploration,* $0 \le \hat{\mathbf{r}}_t[a] \le K$ *and* $\mathrm{Var}(\hat{\mathbf{r}}_t[a] \mid x_t) \le K$.

Proof. The estimator is $Kr_t \le K$ on one action and 0 elsewhere, which gives the range. For the variance, $\mathbb{E}[\hat{\mathbf{r}}_t[a]^2 \mid x_t] = \frac{1}{K} K^2 \mathbb{E}[r_t^2 \mid x_t, a] \le K$, since rewards lie in $[0, 1]$, and the variance is at most the second moment. □

Intuition

Unbiasedness is bought with noise: each observation is 0 with probability $1 - 1/K$ and Kr_t with probability $1/K$. With $K = 10$ actions and reward 0.5, a single IPS observation is either 0 or 5 although the target is 0.5; its variance is $10 \cdot 0.25 - 0.25 = 2.25$, versus at most 0.25

for a $[0,1]$-valued observation. The *range* bound K is what Hoeffding's inequality uses in Theorem 5.4; the *variance* bound K is what Bernstein's inequality would use to sharpen the dependence on K (see the remark after that theorem).

B Notation map and slide/note discrepancies

Concept	**Slides**	**Companion note / these notes**
context, context set	x_t, $\mathcal{X}$	z_t, $\mathcal{Z}$ / x_t, $\mathcal{X}$
context distribution	μ	ρ
action at round t	a_t (CB), I_t (MAB)	A_t / a_t, I_t
policy at round t	π^t	π_t
regret	Regret_T	$\mathrm{Reg}(T)$
IPS vector	$\hat{\mathbf{r}}_t[a]$	$\hat{r}_t[a]$ / $\hat{\mathbf{r}}_t[a]$

Small inconsistencies noticed while merging the two sources, and how these notes treat them:

- **Slide 8:** "Set $N = T/K$" would use all T rounds for exploration; read it as a placeholder with $NK \ll T$. The optimised $N^\star$ is on slide 12 and in Theorem 1.7.
- **Slide 12:** "approaches 0 as T goes to infinity" refers to the *average* regret $\mathrm{Regret}_T/T \sim T^{-1/3}$; the regret itself grows (sub-linearly).
- **Slides 11–12 and 14:** confidence widths are written with $\ln(K/\delta)$ or $\ln(KT/\delta)$, hiding the constant 2 and Hoeffding's factor $\frac{1}{2}$ inside $O(\cdot)$; the proofs here keep $\ln(2K/\delta)$ and $\ln(2KT/\delta)$.
- **Slide 31 vs. Theorem 5.4:** slide $K^{1/3}$, proved bound $K^{2/3}$; see the remark after the theorem.
- **Slide 42:** "highest standard deviation" should read "highest mean plus c standard errors", as the displayed formula says.
- **Slides 46–47:** the hat-bold-r symbol there denotes a *sample from the posterior*, unrelated to the IPS vector of slides 27–35.
- **Slide 40:** the recursive update is written with the likelihood evaluated at the current posterior parameters; the closed form is Corollary 10.2.

Part II

Lecture 2 – Part 2

Structure follows the slide deck (slides 1–55); formal statements and proofs are merged in from the companion PDF at the point where the slides introduce the topic. Material that appears only in the PDF is collected in the Appendix.

1 Recap *(slides 2–11)*

The lecture opens by recalling the bandit material of the previous lectures (slide 2 opens the recap, slide 11 closes it). The point of the recap is to set up the jump from *one-step* decision making to *sequential* decision making.

1.1 From Multi-Armed to Contextual Bandits *(slides 3)*

A multi-armed bandit (MAB) is stateless: the agent picks an action and receives a reward. A *contextual bandit* (CB) adds back some context (a state): before acting, the agent sees a context x_t, and the reward depends on both the context and the arm.

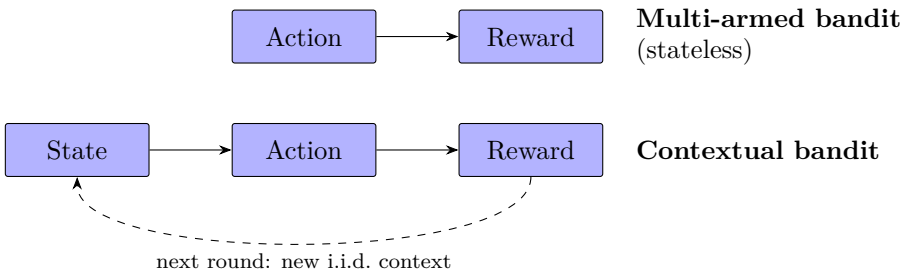


Figure 19: MAB versus contextual bandit (redrawn from slide 3).

Intuition

Think of a MAB as a single slot machine room with no information about who walks in; a CB is the same room where each customer has a profile x_t and the best machine depends on the profile. The crucial point for what follows: in a CB the context is drawn i.i.d., so *my action does not change the next context.* MDPs remove exactly this restriction.

1.2 Contextual Bandits: Interaction *(slides 4)*

For $t = 0, \dots, T-1$:

1. a new i.i.d. context $x_t \in \mathcal{X}$ appears;
2. select an action $a_t \in \mathcal{A}$ based on historical information and the context;
3. observe the reward $r(x_t, a_t)$ (context- and arm-dependent).

For simplicity the rewards are assumed deterministic, "as the context is the challenge here".

1.3 Contextual Bandits: Regret *(slides 5)*

The optimal policy is $\pi^\star = \arg\max_{\pi \in \Pi} \mathbb{E}_{x \sim \mu}[r(x, \pi(x))]$. At every iteration $a_t = \pi^t(x_t)$ is chosen, and the regret compares the total expected reward of always using $\pi^\star$ with that of our learned

sequence of policies:

$$\text{Regret}_T \;=\; T\,\mathbb{E}_{x\sim\mu}[r(x,\pi^\star(x))] \;-\; \sum_{t=0}^{T-1}\mathbb{E}_{x\sim\mu}\Big[r(x,\pi^t(x))\Big]\,.$$

Note that the policies π^t are different at every iteration t.

Intuition

Regret is "how much reward I left on the table by not knowing $\pi^\star$ from day one". If Regret_T grows sublinearly in T, the *average* regret per round goes to 0: eventually the learner behaves like the optimal policy.

1.4 Explore & Commit Algorithm *(slides 6)*

Algorithm 1.1 (Explore & Commit for contextual bandits)**.**

1. For $t = 0, \dots, N-1$ (*explore*): observe $x_t \sim \mu$; sample $a_t \sim \text{Unif}(\mathcal{A})$; observe $r_t = r(x_t, a_t)$; build, for x_t, an unbiased estimate $\hat{r}_t$ of the reward vector, with $\mathbb{E}_{a_t\sim\text{Unif}(\mathcal{A})}\hat{r}_t[a] = r(x_t,a)\ \forall a$:

$$\hat{r}_t[a] = \begin{cases} 0 & a \neq a_t \\ \dfrac{r_t}{1/|\mathcal{A}|} & a = a_t. \end{cases}$$

2. Compute the policy $\hat{\pi} = \arg\max_{\pi\in\Pi}\sum_{t=0}^{N-1}\hat{r}_t[\pi(x_t)]$.
3. For $t = N, \dots, T-1$ (*commit*): observe $x_t \sim \mu$ and play $a_t = \hat{\pi}(x_t)$.

With a suitable N the regret is

$$\text{Regret}_T = O\Big(T^{2/3}K^{1/3}\ln(|\Pi|)^{1/3}\Big)\,, \qquad K = |\mathcal{A}|.$$

Intuition

The division by $1/|\mathcal{A}|$ is *importance weighting*: we only see the reward of the arm we actually played, which happens with probability $1/|\mathcal{A}|$, so we inflate it by $|\mathcal{A}|$ to be right on average. A numerical feel for the rate: doubling the horizon multiplies the regret bound by only $2^{2/3} \approx 1.59$, so the per-round regret decays like $T^{-1/3}$.

1.5 ϵ-Greedy *(slides 7)*

Instead of fixing a threshold for exploring and then committing, interleave exploration and exploitation. For $t = 0, \dots, T$: observe $x_t \sim \mu$; draw

$$a_t \sim p_t = (1-\epsilon)\,\delta(\pi^t(x_t)) + \epsilon\,\text{Unif}(\mathcal{A});$$

observe $r_t = r(x_t, a_t)$; build an unbiased estimate $\hat{r}_t$ for x_t (so that $\mathbb{E}_{a\sim p_t}\hat{r}_t[a] = r(x_t,a)$); update the policy $\pi^{t+1} = \arg\max_{\pi\in\Pi}\sum_{\tau\le t}\hat{r}_\tau[\pi(x_\tau)]$. Here $\epsilon = 0$ means pure exploitation and $\epsilon = 1$ uniform exploration. (The slide only asserts unbiasedness; the usual choice is the importance-weighted estimator $\hat{r}_t[a] = \mathbb{1}[a = a_t]\,r_t/p_t(a_t)$.)

Intuition

Explore & Commit explores first, then stops learning. ϵ-greedy never stops: with $\epsilon = 0.1$, about one round in ten is a random experiment, and the other nine use the current best guess. It is robust to the case where the early estimates were wrong.

1.6 Bayesian Bandits *(slides 8)*

So far no assumption was made about the reward distributions ν_i; only bounds on rewards were derived. In Bayesian bandits we (i) exploit *prior* knowledge of rewards, (ii) update a *posterior* distribution from historical information, and (iii) use the posterior to guide exploration, either through upper confidence bounds (Bayesian UCB) or probability matching (Thompson sampling).

1.7 Gaussian Bayesian Bandits: UCB *(slides 9)*

Now that a distribution is modelled, a notion of confidence comes for free: for Gaussians it is the standard deviation. The UCB rule picks

$$a_t = \arg\max_{i \in K} \; \mu_t(i) + \frac{c\,\sigma_t(i)}{\sqrt{N_t(i)}},$$

where $N_t(i)$ is the number of pulls of arm i and c a constant.

1.8 Gaussian Bayesian Bandits: Thompson Sampling *(slides 10)*

For $t = 0, \ldots, T$: for each arm $i = 1, \ldots, K$ sample $\tilde{r}_t(i)$ independently from $\mathcal{N}(\mu_{t-1}(i), \sigma^2_{t-1}(i))$; pull $I_t = \arg\max_i \tilde{r}_t(i)$; observe r_t; update the posterior $p(\mu_t(i), \sigma^2_t(i) \mid r_t)$. This can be done with other distributions as well. The slide stresses that this is an estimate of the *reward*: in more general MDPs it is replaced by the Q function, i.e. we estimate a distribution over Q.

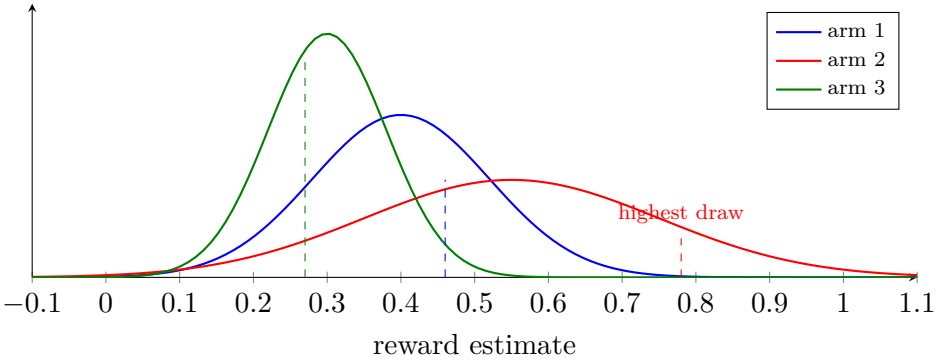


Figure 20: Thompson sampling with Gaussian posteriors (original illustration of the idea on slide 10; the numbers are illustrative). Dashed lines are the independent draws $\tilde{r}(i)$; the uncertain arm 2 gets the highest draw and is pulled.

Intuition

Thompson sampling explores in proportion to uncertainty without any explicit exploration parameter: an arm with a wide posterior occasionally produces a high draw and gets tried; once its posterior narrows and its mean is low, the draws stop beating the others. This is the "probability matching" idea: pull each arm with the probability that it is the best.

2 Sequential Decision Making *(slides 12–14)*

The agent interacts with the environment at discrete time steps, by receiving an observation o_t and a reward r_t from the environment and by taking actions a_t (slide 12).

Such a discrete interaction generates a trajectory, or *history*, at each time step t, which the agent uses to choose its action (slide 13):

$$h_t = (o_0, a_0, r_1, o_1, a_1, \ldots, r_t, o_t, a_t) \quad \text{(indexing convention as on the slide)}.$$

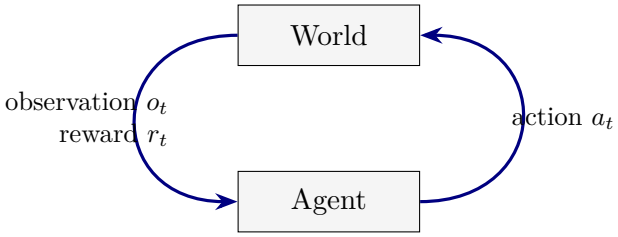


Figure 21: The agent–world loop (slides 12–14, redrawn).

The *state* is a function of the history, $s_t = f(h_t)$, and it is typically hidden or unknown (slide 14).

Intuition

The history grows without bound; no practical agent can condition on all of it. The state is a *summary* of the history that is supposed to keep what matters for the future. Which summary is a good one is exactly the question the Markov assumption answers.

3 Markov Assumption *(slides 15–17)*

Definition 3.1 (Markov state)**.** A state representation is Markov if for every t, every history h_t and every action a_t,

$$p(s_{t+1} \mid s_t, a_t) = p(s_{t+1} \mid h_t, a_t).$$

The future is conditionally independent of the past given the present.

Intuition

If I tell you the current state and my action, nothing in the earlier history can sharpen your prediction of the next state. A chess position is Markov (how we reached it is irrelevant for what can happen next). The position of a moving ball alone is *not*: without its velocity, the past positions carry extra information.



Figure 22: Is this problem Markovian?

Slide 16 asks, over a picture of a robot in a corridor with two pillars, "Is this problem Markovian?" The answer depends on what we put in the state: if the state is only the robot's position but the dynamics depend on, e.g., its velocity or an unobserved internal variable, past observations help predict the future and the state is *not* Markov; adding the missing variables restores the property.

Slide 17 gives the two extremes:

- a state can *always* be made Markov by taking it equal to the whole history, $s_t = h_t$;
- the best case, used in practice, is that the state is (a sufficient statistic of) the latest

observation, $s_t = o_t$. The state is then said to be *fully observable*.

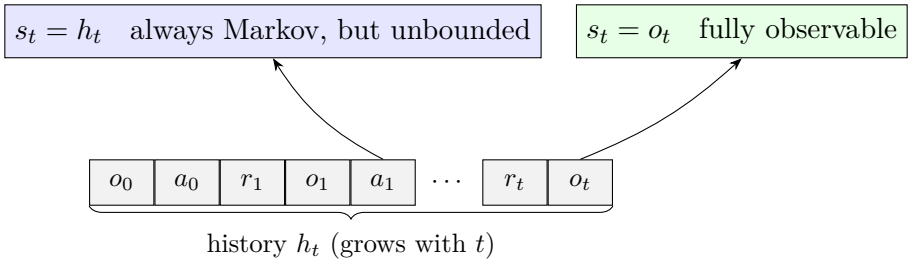


Figure 23: Two extreme choices of state as a function of the history (original illustration of slide 17).

Intuition

Choosing $s_t = h_t$ is a cheat: it is trivially Markov, but the state space explodes with time. Practical RL wants a small state that is still Markov. Fully observable means the observation already contains everything relevant; this is the setting of the rest of the lecture.

4 Markov Decision Process (MDP) *(slides 18–19)*

An MDP is sequential decision making under the Markov assumption: Markovian transition dynamics, full observability, and a (generally) stochastic transition kernel $p(s_{t+1} \mid s_t, a_t)$. Alternative notation: $s_{t+1} \sim p(\cdot \mid s_t, a_t)$ or $s' \sim p(\cdot \mid s, a)$ (slide 19).

Definition 4.1 (Markov decision process)**.** A discounted infinite-horizon MDP is a tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$: a set of states, a set of actions, transition dynamics $p(s' \mid s, a)$ satisfying Definition 3.1, a reward function $r : \mathcal{S} \times \mathcal{A} \to [0, R_{\max}]$, and a discount factor $\gamma \in [0, 1)$. Unless stated otherwise $\mathcal{S}$ and $\mathcal{A}$ are finite.

(The slides introduce γ later, in the "Infinite Horizon Discounted Setting" part; the tuple is stated here in full for reference.)

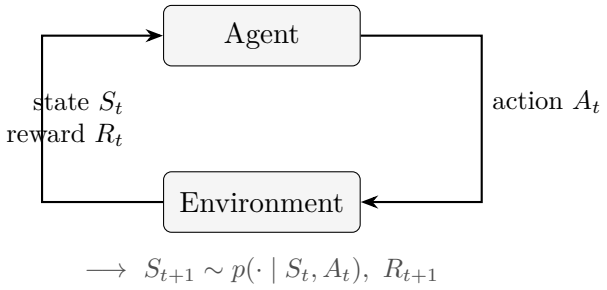


Figure 24: The MDP interaction loop (slides 18–19, redrawn).

Intuition

An MDP is the contextual bandit *with consequences*: the next "context" is the next state s', and it is drawn from a distribution that depends on the action just taken. Every action therefore has two effects: the reward now, and the situation in which the next decision is made.

5 Reward *(slides 20)*

A reward r_t is a scalar signal representing feedback and indicating how well the agent is doing at step t. It is a function of state and action, written $r(s, a)$ (sometimes $r(s', a, s)$). A *cost* is the negative of a reward.

Reward hypothesis (slide 20, flagged as an open question): can all goals be achieved through the maximisation of a numerical reward?

Intuition

The reward is the only way we tell the agent what we want; everything else (what is a good state, a good action) must be *derived* from it. That is why designing rewards is delicate: the agent maximises what we write, not what we meant.

6 Deterministic MDP Example *(slides 21)*

The slide shows a small deterministic MDP of a developer's day: states *Wake Up* ($r = 0$), *Netflix* ($r = -2$, with a self-loop), *Code & Debug* ($r = -3$), *Nap* ($r = 1$), *Deploy* ($r = 10$) and *Sleep* ($r = 3$, terminal).

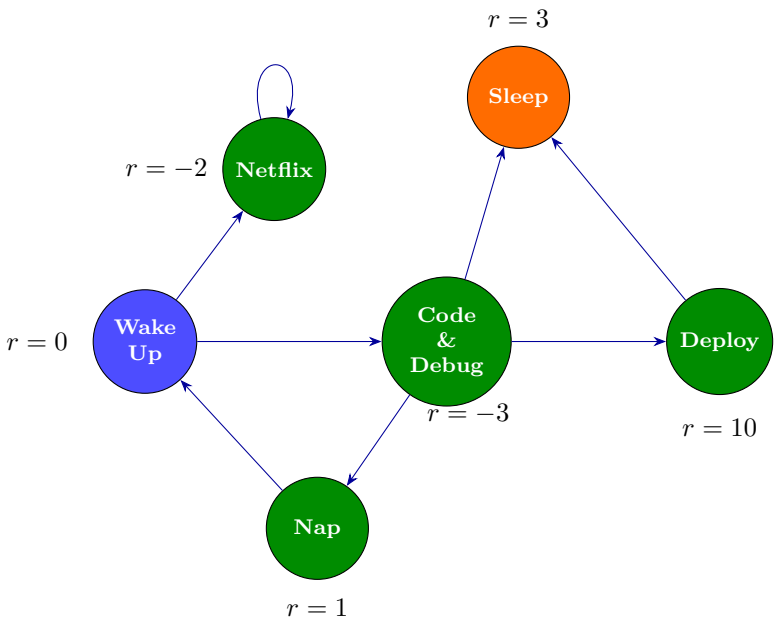


Figure 25: The deterministic MDP of slide 21, redrawn. Rewards are attached to states.

Example 6.1 (Worked numbers, not on the slide)**.** Take $\gamma = 0.9$, collect the reward of a state when leaving it, and treat Sleep as terminal (its reward is collected once). Then:

- $V(\text{Sleep}) = 3$, $V(\text{Deploy}) = 10 + 0.9 \cdot 3 = 12.7$, $V(\text{Netflix}) = -2/(1 - 0.9) = -20$ (it can only loop).
- From Code & Debug: $Q(\text{Code}, \text{Deploy}) = -3 + 0.9 \cdot 12.7 = 8.43$, $Q(\text{Code}, \text{Sleep}) = -3 + 0.9 \cdot 3 = -0.3$. The best choice is Deploy: a "bad" step (reward -3) pays off.
- Hence the path Wake→Code→Deploy→Sleep has discounted return $0 + 0.9 \cdot 8.43 = 7.587$, while Wake→Netflix forever has return $0 + 0.9 \cdot (-20) = -18$.
- Going Code→Nap→Wake gives $V(\text{Nap}) = 1 + 0.9 \cdot 7.587 \approx 7.83$ and $Q(\text{Code}, \text{Nap}) \approx 4.05 < 8.43$, so Nap loses.

Intuition

The immediate reward at Code & Debug is negative (-3), yet it is the right place to go through: all the value sits behind it (Deploy $= 10$). A myopic agent at Wake Up compares -2 (Netflix) with -3 (Code & Debug) and picks Netflix, where it is stuck forever with value -20; maximising the *return* sends it through Code & Debug instead. This is the entire reason we define value functions.

7 Stochastic MDP Example *(slides 22)*

The slide (taken from Sutton & Barto) shows the *recycling robot*: states `high` and `low` battery; actions `search`, `wait`, `recharge`. The transition table of the slide is:

s	a	s'	$p(s' \mid s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1-\alpha$	r_{search}
low	search	high	$1-\beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	–
low	wait	high	0	–
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	–

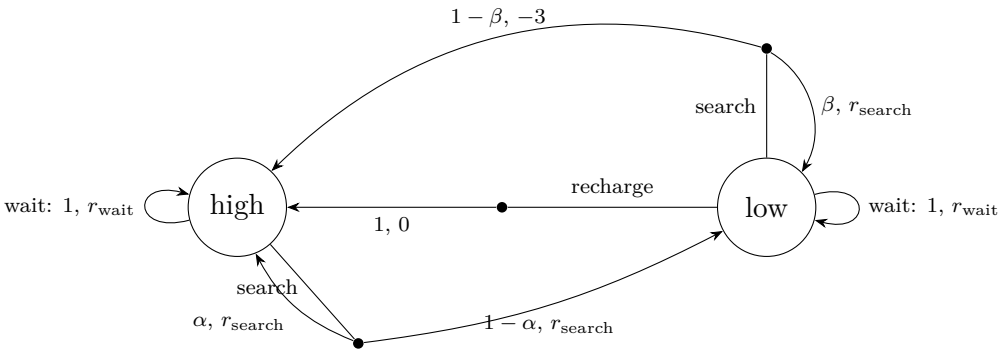


Figure 26: Transition graph of the recycling robot (slide 22, redrawn). Black dots are actions; edge labels are (probability, reward).

Example 7.1 (Worked numbers, not on the slide)**.** Let $\alpha = 0.7$, $\beta = 0.4$, $r_{\text{search}} = 2$, $r_{\text{wait}} = 1$. At `low`, the expected *immediate* reward of `search` is $0.4 \cdot 2 + 0.6 \cdot (-3) = -1.0$, versus $+1$ for `wait`. Searching with a low battery is risky: with probability 0.6 the robot runs flat, is rescued (reward -3) and ends up at `high`.

Intuition

Stochasticity means the same action from the same state can lead to different next states. Here a single decision ("search with a low battery") is a gamble, and the right choice depends on the state: at `high` searching is cheap, at `low` it is not. A policy is precisely a rule for making this state-dependent choice.

8 Policy *(slides 23)*

A policy π is a mapping from (all) states to actions that determines how the agent selects actions. It can be deterministic, $a = \pi(s)$, or stochastic, $\pi(a \mid s)$, $p(a \mid s)$ or $a \sim \pi(\cdot \mid s)$.

Intuition

A policy is a complete *strategy*, not a plan: it says what to do in *every* state, including the ones I might never reach. In the recycling robot there are $2 \cdot 3 = 6$ deterministic policies (two choices at `high`, three at `low`); in general there are $|\mathcal{A}|^{|\mathcal{S}|}$, which is why we will need structure (Bellman equations) rather than brute-force enumeration.

9 Trajectory Probability *(slides 24)*

What is the probability of seeing the trajectory $(s_0, a_0, s_1, a_1, \ldots, s_t, a_t)$ at time t when following π from s_0? Using the Markov property and the fact that π depends only on the current state:

$$\mathbb{P}^{\pi}(s_0, a_0, \ldots, s_t, a_t) = \pi(a_0 \mid s_0)\, p(s_1 \mid s_0, a_0)\, \pi(a_1 \mid s_1) \cdots p(s_t \mid s_{t-1}, a_{t-1})\, \pi(a_t \mid s_t).$$

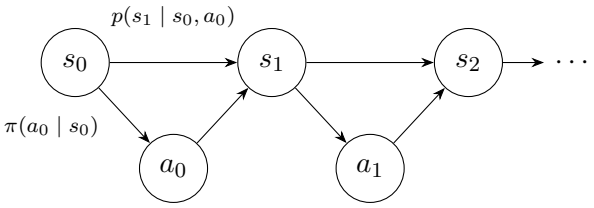


Figure 27: Graphical model of a trajectory (slide 24, redrawn).

Intuition

Each factor is one coin flip: the agent's flip (π) followed by the world's flip (p). Numbers: if $\pi(a_0|s_0) = 0.5$, $p(s_1|s_0, a_0) = 0.9$ and $\pi(a_1|s_1) = 1$, then the probability of the length-2 trajectory (s_0, a_0, s_1, a_1) is $0.5 \cdot 0.9 \cdot 1 = 0.45$.

10 State Visitation Probability *(slides 25–27)*

10.1 Definition *(slides 25)*

What is the probability of visiting (s, a) at time t when following π from s_0? Marginalise the trajectory probability over everything except the last pair:

$$\mathbb{P}^{\pi}_t(s, a; s_0) = \sum_{a_0, s_1, a_1, \ldots, s_{t-1}, a_{t-1}} \mathbb{P}^{\pi}(s_0, a_0, \ldots, s_t = s, a_t = a).$$

Intuition

It answers "if I run the policy many times, in what fraction of the runs am I in (s, a) at time t?" Plotting it for every state gives a heat map that moves and spreads with t.

10.2 Examples *(slides 26–27)*

Slide 26 shows, for a policy that goes to the goal in a gridworld with 10% slip, the visitation probability estimated over 2000 simulations, together with the probability mass in the (absorbing)

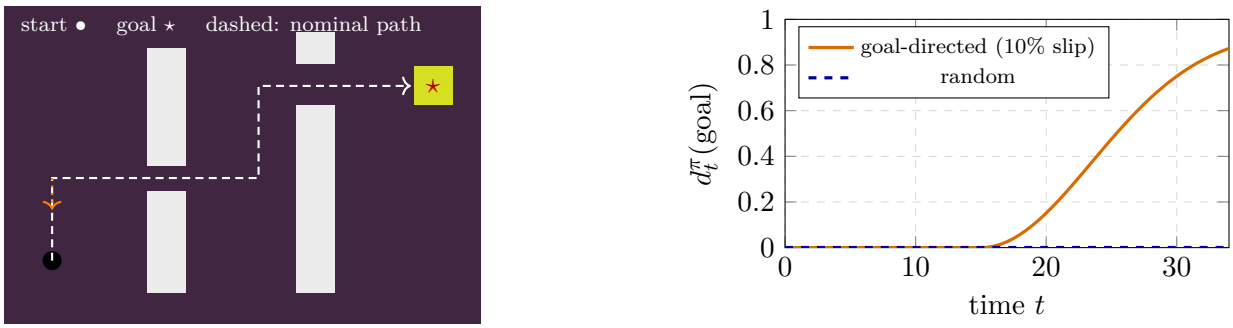


Figure 28: Schematic gridworld (left) and shape of the probability mass at the absorbing goal (right) for a goal-directed policy versus a random one. The curve is an original schematic read off slides 26–27, *not* the data of the slide.

goal as a function of t. Slide 27 shows the same for a *random* policy, on a log scale: the mass at the goal stays essentially 0.

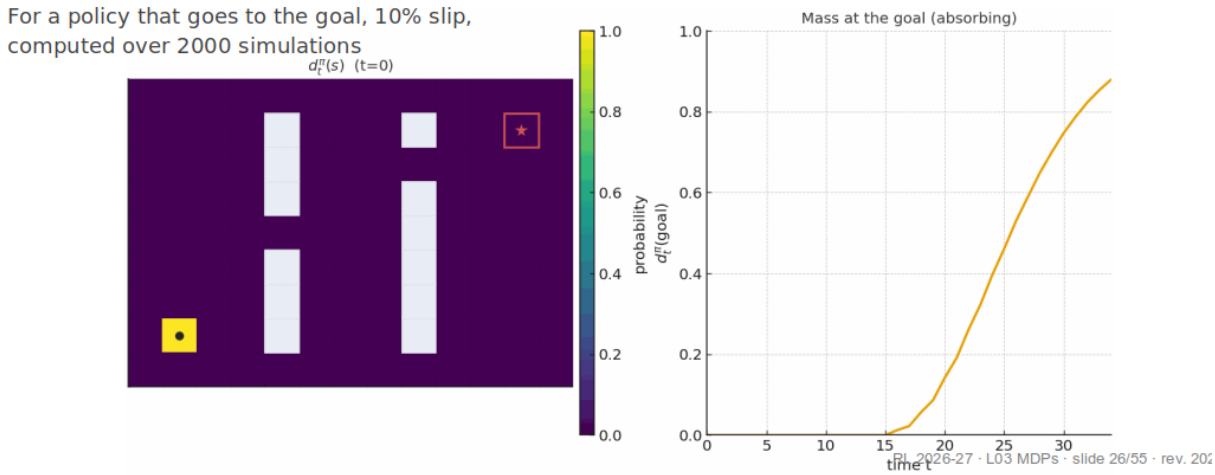


Figure 29: Heat map of $d_t^\pi(s)$ for the goal-directed policy (10% slip, 2000 simulations) with the "mass at the goal" curve.

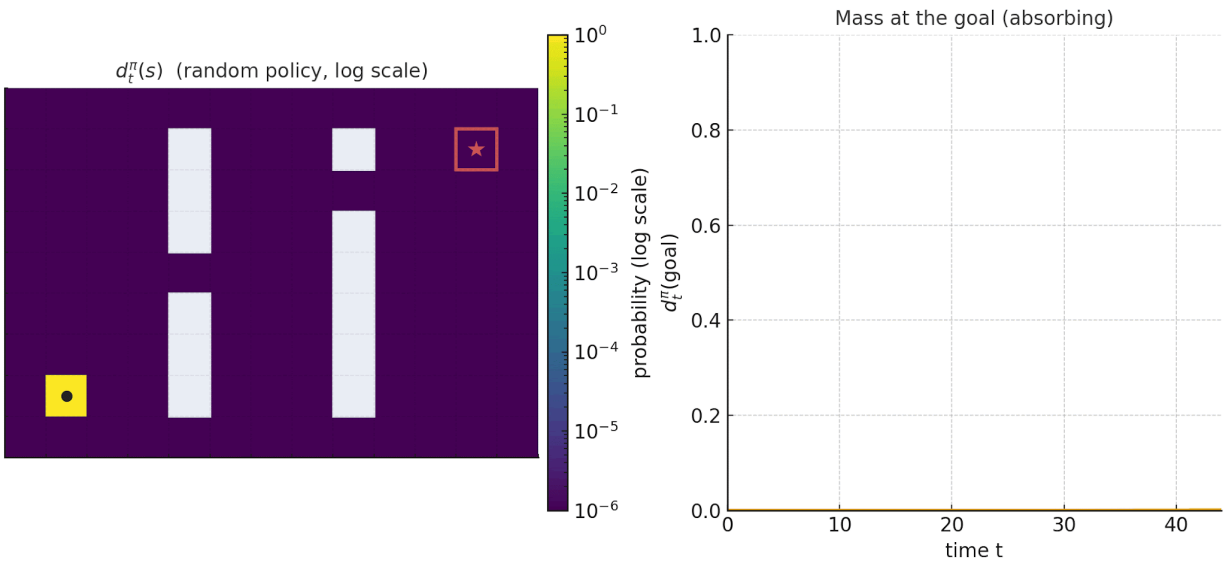


Figure 30: Same for the random policy, probability on log scale.

Intuition

Under the goal-directed policy nothing reaches the goal for the first ∼15 steps (that is the minimum distance), then the mass climbs towards 1; the missing mass is runs that slipped and are still on the way. The random policy diffuses over the grid so thinly that its goal mass is almost 0 at every time: the visitation distribution is a fingerprint of the policy's behaviour.

11 Another Example MDP *(slides 28)*

A robotic arm that must move a ball (slide 28):

- **state:** robot configuration (joint states) and ball position;
- **action:** torque on arm and finger joints;
- **transition:** stochastic, physics plus noise;
- **policy:** mapping from robot state and ball position to torque;
- **cost:** magnitude of the torque and distance to the goal.

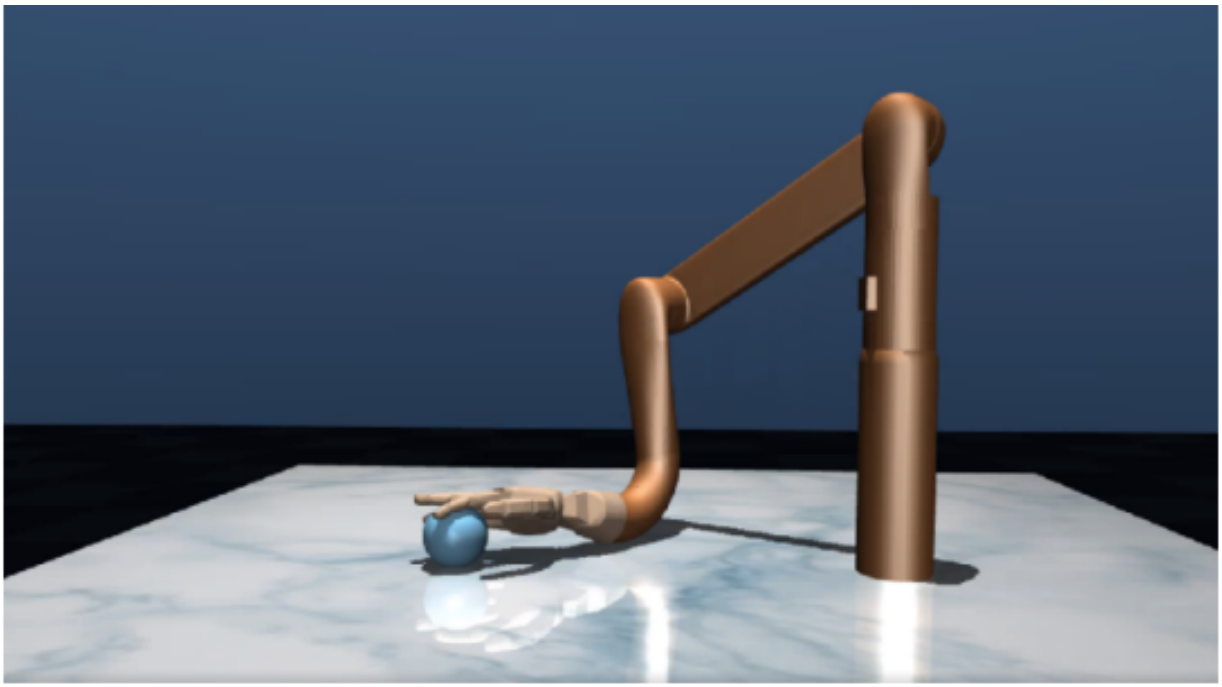


Figure 31: Photo of a robotic arm manipulating a ball

Intuition

Same five ingredients, a very different world: continuous states and actions, physics instead of a table of probabilities, and a cost made of two competing terms (effort and distance to goal). The abstract tuple $(\mathcal{S}, \mathcal{A}, p, r)$ covers both the toy table above and a robot.

12 Infinite Horizon Discounted Setting *(slides 29)*

So far the MDP is $(\mathcal{S}, \mathcal{A}, p, r)$. We now add a discount factor γ to reason about the long-term effects of a policy, with γ in $[0, 1]$ on the slide: $\gamma = 0$ means "I only care about immediate rewards", while $\gamma = 1$ means immediate and future rewards are equally important. The slide leaves "How so?" as a question; it is answered in "Back to Discount Factor" below, where we see that $\gamma = 1$ is a bad idea for infinite tasks and the formal definition (Definition 4.1) takes $\gamma < 1$.

Intuition

γ is a patience dial. At $\gamma = 0.5$ a reward two steps away is worth 0.25 of the same reward now; at $\gamma = 0.99$ it is worth 0.98.

13 Value Function *(slides 30–31)*

We estimate the *goodness* of states and actions by their value, which is also a measure to compare policies. The state-value function of a policy π is the expected discounted return from s (slide 30):

$$\begin{aligned} V^\pi(s_t) &= \mathbb{E}\Big[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \mid s_t\Big] \\ &= \mathbb{E}\left[\sum_{h=0}^{\infty} \gamma^h r_{t+h} \;\middle|\; s_t = s,\ a_h = \pi(s_h),\ s_{h+1} \sim p(\cdot \mid s_h, a_h)\right]. \end{aligned}$$

Slide 31 adds the action-value function, where the first action is fixed and then π is followed:

$$\begin{aligned} Q^\pi(s_t, a_t) = \mathbb{E}\Big[\sum_{h=0}^{\infty} \gamma^h r_{t+h} \;\Big|\; (s_t, a_t) = (s, a), \\ a_h = \pi(s_h),\ s_{h+1} \sim p(\cdot \mid s_h, a_h)\Big]. \end{aligned}$$

Intuition

$V^\pi(s)$: "how good is it to *be* in s if I then follow π?" $Q^\pi(s, a)$: "how good is it to take a *now* in s and then follow π?" The difference between the two is one free first decision. Comparing $Q^\pi(s, a)$ across a is how an agent decides which action is better than the policy's current choice.

14 Back to Discount Factor *(slides 32)*

Setting $\gamma = 1$ for infinite tasks is a bad idea. The quantity $\sum_{h=0}^{\infty} \gamma^h$ is a geometric series, equal to $1/(1-\gamma)$ for $\gamma \in [0, 1)$, so the value of γ approximately determines how many steps ahead we are looking: e.g. $\gamma = 0.99$ corresponds to $\sim$100 time steps.

Lemma 14.1 (Values are finite, and how finite)**.** *For every policy and every state,* $0 \le V^\pi(s) \le \dfrac{R_{\max}}{1-\gamma}$.

Proof. Rewards lie in $[0, R_{\max}]$, so $0 \le \sum_h \gamma^h r_h \le R_{\max} \sum_{h \ge 0} \gamma^h = R_{\max}/(1-\gamma)$, the geometric series converging because $\gamma < 1$. Expectations preserve the bounds. □

Intuition

With $\gamma = 0.99$ the weight γ^h falls to $\gamma^{100} \approx 0.37 \approx 1/e$ after about 100 steps; with $\gamma = 0.9$ the same happens after about 10 steps. If rewards are at most $R_{\max} = 1$, the value can never exceed $1/(1-0.99) = 100$; at $\gamma = 1$ the bound is infinite and the sum of rewards of an endless task need not exist at all. The discount is what makes "the value of a state" a well-defined finite number.

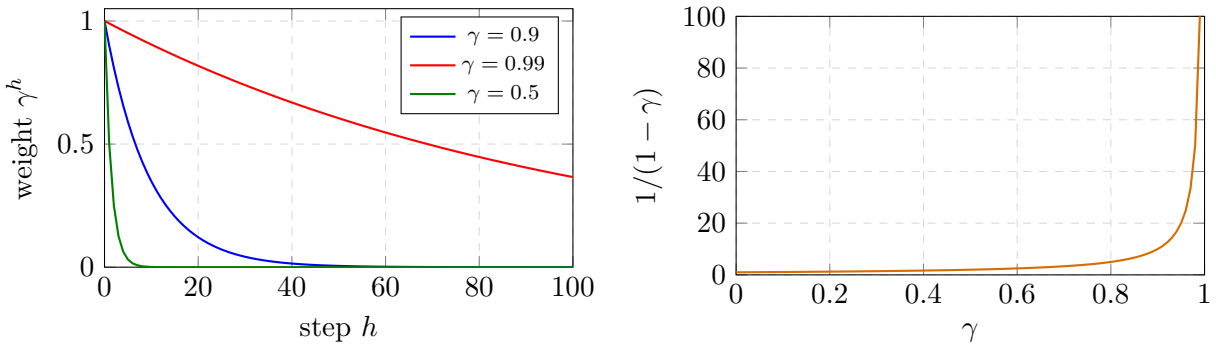


Figure 32: Left: discount weights γ^h. Right: the effective horizon $1/(1-\gamma)$ diverges as $\gamma \to 1$ (original figures for slide 32).

15 Bellman Equation *(slides 33–36)*

The value of a state is expanded in terms of the current reward and the value of the next states according to the policy (slide 33): for a deterministic π,

$$V^\pi(s_t) = \mathbb{E}_\pi\Big[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \mid s_t\Big] = r_t + \gamma\, \mathbb{E}_{s' \sim p(\cdot \mid s_t, \pi(s_t))}\big[V^\pi(s')\big]\,,$$

and likewise for Q (slides 34–36):

$$Q^\pi(s_t, a_t) = r_t + \gamma\, \mathbb{E}_{s' \sim p(\cdot \mid s_t, a_t)}\big[V^\pi(s')\big]\,.$$

In the first equation r is a function of s and $\pi(s)$; in the second, of s and a. As a result $V^\pi(s) = Q^\pi(s, \pi(s))$ (slide 36).

Theorem 15.1 (Bellman expectation equations)**.** *For every policy π and all s, a:*

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi(\cdot \mid s)}\Big[r(s,a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot \mid s,a)}\big[V^\pi(s')\big]\Big], \\ Q^\pi(s,a) &= r(s,a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot \mid s,a)}\big[V^\pi(s')\big], \\ V^\pi(s) &= \mathbb{E}_{a \sim \pi(\cdot \mid s)}\big[Q^\pi(s,a)\big]. \end{aligned}$$

For a deterministic policy the first reads $V^\pi(s) = r(s,\pi(s)) + \gamma \mathbb{E}_{s' \sim p(\cdot \mid s, \pi(s))}[V^\pi(s')]$ and the last $V^\pi(s) = Q^\pi(s, \pi(s))$.

Proof. Split the return after the first step using $G_0 = r(s_0, a_0) + \gamma G_1$: $V^\pi(s) = \mathbb{E}^\pi[G_0 \mid s_0 = s] = \mathbb{E}^\pi[r(s_0, a_0) \mid s_0 = s] + \gamma \mathbb{E}^\pi[G_1 \mid s_0 = s]$. Condition the second term on (a_0, s_1) and use the tower rule. By Definition 3.1 and the fact that π depends only on the current state, the conditional distribution of the trajectory from s_1 onwards given (s_0, a_0, s_1) is the same as that of a trajectory started at s_1; hence $\mathbb{E}^\pi[G_1 \mid s_0 = s, a_0 = a, s_1 = s'] = V^\pi(s')$. Averaging over $s' \sim p(\cdot \mid s, a)$ and then over $a \sim \pi(\cdot \mid s)$ gives the first identity; the argument for Q^π is the same with a_0 fixed. □

Intuition

The equation says: *value now = reward now + discounted value of where I land.* It turns an infinite sum into a one-step consistency condition. Example (deterministic): if $r(s, \pi(s)) = 1$, $\gamma = 0.9$ and the next state always has value $V^\pi(s') = 10$, then $V^\pi(s) = 1 + 0.9 \cdot 10 = 10$. The two lines of the theorem differ only in what is averaged: V averages over my action choice (policy), Q has the first action already fixed and averages only over the environment's randomness.

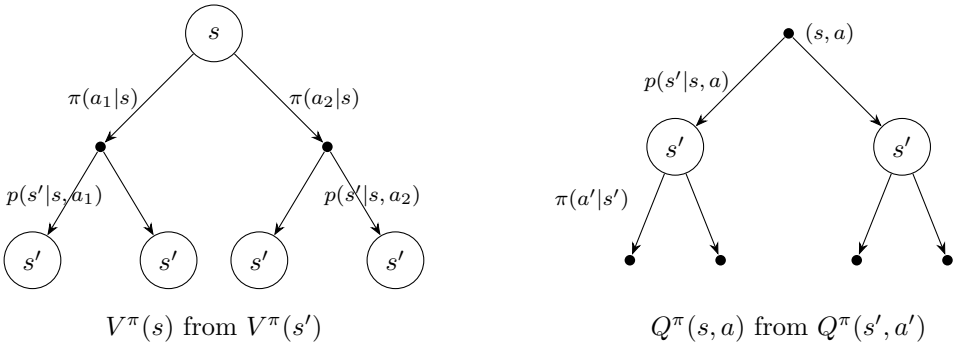


Figure 33: Backup diagrams for the Bellman expectation equations (original figure for slides 33–36).

16 Discounted State-Action Distribution *(slides 37–38)*

Definition 16.1 (Discounted state-action distribution)**.**

$$d^\pi_{s_0}(s,a) = (1-\gamma)\sum_{h=0}^{\infty}\gamma^h\,\mathbb{P}^\pi_h(s,a;s_0).$$

For an initial distribution μ write $d^\pi_\mu(s,a) = \mathbb{E}_{s_0\sim\mu}[d^\pi_{s_0}(s,a)]$.

Lemma 16.2 (It is a distribution, and that is the point of the $(1-\gamma)$)**.** *$d^\pi_{s_0}$ is a probability distribution over $\mathcal{S}\times\mathcal{A}$.*

Proof. Non-negativity is clear. Summing over (s,a) and exchanging the (absolutely convergent) sums, $\sum_{s,a} d^\pi_{s_0}(s,a) = (1-\gamma)\sum_h \gamma^h \sum_{s,a}\mathbb{P}^\pi_h(s,a;s_0) = (1-\gamma)\sum_h\gamma^h = 1$, since $\mathbb{P}^\pi_h(\cdot;s_0)$ is a distribution for each h. □

This matches the slide's remark: "this gives us a probability distribution (remember $\sum_h\gamma^h = 1/(1-\gamma)$)". The companion identity expressing discounted sums as expectations under $d^\pi_{s_0}$ is not on the slides and is given in Appendix A.1.

Intuition

Take the visitation probabilities of the previous section, weight time step h by γ^h and sum them: this is "how much total (discounted) time I spend in (s,a)". Without the factor $(1-\gamma)$ the total mass would be $1/(1-\gamma)$ (e.g. 100 for $\gamma = 0.99$); multiplying by $(1-\gamma)$ rescales it to 1 so it can be used as a genuine probability distribution. Early time steps get most of the weight, which is the same patience dial as before.

17 Optimal Policy *(slides 39–40)*

Definition 17.1 (Optimal value and optimal policy)**.** $V^\star(s) = \sup_\pi V^\pi(s)$ and $Q^\star(s,a) = \sup_\pi Q^\pi(s,a)$. A policy $\pi^\star$ is optimal if $V^{\pi^\star}(s) = V^\star(s)$ for every s.

Slides 39–40 state that for infinite-horizon MDPs there *always* exists a deterministic policy $\pi^\star$ such that

$$V^{\pi^\star}(s) \geq V^\pi(s) \qquad \forall s,\pi,$$

i.e. $\pi^\star$ dominates all other policies in each state; it returns optimal actions $a^\star$, and $V^{\pi^\star} = V^\star$, $Q^{\pi^\star} = Q^\star$. The existence statement is proved constructively by Theorem 13.1 below.

Intuition

It is far from obvious that one policy can be best *in all states simultaneously*; in many optimisation problems the best choice for one objective hurts another. Here it is true, and it is also true that we never need randomness: a table of "best action in each state" suffices. This is what makes the search for optimal behaviour tractable.

18 Bellman Optimality *(slides 41–42)*

Theorem 18.1 (Bellman optimality equation)**.**

$$V^\star(s) = \max_a \Big[r(s,a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)} V^\star(s') \Big],$$
$$Q^\star(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)} \Big[\max_{a'} Q^\star(s',a') \Big].$$

On slide 42 the bracket $r(s,a)+\gamma\mathbb{E}_{s'}V^\star(s')$ is annotated as $Q^\star(s,a)$, so that $V^\star(s) = \max_a Q^\star(s,a)$.

Intuition

The Bellman *expectation* equation averages over the policy's action; the Bellman *optimality* equation replaces that average by a max: the optimal agent simply takes the best action. All the difficulty of looking arbitrarily far into the future is hidden in $V^\star(s')$: if I knew the optimal value of every neighbour, choosing the best action would be a one-step problem.

19 Bellman Optimality Example *(slides 43–45)*

Assume we know $V^\star$ at s' and s'', the two successors of s reached by actions a' and a'' (slide 43). Try a', get $r(s,a')$ and compute $Q^\star(s,a') = r(s,a') + \gamma V^\star(s')$; try a'' and compute $Q^\star(s,a'') = r(s,a'') + \gamma V^\star(s'')$ (slide 44). Then (slide 45)

$$V^\star(s) = \max_{a',a''} \{Q^\star(s,a'),\, Q^\star(s,a'')\}.$$

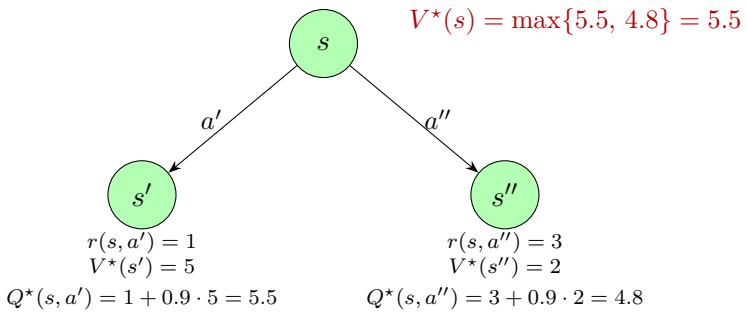


Figure 34: The one-step lookahead of slides 43–45 with illustrative numbers ($\gamma = 0.9$; the numbers are mine, not the slide's).

Intuition

With the numbers above the action with the *larger immediate reward* (a'', reward 3) is *not* the best: a' leads to a state worth 5, and $1 + 4.5 > 3 + 1.8$. This is the one-step trick that

everything else in the course rests on: evaluate each action as "reward + discounted value of where it leads" and take the best.

20 Bellman Optimality (Theorem 1) *(slides 46–49)*

Theorem 20.1 (Theorem 1 — greedy with respect to $Q^\star$ is optimal)**.** *Let* $\hat{\pi}(s) \in \arg\max_a Q^\star(s,a)$*. Then* $V^{\hat{\pi}} = V^\star$*, and therefore* $\pi^\star = \hat{\pi}$ *is an optimal policy.*

Proof as shown on the slides (slides 46–48)

Starting from $V^\star(s) = \max_a [r(s,a) + \gamma \mathbb{E}_{s'} V^\star(s')]$ and unrolling the equation along $\hat{\pi}$:

$$\begin{aligned} V^\star(s) &= r(s,\pi^\star(s)) + \gamma \mathbb{E}_{s'} V^\star(s') \\ &\le \max_a \Big[r(s,a) + \gamma \mathbb{E}_{s'} V^\star(s') \Big] = r(s,\hat{\pi}(s)) + \gamma \mathbb{E}_{s'} V^\star(s') \\ &\le r(s,\hat{\pi}(s)) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,\hat{\pi}(s))} \Big[r(s',\hat{\pi}(s')) + \gamma \mathbb{E}_{s''} V^\star(s'') \Big] \\ &\le \cdots \le \mathbb{E}\Big[r(s,\hat{\pi}(s)) + \gamma r(s',\hat{\pi}(s')) + \cdots \Big] = V^{\hat{\pi}}(s), \end{aligned}$$

so $V^{\hat{\pi}} \ge V^\star$ (the leftover term $\gamma^k \mathbb{E} V^\star(s_k) \to 0$ since $V^\star$ is bounded, Lemma 14.1); and $V^\star \ge V^{\hat{\pi}}$ holds by definition of $V^\star$ as a supremum. Hence $V^{\hat{\pi}} = V^\star$ and, by slide 49, $\pi^\star = \arg\max_a Q^\star(s,a)$ is an optimal policy.

Proof in the companion text (fixed-point argument)

Proof. By Theorem 18.1, $V^\star = \mathcal{B} V^\star$ and $Q^\star(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}[V^\star(s')]$ (the operators $\mathcal{B}, \mathcal{B}^{\hat{\pi}}$ are recalled in Appendix A.2). Since $\hat{\pi}(s)$ maximises $Q^\star(s,\cdot)$,

$$(\mathcal{B} V^\star)(s) = \max_a Q^\star(s,a) = Q^\star(s,\hat{\pi}(s)) = r(s,\hat{\pi}(s)) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,\hat{\pi}(s))}\big[V^\star(s')\big] = (\mathcal{B}^{\hat{\pi}} V^\star)(s),$$

so $V^\star$ is a fixed point of $\mathcal{B}^{\hat{\pi}}$. By Theorem 15.1 so is $V^{\hat{\pi}}$, and both are bounded by Lemma 14.1. A bounded fixed point of $\mathcal{B}^{\hat{\pi}}$ is unique. If u and v are two, then for every s

$$|u(s) - v(s)| = \gamma \Big| \mathbb{E}_{s'} \big[u(s') - v(s') \big] \Big| \le \gamma \|u - v\|_\infty,$$

because the rewards cancel, so $\|u - v\|_\infty \le \gamma \|u - v\|_\infty$. The norm is finite and $\gamma < 1$, hence $u = v$. Therefore $V^{\hat{\pi}} = V^\star$, and $\hat{\pi}$ attains the supremum of Definition 17.1 at every state. □

Intuition

Theorem 1 is the license to *act*: once someone hands me $Q^\star$, I do not need to plan any more; the single-step rule "pick the action with the largest $Q^\star(s,\cdot)$" is already globally optimal. The slide proof says it as a sandwich: following $\hat{\pi}$ can't be better than optimal ($V^{\hat{\pi}} \le V^\star$), and by unrolling the optimality equation it can't be worse either ($V^{\hat{\pi}} \ge V^\star$). The book proof says the same thing differently: $V^\star$ and $V^{\hat{\pi}}$ solve the same one-policy equation, and that equation has only one bounded solution because each application shrinks distances by $\gamma < 1$.

21 Bellman Optimality (Theorem 2) *(slides 50–55)*

Theorem 21.1 (Theorem 2 — the optimality equation has one solution)**.** *If $V : \mathcal{S} \to \mathbb{R}$ satisfies $V(s) = \max_a \left[r(s,a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)} V(s')\right]$ for every s, then $V = V^\star$.*

The slides (51–53) ask us to check that $\|V - V^\star\|_\infty = 0$ and then unroll the difference.

Proof. Both V and $V^\star$ satisfy the same equation, so for every s,

$$\begin{aligned} |V(s) - V^\star(s)| &= \Big| \max_a \left[r(s,a) + \gamma \mathbb{E}_{s'} V(s')\right] - \max_a \left[r(s,a) + \gamma \mathbb{E}_{s'} V^\star(s')\right] \Big| \\ &\le \max_a \Big| \left[r(s,a) + \gamma \mathbb{E}_{s'} V(s')\right] - \left[r(s,a) + \gamma \mathbb{E}_{s'} V^\star(s')\right] \Big| \\ &= \max_a \gamma \Big| \mathbb{E}_{s' \sim p(\cdot|s,a)} \left[V(s') - V^\star(s')\right] \Big| \\ &\le \gamma \max_a \mathbb{E}_{s' \sim p(\cdot|s,a)} |V(s') - V^\star(s')|. \end{aligned}$$

The first inequality is $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$; the last is Jensen. Applying the same bound to $|V(s') - V^\star(s')|$ and iterating k times,

$$|V(s) - V^\star(s)| \le \gamma^k \max_{a_0, \dots, a_{k-1}} \mathbb{E}_{s_k} |V(s_k) - V^\star(s_k)| \le \gamma^k \|V - V^\star\|_\infty,$$

and since $\gamma < 1$ and $\|V - V^\star\|_\infty$ is finite, the right-hand side tends to 0. Hence $|V(s) - V^\star(s)| = 0$ for every s. □

Slide 54 illustrates the shrinking bound $\gamma^h \|V - V^\star\|_\infty$ as the number of future steps h grows (its example uses $\gamma = 0.92$ and a baseline $\|V - V^\star\|_\infty = 1$).

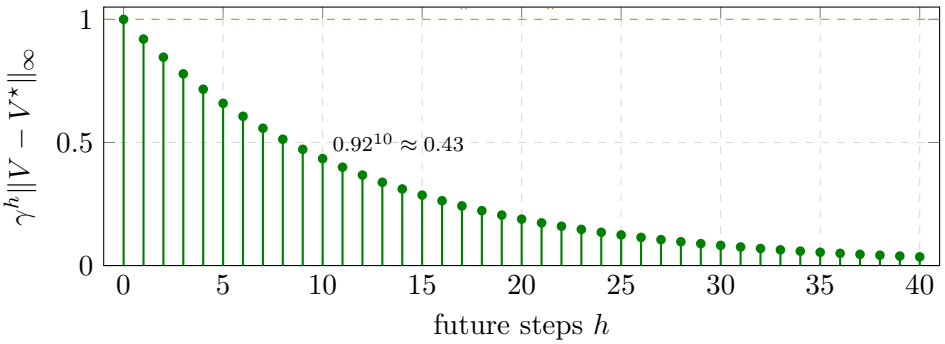


Figure 35: The bound $\gamma^h \|V - V^\star\|_\infty$ for $\gamma = 0.92$, redrawn from the idea of slide 54 (values computed here).

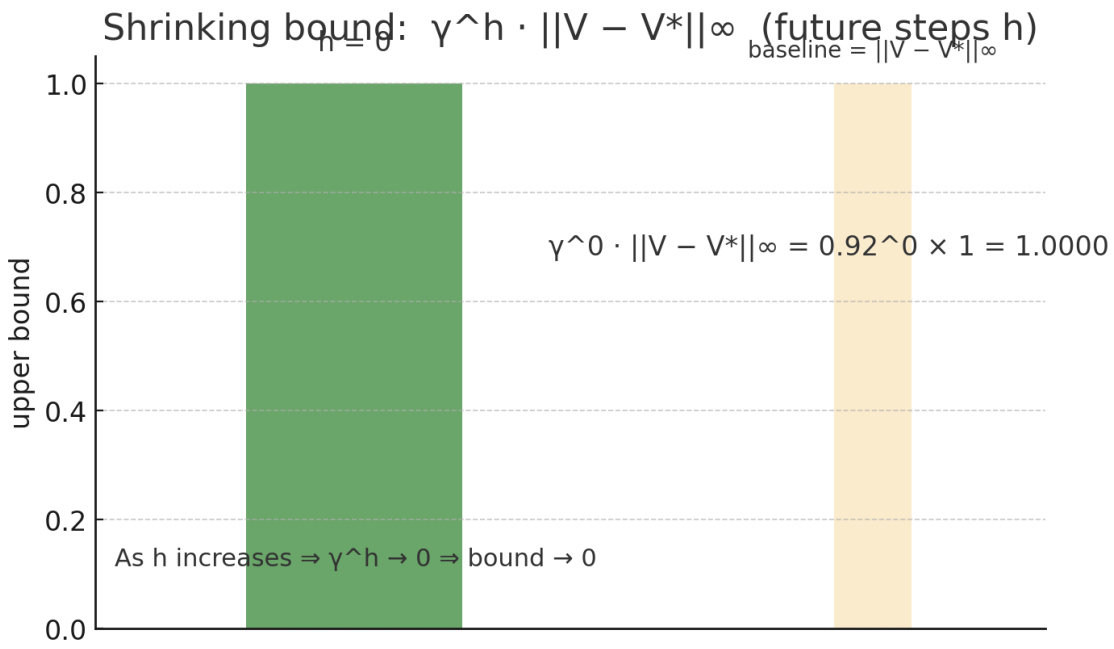


Figure 36: Shrinking bound

Intuition

Imagine a "fake" value function V that satisfies the optimality equation and is not $V^\star$. Each time we expand the equation one step, the gap can be at most γ times what it was; after k steps it is at most γ^k times the original gap, which melts away (with $\gamma = 0.92$, $0.92^{56} < 0.01$). The only way for the gap to survive all expansions is to be 0. Slide 55 states the consequence: *we can focus on one step at a time*, leaving the rest of the problem to $V(s')$, and any V satisfying that one-step formula is in fact $V^\star$.

22 Summary

Object / result	Formula	What it tells us	Slides
Markov state	$p(s_{t+1}\|s_t, a_t) = p(s_{t+1}\|h_t, a_t)$	the present summarises the past	15–17
MDP	$(\mathcal{S}, \mathcal{A}, p, r, \gamma)$	sequential decisions with consequences	18–29
V^π, Q^π	$\mathbb{E}[\sum_h \gamma^h r_{t+h} \mid \cdot]$	how good a state / action is under π	30–31
Finite values	$0 \le V^\pi \le R_{\max}/(1-\gamma)$	why $\gamma < 1$	32
Bellman expectation	$V^\pi(s) = \mathbb{E}_a[r + \gamma \mathbb{E}_{s'} V^\pi(s')]$	one-step consistency of a given policy	33–36
$d^\pi_{s_0}$	$(1-\gamma)\sum_h \gamma^h \mathbb{P}^\pi_h$	discounted occupancy, a true distribution	37–38
Optimal $V^\star, Q^\star$	$\sup_\pi V^\pi$, $\sup_\pi Q^\pi$	a deterministic policy dominates all others	39–40
Bellman optimality	$V^\star(s) = \max_a[r + \gamma \mathbb{E}_{s'} V^\star(s')]$	one-step lookahead with a max	41–45
Theorem 1	$\hat\pi \in \arg\max_a Q^\star \Rightarrow V^{\hat\pi} = V^\star$	greedy w.r.t. $Q^\star$ is optimal	46–49
Theorem 2	$V = \max_a[r + \gamma \mathbb{E} V] \Rightarrow V = V^\star$	$V^\star$ is the unique solution	50–55

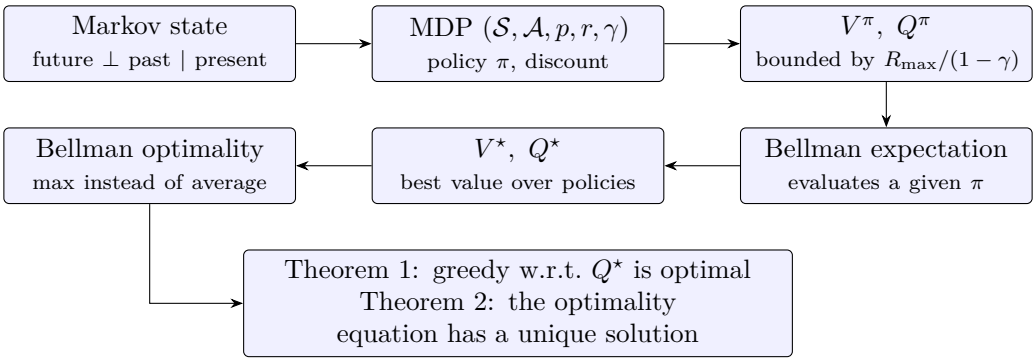


Figure 37: Roadmap of the lecture: from the Markov assumption to optimal policies.

Key take-aways

- An MDP is a contextual bandit whose next context depends on the action taken; the Markov property means the current state is all the agent needs to know.
- The discount factor $\gamma < 1$ keeps the value finite, with $0 \le V^\pi \le R_{\max}/(1-\gamma)$, and $1/(1-\gamma)$ is the effective horizon ($\gamma = 0.99 \Rightarrow \sim$100 steps).
- Bellman equations turn an infinite-horizon sum into a one-step relation: value now = reward + discounted value of the next state (average for a policy, max for optimality).
- Theorem 1: acting greedily with respect to $Q^\star$ is optimal, so a deterministic stationary policy always suffices.
- Theorem 2: $V^\star$ is the *only* solution of the Bellman optimality equation, thanks to the γ-contraction; any V that satisfies the one-step formula is $V^\star$.

A Appendix: supplementary material (in the companion PDF, not on the slides)

The following results appear in the companion PDF but have no corresponding slide; they are *not* part of the slide flow above.

A.1 Discounted sums as expectations under $d^\pi_{s_0}$

Lemma A.1 (Second part of Lemma 5 of the PDF)**.** *For every bounded f,*

$$\mathbb{E}^\pi\left[\sum_{h=0}^{\infty}\gamma^h f(s_h,a_h)\;\middle|\; s_0\right] = \frac{1}{1-\gamma}\,\mathbb{E}_{(s,a)\sim d^\pi_{s_0}}[f(s,a)].$$

Proof. It is the same exchange of (absolutely convergent) sums used in the proof of Lemma 16.2, applied to f: $\mathbb{E}^\pi[\sum_h \gamma^h f(s_h,a_h)\mid s_0] = \sum_h \gamma^h \sum_{s,a}\mathbb{P}^\pi_h(s,a;s_0)f(s,a) = \frac{1}{1-\gamma}\sum_{s,a} d^\pi_{s_0}(s,a)f(s,a)$. □

Intuition

Choosing $f = r$ gives $V^\pi(s_0) = \frac{1}{1-\gamma}\mathbb{E}_{(s,a)\sim d^\pi_{s_0}}[r(s,a)]$: the value is the average reward under the discounted occupancy, scaled by the effective horizon $1/(1-\gamma)$. For $\gamma = 0.9$, if the average reward under d^π is 0.5, the value is $0.5/0.1 = 5$.

A.2 Bellman operators (notation used in the PDF's proofs)

The proof of Theorem 13.1 in the companion text uses two operators acting on bounded functions $V : \mathcal{S} \to \mathbb{R}$:

$$(\mathcal{B}V)(s) = \max_a \Big[r(s,a) + \gamma \mathbb{E}_{s'\sim p(\cdot|s,a)}V(s')\Big],$$
$$(\mathcal{B}^\pi V)(s) = r(s,\pi(s)) + \gamma \mathbb{E}_{s'\sim p(\cdot|s,\pi(s))}V(s') \quad (\pi \text{ deterministic}).$$

With this notation Theorem 18.1 reads $V^\star = \mathcal{B}V^\star$, Theorem 15.1 reads $V^\pi = \mathcal{B}^\pi V^\pi$, and Theorem 21.1 says that $\mathcal{B}$ has a unique fixed point. The slides never name these operators.

Part III

Lecture 3

Abstract

This lecture answers the question left open by the MDP lectures: *how do we actually compute* V^π and, above all, the optimal policy π^*? We first compute V^π exactly (a linear system) and iteratively (a fixed-point iteration), introduce *contraction mappings* as the tool that makes iteration converge, show that both Bellman backups are γ-contractions, and conclude with **value iteration**, its convergence rate, the quality of the greedy policy it returns, and the number of iterations needed for an ϵ-optimal policy.

How these notes are organised

- **Sections follow the slide deck**, with the slide range in italics next to each title. Formal material from the companion note is merged where the slides introduce the topic.
- **Numbering.** Definitions, theorems and corollaries *1–8* and algorithms *A, B* keep the numbering of the companion note, so that its self-check list and proof videos map directly (see Appendix B). Items numbered *S, L, E, F* are not in the companion note: S/L = propositions/lemmas derived on the slides (or needed for completeness), E = worked examples, F = facts recalled from the previous lecture (L03).
- **Coloured boxes.** Definitions (green), theorems/corollaries (red), propositions/lemmas (orange), examples (teal), algorithms (violet), recalled facts (grey), and blue *Intuition* boxes that explain the "why" in plain words.
- **Running example.** A tiny two-state, two-action MDP (Example 3.2) is used for all worked numbers and for the data-driven plots.
- **Supplementary material** not covered on the slides (the existence/rate part of Banach's theorem) is in Appendix A, clearly labelled.

Throughout, rewards lie in $[0, 1]$ (so $R_{\max} = 1$), $\gamma \in [0, 1)$, $\cdot$ is the sup-norm, and ln is the natural logarithm (the slides write log for it).

1 Recap: Sequential Decision Making and MDPs *(slides 2–12)*

This block repeats what the previous lecture (L03) established. Everything later is built on it, and three of the results are quoted from L03 without proof (Facts F1–F4 below).

1.1 Sequential Decision Making and the MDP *(slides 3–4)*

The agent interacts with its environment at discrete timesteps: at each step it receives an observation o_t and a reward r_t from the environment and answers with an action a_t (fig:loop). The *state* is a function of the history, $s_t = f(h_t)$, and is typically hidden or unknown.

A **Markov Decision Process** is sequential decision making under the Markov assumption: the transition dynamics are Markovian, the state is fully observable, and the dynamics are (in general) stochastic, $p(s_{t+1} \mid s_t, a_t)$. Equivalent shorthand: $s_{t+1} \sim p(\cdot \mid s_t, a_t)$ or $s' \sim p(\cdot \mid s, a)$. An MDP is the tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ with set of states $\mathcal{S}$, set of actions $\mathcal{A}$, transition kernel p, reward $r(s, a)$ and discount γ.

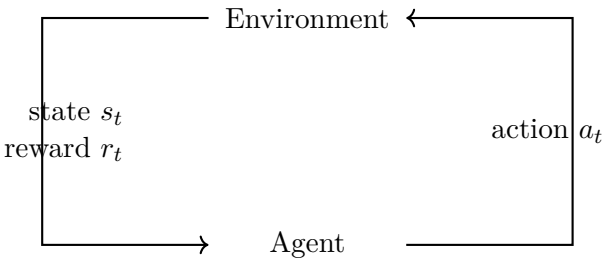


Figure 38: The agent–environment loop (redrawn from slides 3–4). The environment answers each action with a new state/observation and a reward.

Intuition

The Markov property says that *the present state is a sufficient summary of the past*: once you know s_t and a_t, the history tells you nothing more about where you will go next. That is why a policy can be a function of the current state alone, and why value functions can be defined state by state.

1.2 Policies, Value Functions and Q-Functions *(slides 5–6)*

A **policy** π is a mapping from (all) states to actions; it determines how the agent selects actions. It can be *deterministic*, $a = \pi(s)$, or *stochastic*, $\pi(a \mid s)$ (also written $p(a \mid s)$ or $a \sim \pi(\cdot \mid s)$).

The **value function** and **Q-function** of π measure how good states and actions are, and give a way to compare policies:

$$V^\pi(s) = \mathbb{E}_\pi\Big[\sum_{h\geq 0} \gamma^h r_{t+h} \;\Big|\; s_t = s\Big], \qquad Q^\pi(s,a) = \mathbb{E}_\pi\Big[\sum_{h\geq 0} \gamma^h r_{t+h} \;\Big|\; s_t = s,\, a_t = a\Big],$$

where after the first step actions follow π and next states are drawn from p. For infinite-horizon MDPs there always exists a *deterministic* policy π^* with $V^{\pi^*}(s) \geq V^\pi(s)$ for all s and all π: the optimal policy *dominates every other policy in every state*. We write $V^* = V^{\pi^*}$ and $Q^* = Q^{\pi^*}$.

Intuition

Comparing two policies means comparing two vectors $V^\pi, V^{\pi'} \in^{|\mathcal{S}|}$, which is only a *partial* order: π might be better in one state and worse in another. The nontrivial fact is that for

discounted infinite-horizon MDPs a single deterministic π^* is best *in every state at once*, so "find the optimal policy" is a well-posed goal.

1.3 The Discount Factor 7

Setting $\gamma = 1$ for infinite tasks is a bad idea: the sum of rewards may diverge. For $\gamma \in [0, 1)$ the weights form a geometric series,

$$\sum_{h=0}^{\infty} \gamma^h = \frac{1}{1-\gamma},$$

so the value of γ roughly determines *how many steps ahead we are looking*: $\gamma = 0.99$ corresponds to a horizon of about 100 steps (the slide rounds this to "99 timesteps ahead"). This also bounds all values:

[L03-1, Lemma 3 — range of values] If $r(s, a) \in [0, R_{\max}]$ then $0 \le V^\pi(s), V^*(s), Q^\pi(s, a), Q^*(s, a) \le R_{\max}/(1-\gamma)$. (Each is a discounted sum of terms in $[0, R_{\max}]$, bounded by $R_{\max} \sum_h \gamma^h$.) With $R_{\max} = 1$ this is the interval $[0, 1/(1-\gamma)]$ used on the slides.

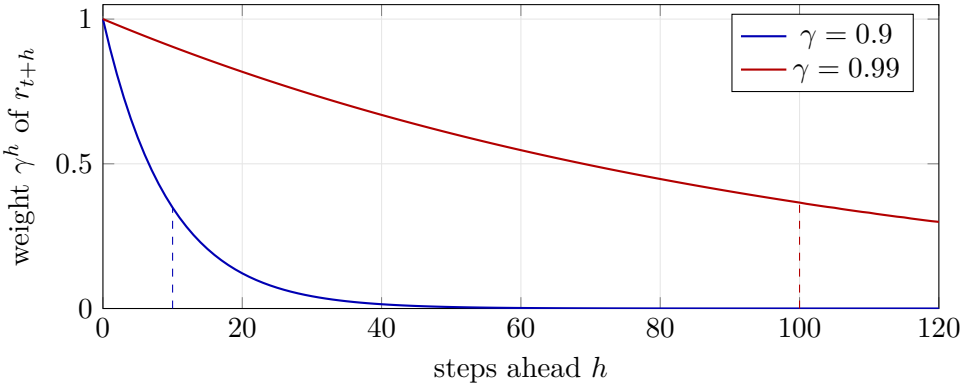


Figure 39: Weight of the reward received h steps ahead (original figure; slide 7 only describes this in prose). Dashed lines mark the effective horizon $1/(1-\gamma)$, where the weight has dropped to about $e^{-1} \approx 0.37$.

Intuition

Think of γ as a "patience" dial. With $\gamma = 0.9$ rewards more than ~ 10 steps away barely matter; with $\gamma = 0.99$ the agent looks ~ 100 steps ahead. The same number $1/(1-\gamma)$ reappears in this lecture as (i) the largest possible value, (ii) the initialisation range for the iterations, and (iii) the factor that inflates the error of the greedy policy. Larger γ means a longer horizon and harder computation.

1.4 Bellman Expectation Equation 8

The value of a state is expanded in terms of the current reward and the value of the next states, according to the policy:

$$V^\pi(s) = r(s, \pi(s)) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,\pi(s))}\big[V^\pi(s')\big],$$
$$Q^\pi(s, a) = r(s, a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)}\big[V^\pi(s')\big], \qquad V^\pi(s) = Q^\pi(s, \pi(s)).$$

In the first line the reward is a function of s and $\pi(s)$; in the second, of s and a.

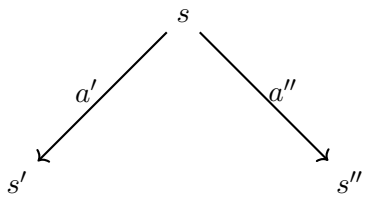
[L03-1, Theorem 6 — Bellman expectation equation] For every policy π and every state s, the display above holds. In operator language (Definition 6.2): $V^\pi = V^\pi$.

Intuition

A one-step lookahead: *what you get now* plus *the discounted value of wherever you land*, averaged over the randomness of the transition. It turns an infinite sum into a *local* consistency condition that must hold in all states simultaneously, and that is exactly what we will exploit to compute V^π.

1.5 Bellman Optimality *(slides 9–12)*

Example (slide 9). Suppose we already know V^* at the successor states s' and s''. From s we try each action: taking a' gives reward $r(s,a')$ and leads to s', so $Q^*(s,a') = r(s,a') + \gamma V^*(s')$; likewise $Q^*(s,a'') = r(s,a'') + \gamma V^*(s'')$. Then $V^*(s) = \max\{Q^*(s,a'), Q^*(s,a'')\}$ (fig:tree; the slide draws deterministic transitions, with stochastic ones $V^*(s')$ becomes an expectation).



$$Q^*(s,a') = r(s,a') + \gamma V^*(s')$$
$$Q^*(s,a'') = r(s,a'') + \gamma V^*(s'')$$

$$V^*(s) = \max\{\, Q^*(s,a'),\ Q^*(s,a'') \,\}$$

Figure 40: Bellman optimality backup in one picture (redrawn from slide 9): try every action, evaluate it with the (assumed known) values of the successors, keep the best.

In general,

$$V^*(s) = \max_a \Big[r(s,a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)} V^*(s') \Big], \qquad Q^*(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)} \Big[\max_{a'} Q^*(s',a') \Big].$$

Two results from L03 are quoted on the slides:

[Slide 10 "Theorem 1", L03-1 Theorem 8 — Bellman optimality] V^* and Q^* satisfy the two equations above, and the greedy policy $\hat\pi(s) \in \arg\max_a Q^*(s,a)$ satisfies $V^{\hat\pi} = V^*$. Hence $\pi^*(s) = \arg\max_a Q^*(s,a)$ is an optimal policy.

[Slide 11 "Theorem 2", L03-1 Theorem 10 — uniqueness] For any V, if $V(s) = \max_a [r(s,a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)} V(s')]$ for all s, then $V = V^*$.

Intuition

The optimality equation lets us *focus on one step at a time*: we only choose the best first action and leave the rest of the problem to $V(s')$. Fact 1.5 says that any function that is locally self-consistent in this sense *is* V^*: there is nothing else to look for. Fact 1.5 converts Q^* into a policy by a plain arg max. Together they say: *find the unique solution of the Bellman optimality equation and you are done.* How to find it is the subject of the rest of the lecture. (The proof of uniqueness in L03 was a long unrolling argument; Corollary 12.1 below re-proves it in one line.) 12 closes the recap.

2 Policy Evaluation *(slides 13–15)*

Question. Given an MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ and a policy π, how can we compute the goodness of π, i.e. V^π?

Why is this interesting? There are $|\mathcal{A}|^{|\mathcal{S}|}$ possible deterministic policies, and we want to find the optimal one. To find it, we need to be able to *evaluate* candidates.

Intuition

Policy evaluation is the "scoring function" of reinforcement learning: before you can say that one policy is better than another you need a number (really, a vector with one entry per state) that says how well each one does. Everything in this lecture is about computing such numbers cheaply: first for a *given* policy, then for the *best* policy at once.

3 Exact Policy Evaluation *(slides 16–22)*

Note: the companion note marks this block as "on the slides only": it is not proved there, so prepare it from the slides.

Given $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ and π, what is V^π? We know that the Bellman expectation equation holds for *all* states:

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, \pi(s))\, V^\pi(s').$$

How many linear constraints do we have? $|\mathcal{S}|$, one per state, in the $|\mathcal{S}|$ unknowns $V^\pi(s)$. We can combine all the constraints (slide credit: Wen Sun). Define the reward vector and the transition matrix induced by π:

$$r^\pi \in^{|\mathcal{S}|},\ (r^\pi)_s = r(s, \pi(s)); \qquad P^\pi \in^{|\mathcal{S}| \times |\mathcal{S}|},\ (P^\pi)_{s,s'} = p(s' \mid s, \pi(s)).$$

Then the $|\mathcal{S}|$ equations read $V^\pi = r^\pi + \gamma P^\pi V^\pi$ (fig:matrix), which we can solve for V^π.

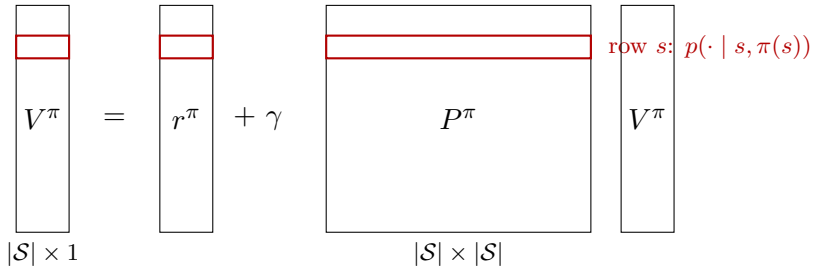


Figure 41: The $|\mathcal{S}|$ Bellman expectation equations stacked in matrix form (redrawn from slides 20–22). The highlighted row is the equation for one state s: $V^\pi(s) = r(s, \pi(s)) + \gamma \langle p(\cdot \mid s, \pi(s)), V^\pi \rangle$.

Proposition 3.1 (Exact policy evaluation, slides 21–22)**.** *For $\gamma < 1$, the matrix $I - \gamma P^\pi$ is invertible and*

$$V^\pi = (I - \gamma P^\pi)^{-1} r^\pi = \sum_{k \geq 0} \gamma^k (P^\pi)^k r^\pi.$$

Computing it by matrix inversion costs $O(|\mathcal{S}|^3)$.

Proof. P^π is row-stochastic (entries ≥ 0, each row sums to 1), so its induced ∞-norm is 1 and $\gamma P^\pi x \leq \gamma x$. If $(I - \gamma P^\pi)x = 0$ then $x = \gamma P^\pi x$, so $x \leq \gamma x$, forcing $x = 0$ since $\gamma < 1$; hence the matrix is invertible. The Neumann series $\sum_k (\gamma P^\pi)^k$ converges (its terms have norm $\leq \gamma^k$) and equals the inverse. Rearranging $V^\pi = r^\pi + \gamma P^\pi V^\pi$ gives $(I - \gamma P^\pi)V^\pi = r^\pi$. Inverting an $|\mathcal{S}| \times |\mathcal{S}|$ matrix takes $O(|\mathcal{S}|^3)$ operations. □

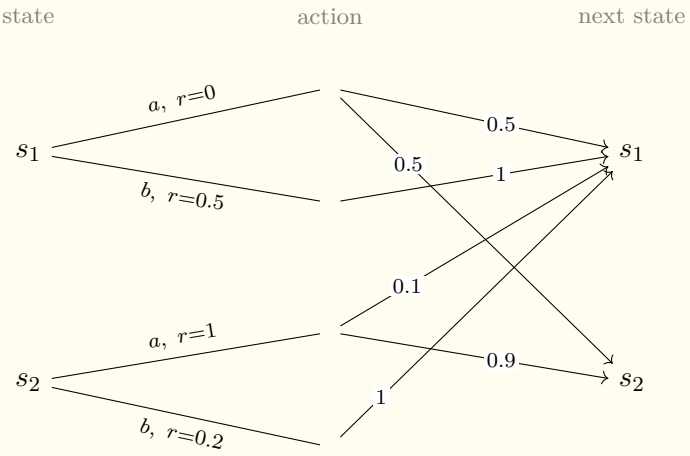
Intuition

The Neumann series is the Bellman equation unrolled: $(P^\pi)^k r^\pi$ is the expected reward k steps ahead, and γ^k discounts it, so V^π is literally the *expected discounted sum of rewards*. The closed form is "nice but computationally expensive" (the slide's own verdict): cubic in the number of states is fine for dozens of states but hopeless for millions. This motivates an

approach that only needs matrix–vector products.

Example 3.2 (A two-state MDP, used throughout)**.** Take $\mathcal{S} = \{s_1, s_2\}$, $\mathcal{A} = \{a, b\}$, $\gamma = 0.9$, with

(s, action)	r	$p(s_1 \mid \cdot)$	$p(s_2 \mid \cdot)$
(s_1, a)	0	0.5	0.5
(s_1, b)	0.5	1	0
(s_2, a)	1	0.1	0.9
(s_2, b)	0.2	1	0



Filled dots are state-action pairs; the arrows leaving a dot are the stochastic transitions, labelled with their probabilities. For the policy $\pi = (a, a)$ (always play a):

$$P^\pi = \begin{pmatrix} 0.5 & 0.5 \\ 0.1 & 0.9 \end{pmatrix}, \quad r^\pi = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad I - \gamma P^\pi = \begin{pmatrix} 0.55 & -0.45 \\ -0.09 & 0.19 \end{pmatrix}, \quad \det = 0.064,$$

$$V^\pi = (I - \gamma P^\pi)^{-1} r^\pi = \frac{1}{0.064} \begin{pmatrix} 0.19 & 0.45 \\ 0.09 & 0.55 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 7.03125 \\ 8.59375 \end{pmatrix}.$$

It will turn out that this policy is in fact optimal for this MDP: $V^* = (7.03125, 8.59375)$.

4 Fixed-Point Iteration *(slides 23–24)*

What is a **fixed point**? A point x such that $x = f(x)$. How can we find such points?

1. Initialise x_0.
2. Repeat $x_{i+1} = f(x_i)$.
3. Stop at convergence, when x does not change any more.

Geometrically (slide 24), fixed points are the intersections of $y = f(x)$ with the line $y = x$. The iteration is a "cobweb": from (x_i, x_i) on the diagonal go vertically to the curve at (x_i, x_{i+1}), then horizontally back to the diagonal at (x_{i+1}, x_{i+1}). The slide's example is $f(x) = \cos x$, with fixed point $x^* \approx 0.739085$.

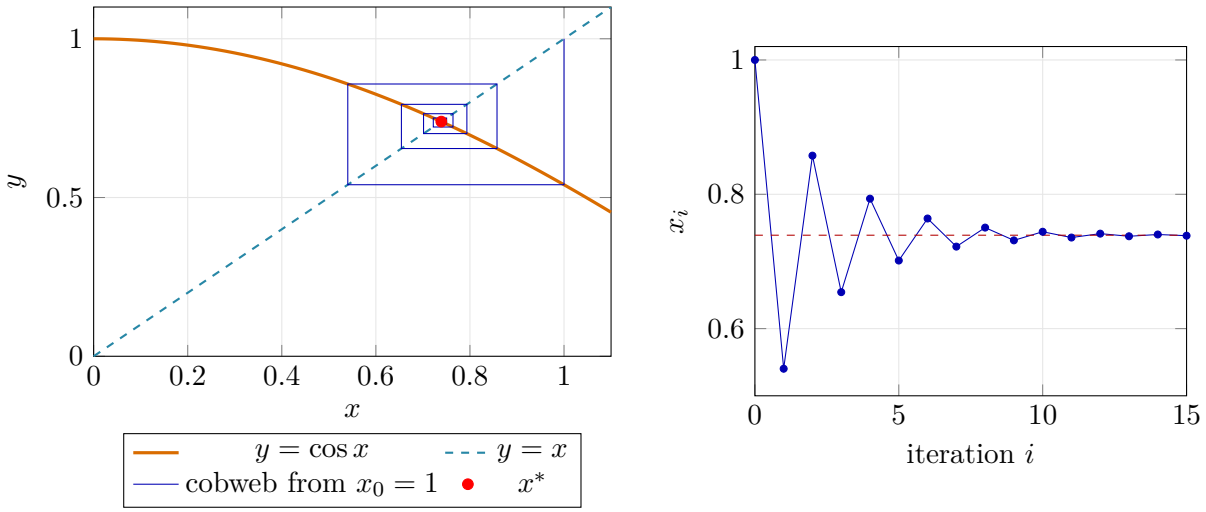


Figure 42: Fixed-point iteration of $f(x) = \cos x$ from $x_0 = 1$ (original figure in the style of slide 24; values computed exactly). Left: cobweb diagram. Right: $x_i \to x^* \approx 0.739085$.

Intuition

Fixed-point iteration is "keep applying the rule until it stops changing anything". Whether this works depends on the rule: with $f(x) = \cos x$ each step lands closer to x^* (the cobweb spirals in), but for $f(x) = 2x$ the iterates would run away from the fixed point 0. The property that guarantees spiralling *in* is that f brings any two points closer together, which is exactly what the next section calls a contraction.

5 Contractions *(slides 25–28)*

Convergence to a fixed point is possible thanks to the existence of **contraction mappings**.

1

Definition 5.1 (Contraction)**.** Let (M, d) be a metric space. An operator $f : M \to M$ is a *k-contraction* if there is $k \in [0, 1)$ with

$$d(f(x), f(y)) \leq k\, d(x, y) \qquad \text{for all } x, y \in M.$$

(On the slides: $f(x) - f(x') \leq kx - x'$ for $k \in [0, 1)$.)

2

Theorem 5.2 (Banach fixed-point theorem)**.** *A k-contraction on a non-empty* complete *metric space has* exactly one *fixed point x^*, and for any starting point x_0 the iteration $x_{i+1} = f(x_i)$ converges to it, with $d(x_i, x^*) \leq k^i\, d(x_0, x^*)$.*

Proof of uniqueness (this is the part on slide 25: "a contraction mapping has at most one fixed point"). If $f(x) = x$ and $f(y) = y$ then $d(x, y) = d(f(x), f(y)) \leq k\, d(x, y)$, and since $k < 1$ this forces $d(x, y) = 0$, i.e. $x = y$. □

The existence of a fixed point, the convergence of the iteration and the rate k^i are not proved on the slides; their proof is in Appendix A (supplementary).

Intuition

A contraction is a map that *squeezes space*: any two points end up closer together after one application, by a factor of at least $k < 1$. Two fixed points would stay at the same distance after applying f, which contradicts the squeezing, so there is at most one. Iterating f keeps squeezing the whole space towards a single point, which is the fixed point; completeness just guarantees that this point exists inside the space instead of being a "hole". The rate k^i is the geometric decay that all later bounds inherit.

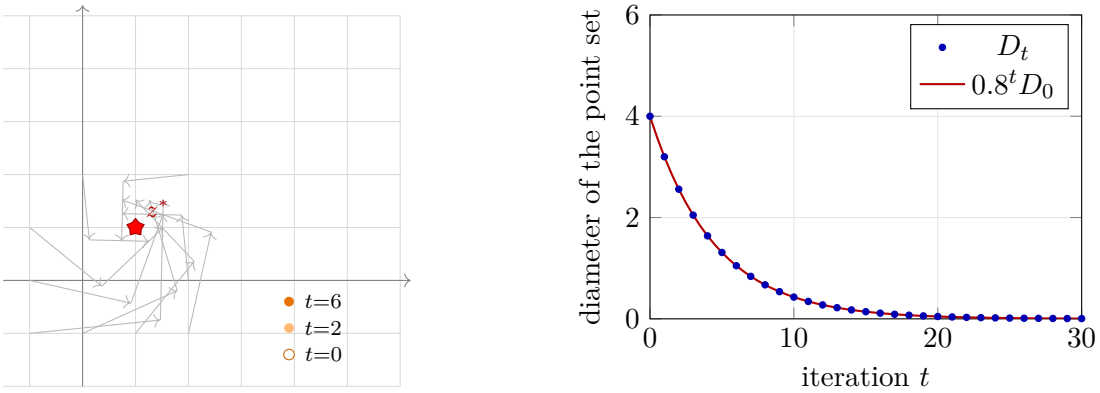


Figure 43: A contraction in ^{2} (original figure in the style of slide 26): $f(z) = 0.8\, R_{30^\circ} z + c$ with unique fixed point z^*. Left: nine starting points and their images after 2 and 6 steps all collapse onto z^*. Right: the diameter of the point set decays exactly like $k^t D_0$ (here f is a similarity, so the contraction inequality is tight).

Example 5.3 (Why cos is a contraction on $[0,1]$)**.** *(Added example connecting the cobweb of slide 24 to Banach's theorem; not on the slides.)* On $M = [0,1]$ (complete) cos maps into $[\cos 1, 1] \approx [0.540, 1] \subset M$, and by the mean value theorem $\cos x - \cos y = \sin \xi\, x - y \leq \sin(1)\, x - y$ with $\sin 1 \approx 0.841$. So cos is a 0.841-contraction on $[0,1]$; Theorem 1.4 gives a unique fixed point ($x^* \approx 0.739085$) and $x_i - x^* \leq 0.841^i x_0 - x^*$. In fig:cobweb, $x_0 = 1$ gives $x_5 - x^* \approx 0.038$, comfortably below the bound $0.841^5 \cdot 0.261 \approx 0.110$.

5.1 Contraction Operator *(slides 27–28)*

In the simplest case the contraction can be an *operator*, even as simple as a matrix O acting on value vectors:

$$OV - OV' \leq \gamma V - V'$$

(we can write γ instead of k, since both lie in $[0,1)$). The slide's demo takes $O = \gamma I$, "pure shrink": every vector is scaled by γ, so the distance between OV and OV' is exactly γ times the distance between V and V' (fig:operator).

Intuition

The point of this slide is the *reading* of "contraction" for value functions: think of each value function as a point in $^{|\mathcal{S}|}$ (one coordinate per state); an operator is a contraction when it makes two different value functions *more similar*. The Bellman backups we are about to study are not as trivial as γI, but they shrink distances in the same way, by at least a factor γ in sup-norm.

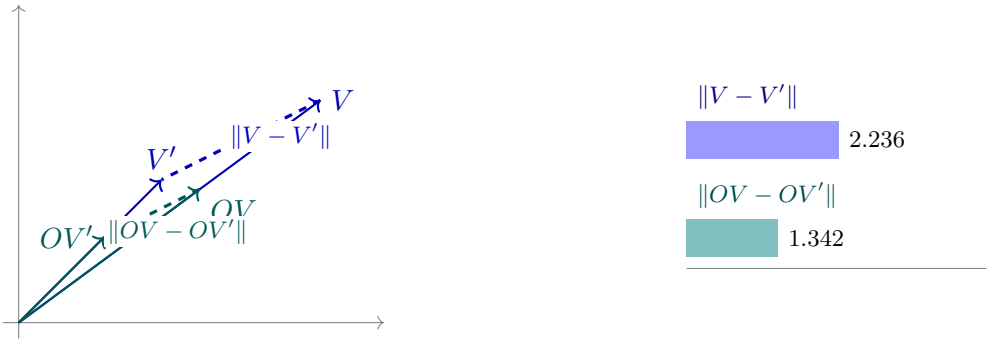


Figure 44: Contraction operator $O = \gamma I$ with $\gamma = 0.6$ (original figure in the style of slide 28; Euclidean norm). Applying O shrinks the distance between any two value vectors from 2.236 to $1.342 = 0.6 \times 2.236$.

6 Iterative Policy Evaluation *(slides 29–32)*

Instead of inverting a matrix, we exploit the fixed-point structure $V^\pi = r^\pi + \gamma P^\pi V^\pi$ and *iterate* it. We assume rewards are in $[0, 1]$.

Algorithm 6.1 (Iterative policy evaluation)**.** *Input:* an MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ and a policy π.

1. Initialise $V_0 \in [0, 1/(1-\gamma)]^{|\mathcal{S}|}$ (typically $V_0 = 0$).
2. Until convergence: $V_{i+1} = r^\pi + \gamma P^\pi V_i$ (matrix form: all states are updated at once).

In operator notation, $V_{i+1} = V_i$: this is the *Bellman expectation backup*.

(In the companion note: "repeat $V_{i+1} = V_i$ until $V_{i+1} - V_i$ is small.")

Why initialise in $[0, 1/(1-\gamma)]$? Under the assumption $r(s, a) \in [0, 1]$, the maximum possible value is $1 + \gamma + \gamma^2 + \cdots = 1/(1-\gamma)$ (Fact 1.3). There is no reason to initialise higher than that, since no true value lives there.

3

Definition 6.2 (Bellman operators)**.** For a policy π, a value function V and a state-action function Q:

$$(V)(s) = r(s, \pi(s)) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,\pi(s))}\big[V(s')\big] \qquad \text{(expectation backup, slide 32)},$$

$$(V)(s) = \max_a \Big[r(s, a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)}\big[V(s')\big]\Big] \qquad \text{(optimality backup, slide 44)},$$

$$(Q)(s, a) = r(s, a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)}\Big[\max_{a'} Q(s', a')\Big] \qquad \text{(on } Q\text{, slide 48)}.$$

is *affine* ($V = r^\pi + \gamma P^\pi V$); is *not* linear, because of the maximum. (Part of the literature writes T for .)

Intuition

A "backup" is one round of "look one step ahead and update". The expectation backup asks: *if I act according to π for one step and then trust my current guess V, what is the value of each state?* The optimality backup asks the same but lets me *pick the best action* in the first step. Facts 1.4 and 1.5 say precisely that V^π and V^* are the states of the world these backups leave untouched.

Example 6.3 (Iterative evaluation on the two-state MDP)**.** For $\pi = (a, a)$ in Example 3.2 and $V_0 = 0$ (so $V_0 - V^\pi = 8.594$):

i	$V_i(s_1)$	$V_i(s_2)$	$V_i - V^\pi$	$\gamma^i V_0 - V^\pi$
0	0.000	0.000	8.594	8.594
1	0.000	1.000	7.594	7.734
2	0.450	1.810	6.784	6.961
3	1.017	2.507	6.087	6.265
4	1.586	3.122	5.472	5.638
5	2.118	3.671	4.922	5.075
6	2.605	4.165	4.429	4.567
7	3.046	4.608	3.986	4.110
8	3.444	5.006	3.587	3.699
9	3.803	5.365	3.229	3.329
10	4.126	5.688	2.906	2.996
11	4.416	5.979	2.615	2.697
12	4.678	6.240	2.354	2.427
13	4.913	6.476	2.118	2.184
14	5.125	6.687	1.906	1.966
15	5.315	6.878	1.716	1.769

The first sweep gives $V_1 = r^\pi = (0, 1)$; each later sweep propagates reward information one step further. The true error always stays below the geometric bound (last column), as proved next.

7 Iterative Policy Evaluation Theorem *(slides 33–37)*

The slides' claim is that, for any two value functions U and V and for all $s \in \mathcal{S}$,

$$(U)(s) - (V)(s) \le \gamma U - V, \qquad V = \max_s V(s)$$

("the largest difference between state values", worst case scenario to infinity). The companion note packages this together with the analogous claim for (proved later, slides 44–47):

4

Theorem 7.1 (Both operators are γ-contractions in the sup-norm)**.** *For all value functions U, V:*

$$\textit{(i)}\ \ U - V \le \gamma U - V, \qquad \textit{(ii)}\ \ U - V \le \gamma U - V.$$

The same holds for acting on state-action functions Q.

Proof of (i), as on slides 33–37. For the rewards cancel. For every state s,

$$\begin{aligned}
(U)(s) - (V)(s) &= \gamma \left| \mathbb{E}_{s'} [U(s') - V(s')] \right| \\
&\le \gamma\, \mathbb{E}_{s'} U(s') - V(s') && \text{(Jensen: } \cdot \text{ is convex)} \\
&\le \gamma\, U - V && \text{(an average is at most the maximum).}
\end{aligned}$$

The inequality holds for every s, so also for the state where the left-hand side is largest; taking the maximum over s gives (i). The proof of (ii) is in Section 10. □

Intuition

Two things happen in the proof. First, both backups use the *same* rewards, so these cancel: the only thing that differs is the next-state values, scaled by γ. Second, the next-state values are *averaged* by probabilities that sum to one, and an average of numbers each at most $U - V$ in size cannot exceed $U - V$. So no state can end up further apart than γ times the worst gap before. As the slide remarks, we are not secretly using the sup-norm on the left-hand side: the bound holds *for each state*, including the one with the largest difference.

Slide 34 draws the conclusion: *hence the Bellman expectation backup is a contraction mapping and V^π is its fixed point.* This follows from Theorem 1.4 applied to on $(^{|\mathcal{S}|}, \cdot)$, which is complete (a finite-dimensional normed space): by Fact 1.4, $V^\pi = V^\pi$, so V^π is a fixed point, and by Banach's uniqueness it is the only one. (This is part (a) of Corollary 12.1, stated in full in the Value Iteration section.)

8 Iterative Policy Evaluation: Iterations 38

6

Corollary 8.1 (Convergence and cost of iterative policy evaluation)**.** *$V_i - V^\pi \le \gamma^i V_0 - V^\pi$, so $V_i - V^\pi \le \epsilon$ after*

$$i \;\ge\; \frac{\ln(V_0 - V^\pi/\epsilon)}{\ln(1/\gamma)}$$

iterations, each costing $O(|\mathcal{S}|^2)$ arithmetic operations. Hence an ϵ-accurate value function costs $O(|\mathcal{S}|^2 \ln(1/\epsilon))$ (treating γ and $V_0 - V^\pi$ as constants).

Proof. The rate is Theorem 1.4 with $k = \gamma$, applied to whose unique fixed point is V^π. Solving $\gamma^i V_0 - V^\pi \le \epsilon$ for i (take logarithms; $\ln\gamma = -\ln(1/\gamma) < 0$ flips the inequality) gives the count. Each sweep updates $|\mathcal{S}|$ states and each update averages over $|\mathcal{S}|$ successors, i.e. one matrix–vector product $P^\pi V_i$. □

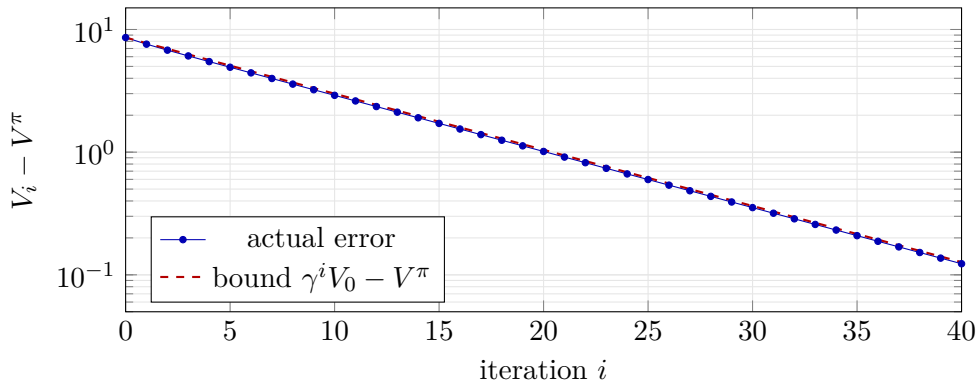


Figure 45: Iterative policy evaluation on Example 3.2 with $\pi = (a, a)$, $\gamma = 0.9$, $V_0 = 0$. On a log scale the error is a straight line: geometric convergence. The error decays geometrically and remains below the contraction bound.

Intuition

Worked numbers. With $\gamma = 0.9$, $\epsilon = 0.01$ and $V_0 - V^\pi = 8.594$: $i \ge \ln(859.4)/\ln(1/0.9) = 6.756/0.1054 = 64.1$, so 65 sweeps (indeed $0.9^{65} \cdot 8.594 \approx 0.009$). Each sweep is only $|\mathcal{S}|^2 = 4$ multiplications here. The *accuracy* costs only a logarithm (10× smaller ϵ adds

$\ln 10/\ln(1/\gamma) \approx 22$ sweeps), while the *horizon* $\ln(1/\gamma) \approx 1-\gamma$ drives the cost. Compared with $O(|\mathcal{S}|^3)$ for exact evaluation, iteration wins when $|\mathcal{S}|$ is large and ϵ is moderate.

9 How to Find the Optimal Policy? *(slides 39–43)*

Now what we are really interested in is finding the optimal policy π^*.

Naive approach (slides 40–42): we know how to do policy evaluation, so

- for each possible policy, for all states,
- do policy evaluation and compute $V^\pi(s)$,
- choose π' such that $V^{\pi'}(s) \geq V^\pi(s)$ (keep the best).

If we use exact policy evaluation the cost is $O(|\mathcal{A}|^{|\mathcal{S}|}\,|\mathcal{S}|^3)$: $|\mathcal{A}|^{|\mathcal{S}|}$ policies, each needing a cubic solve.

Intuition

The slide's horrified emoji is justified. The number of deterministic policies grows *exponentially* in the number of states: for $|\mathcal{A}| = 2$, $|\mathcal{S}| = 10$ there are 1024 policies (about 10^6 operations in total), but for $|\mathcal{S}| = 30$ there are $2^{30} \approx 10^9$ policies and about 2.9×10^{13} operations. Enumeration treats the policies as unrelated black boxes. The better idea (slide 43): **let's use the Bellman optimality equation!** Rather than searching over policies, search for the one function V^* (or Q^*) that satisfies it, and read the policy off with an arg max (Fact 1.5).

10 The Bellman Optimality Backup is a Contraction *(slides 44–47)*

Set $\gamma < 1$ and use the sup-norm $V = \max_s V(s)$. Define the (non-linear) operator V as the Bellman optimality equation applied to V (Definition 6.2):

$$(V)(s) = \max_a \Big[r(s,a) + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s,a)\, V(s') \Big].$$

We need one elementary fact about maxima, used implicitly on the slides:

Lemma 10.1 (The maximum is 1-Lipschitz)**.** *For any two functions f, g on a finite set of actions, $\big|\max_a f(a) - \max_a g(a)\big| \leq \max_a f(a) - g(a)$.*

Proof. Let a_f attain $\max_a f$. Then $\max_a f - \max_a g \leq f(a_f) - g(a_f) \leq \max_a f(a) - g(a)$. Exchanging the roles of f and g gives the other sign. □

Proof of Theorem 7.1 (ii), following slides 45–47. Fix two value functions V_k, V_j and a state s. Using Lemma 10.1 with $f(a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V_k(s')$ and g the same with V_j, the reward terms cancel:

$$\begin{aligned} \big|(V_k)(s) - (V_j)(s)\big| &\leq \max_a \Big| \gamma \sum_{s'} p(s' \mid s,a)\big(V_k(s') - V_j(s')\big) \Big| \\ &\leq \max_a \gamma \sum_{s'} p(s' \mid s,a)\, V_k - V_j \\ &= \gamma V_k - V_j, \end{aligned}$$

where the second step bounds each $V_k(s') - V_j(s')$ by the sup-norm, and the last equality uses $\sum_{s'} p(s' \mid s,a) = 1$. Taking the maximum over s gives $V_k - V_j \leq \gamma V_k - V_j$. □

Intuition

If you apply to two different value functions, *the distance between them shrinks* (slide 46): makes value functions closer by at least a factor γ (slide 47). The new ingredient compared with the expectation backup is the max, and Lemma 10.1 says it cannot amplify differences: changing a few action values by at most δ changes the best of them by at most δ. So even though is non-linear, it inherits the same squeezing property as .

11 Bellman Operator for Q 48

Since $Q : \mathcal{S} \times \mathcal{A} \to$, also $Q : \mathcal{S} \times \mathcal{A} \to$:

$$(Q)(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)} \Big[\max_{a'} Q(s',a') \Big],$$

and the norm is now $Q = \max_{s,a} Q(s,a)$.

Proof of the last claim of Theorem 7.1 (the Q-version). Rewards cancel, and the maximum now sits *inside* the expectation. For every (s,a),

$$(Q)(s,a) - (Q')(s,a) = \gamma \Big| \mathbb{E}_{s'} \big[\max_{a'} Q(s',a') - \max_{a'} Q'(s',a') \big] \Big| \le \gamma\, \mathbb{E}_{s'} \max_{a'} Q(s',a') - Q'(s',a') \le \gamma Q - Q',$$

using $\mathbb{E}X \le \mathbb{E}X$, Lemma 10.1, and that an average is at most the maximum. Taking the maximum over (s,a) finishes the proof. □

Intuition

Nothing new happens: the Q-table has $|\mathcal{S}||\mathcal{A}|$ entries instead of $|\mathcal{S}|$, and the same "rewards cancel, averages cannot exceed the worst case" argument applies. Working with Q is convenient because a policy can be read off from Q without a model: $\arg\max_a Q(s,a)$.

12 Value Iteration *(slides 49–53)*

We can obtain $Q^* = Q^*$, since Q^* is a fixed point of $Q = Q$ (and all of this also holds for V^*).

5

Corollary 12.1 (What the fixed points are)**.** (a) *V^π is the unique fixed point of ;* (b) *V^* is the unique fixed point of on value functions;* (c) *Q^* is the unique fixed point of on state-action functions.*

Proof. Fact 1.4 says $V^\pi = V^\pi$ and Fact 1.5 says $V^* = V^*$ and $Q^* = Q^*$, so each is a fixed point. The spaces $(^{|\mathcal{S}|}, \cdot)$ and $(^{|\mathcal{S}||\mathcal{A}|}, \cdot)$ are complete, and Theorem 7.1 says the operators are γ-contractions, so Theorem 1.4 says each has only one fixed point. □

(This also re-proves Fact 1.5, slide 11, in one line: any V with $V = V$ is the fixed point of , hence equals V^*; this is what the long unrolling argument in L03 did by hand.)

Algorithm 12.2 (Value iteration)**.** *Input:* an MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$.

1. Initialise Q_0 with entries in $[0, 1/(1-\gamma)]$ (typically $Q_0 = 0$).
2. Until convergence, for all states and actions:

$$Q_{i+1}(s,a) = (Q_i)(s,a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) \max_{a'} Q_i(s',a').$$

3. Return the greedy policy $\pi_i(s) \in \arg\max_a Q_i(s,a)$ (slide 54).

7

Theorem 12.3 (Value iteration converges geometrically)**.** *$Q_i - Q^* \le \gamma^i Q_0 - Q^*$, and each iteration costs $O(|\mathcal{S}|^2|\mathcal{A}|)$.*

Proof. By Theorem 7.1 and Corollary 12.1(c), $Q_{i+1} - Q^* = Q_i - Q^* \le \gamma Q_i - Q^*$, and induction gives $Q_i - Q^* \le \gamma^i Q_0 - Q^*$ (slide 51). For the cost (slide 53): compute $m_i(s') = \max_{a'} Q_i(s',a')$ once per state ($|\mathcal{S}||\mathcal{A}|$ operations); then each of the $|\mathcal{S}||\mathcal{A}|$ entries needs a sum over $|\mathcal{S}|$ successors, using the shared m_i. Total $O(|\mathcal{S}|^2|\mathcal{A}|)$. □

Intuition

Value iteration is the same fixed-point iteration as in the cobweb picture, run on the *vector* Q with the operator : start from any guess, repeatedly "look one step ahead and take the best action", and the guess is pulled towards Q^* at geometric speed. No search over policies is needed: compare $O(|\mathcal{S}|^2|\mathcal{A}|)$ per iteration, for about $\ln(1/\epsilon)/(1-\gamma)$ iterations, with the $|\mathcal{A}|^{|\mathcal{S}|}$ of the naive approach. A useful way to see how it works: after i iterations, $Q_i(s,a)$ is the best total discounted reward you can collect in the first i steps (from a terminal value of Q_0), so reward information spreads one step per iteration.

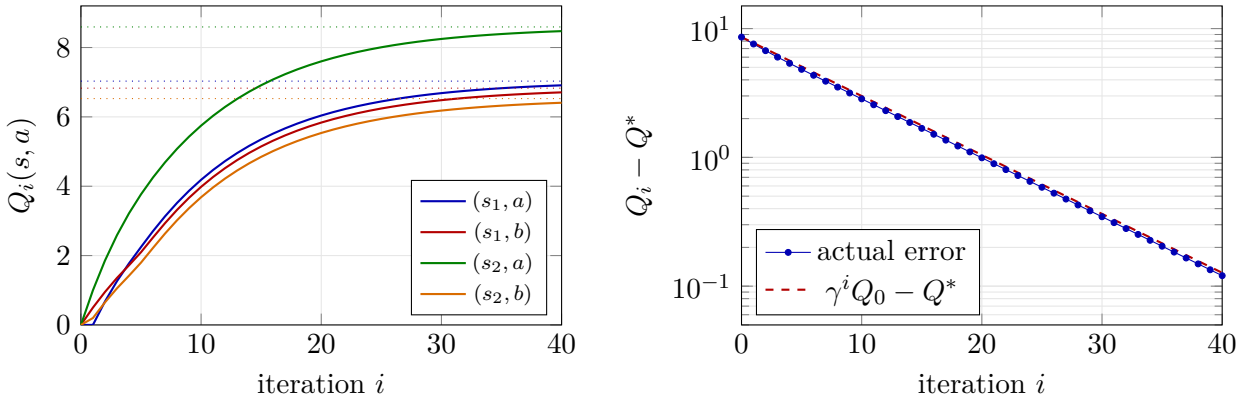


Figure 46: Value iteration on Example 3.2 ($\gamma = 0.9$, $Q_0 = 0$), in the spirit of the demo on slide 52. Left: the four Q-values converge to Q^* (dotted lines). Right: the error stays below the geometric bound of Theorem 12.3.

Example 12.4 (First iterations of value iteration)**.** $Q_1 = r$ (from $Q_0 = 0$). Then, e.g., $Q_2(s_1,a) = 0 + 0.9\,(0.5 \cdot \max Q_1(s_1,\cdot) + 0.5 \cdot \max Q_1(s_2,\cdot)) = 0.9(0.5 \cdot 0.5 + 0.5 \cdot 1) = 0.675$, matching the table in Example 13.2. The fixed point is $Q^* = \begin{pmatrix} 7.03125 & 6.828125 \\ 8.59375 & 6.528125 \end{pmatrix}$ (rows s_1, s_2; columns a, b), so $V^* = (7.03125,\ 8.59375)$ and $\pi^* = (a, a)$.

13 Policy from Value Iteration *(slides 54–65)*

We know that $\pi^*(s) = \arg\max_a Q^*(s,a)$, and since $Q_i(s,a) \approx Q^*(s,a)$ we can choose

$$\pi_i(s) = \arg\max_a Q_i(s,a).$$

What is the quality of such a policy? For all states,

Theorem 13.1 (Quality of the greedy policy)**.** *Let $\pi_i(s) \in \arg\max_a Q_i(s,a)$. Then for every state s,*

$$V^{\pi_i}(s) \;\geq\; V^*(s) - \frac{2Q_i - Q^*}{1-\gamma} \;\geq\; V^*(s) - \frac{2\gamma^i Q_0 - Q^*}{1-\gamma}.$$

Proof. Write $\varepsilon = Q_i - Q^*$ and fix s. Using $V^{\pi_i}(s) = Q^{\pi_i}(s,\pi_i(s))$ and $V^*(s) = Q^*(s,\pi^*(s))$ (slide 56), add and subtract $Q^*(s,\pi_i(s))$ (slide 57):

$$V^{\pi_i}(s) - V^*(s) = \underbrace{\left[Q^{\pi_i}(s,\pi_i(s)) - Q^*(s,\pi_i(s))\right]}_{\text{(I)}} + \underbrace{\left[Q^*(s,\pi_i(s)) - Q^*(s,\pi^*(s))\right]}_{\text{(II)}}.$$

Term (I). The two Q's share the same reward term, which cancels (slide 58), so

$$\text{(I)} = \gamma\, \mathbb{E}_{s'\sim p(\cdot|s,\pi_i(s))}\left[V^{\pi_i}(s') - V^*(s')\right].$$

Term (II). Insert Q_i twice (slides 59–62):

$$\text{(II)} = \underbrace{\left[Q^*(s,\pi_i(s)) - Q_i(s,\pi_i(s))\right]}_{\geq -\varepsilon} + \underbrace{\left[Q_i(s,\pi_i(s)) - Q_i(s,\pi^*(s))\right]}_{\geq 0} + \underbrace{\left[Q_i(s,\pi^*(s)) - Q^*(s,\pi^*(s))\right]}_{\geq -\varepsilon} \;\geq\; -2\varepsilon.$$

The middle bracket is non-negative because π_i maximises Q_i; the outer two are each at least $-\varepsilon$ by definition of ε.

Putting it together. Let $\delta = \min_s \left[V^{\pi_i}(s) - V^*(s)\right] \leq 0$ (no policy beats V^*). Taking the state attaining the minimum, and bounding the expectation in (I) below by δ,

$$\delta \geq \gamma\delta - 2\varepsilon \implies \delta(1-\gamma) \geq -2\varepsilon \implies \delta \geq -\frac{2\varepsilon}{1-\gamma}.$$

This is the first inequality (it holds for every s since δ is the minimum over s). The second inequality is Theorem 12.3. □

Remark (The slides' "repeat" version, slides 63–65). The slides stop at $V^{\pi_i}(s) - V^*(s) \geq \gamma\, \mathbb{E}_{s'}[V^{\pi_i}(s') - V^*(s')] - 2\gamma^i Q_0 - Q^*$, using $Q_i - Q^* \leq \gamma^i Q_0 - Q^*$ twice, and then say "repeat and get $1/(1-\gamma)$": plugging the same bound into the expectation on the right again and again produces the geometric series $2\gamma^i Q_0 - Q^*\,(1 + \gamma + \gamma^2 + \cdots) = 2\gamma^i Q_0 - Q^*/(1-\gamma)$. The δ-trick above is the same argument written in one line.

Intuition

Why a factor 2? The greedy choice compares actions using Q_i instead of Q^*; each of the two actions being compared (the one we pick and the true best) can be mis-valued by up to ε, in opposite directions, so the *one-step* regret is up to 2ε. *Why $1/(1-\gamma)$?* A one-step mistake is not a one-time cost: the policy keeps following the same imperfect scores in all later states, so mistakes accumulate over the whole effective horizon. Note the guarantee is about the *policy actually executed*, V^{π_i}, not about Q_i.

Example 13.2 (The greedy policy on the two-state MDP)**.** Greedy policies $\pi_i = (\pi_i(s_1), \pi_i(s_2))$ from value iteration, with $Q_0 - Q^* = 8.594$, so the bound of Theorem 13.1 is $\frac{2\gamma^i}{1-\gamma} \cdot 8.594 = 171.9\,\gamma^i$:

i	$Q_i(s_1,a)$	$Q_i(s_1,b)$	$Q_i(s_2,a)$	$Q_i(s_2,b)$	$Q_i - Q^*$	π_i	$\max_s(V^* - V^{\pi_i})$	bound
0	0.000	0.000	0.000	0.000	8.594	(a,a)	0.000	171.875
1	0.000	0.500	1.000	0.200	7.594	(b,a)	2.031	154.688
2	0.675	0.950	1.855	0.650	6.739	(b,a)	2.031	139.219
3	1.262	1.355	2.588	1.055	6.006	(b,a)	2.031	125.297
4	1.774	1.720	3.218	1.419	5.375	(a,a)	0.000	112.767
5	2.247	2.097	3.766	1.797	4.827	(a,a)	0.000	101.490
6	2.706	2.522	4.253	2.222	4.341	(a,a)	0.000	91.341
8	3.519	3.318	5.080	3.018	3.514	(a,a)	0.000	73.987
10	4.185	3.982	5.747	3.682	2.846	(a,a)	0.000	59.929
15	5.351	5.147	6.913	4.847	1.681	(a,a)	0.000	35.388
20	6.039	5.836	7.601	5.536	0.992	(a,a)	0.000	20.896
30	6.685	6.482	8.248	6.182	0.346	(a,a)	0.000	7.286
40	6.911	6.707	8.473	6.407	0.121	(a,a)	0.000	2.540

At $i = 0$ all Q-values tie. For $i = 1, 2, 3$ the greedy policy is (b, a) and loses 2.031 in s_1 (it takes the immediate reward 0.5 of b instead of the long-run better a); from $i = 4$ on, it is $(a, a) = \pi^*$ with zero loss.

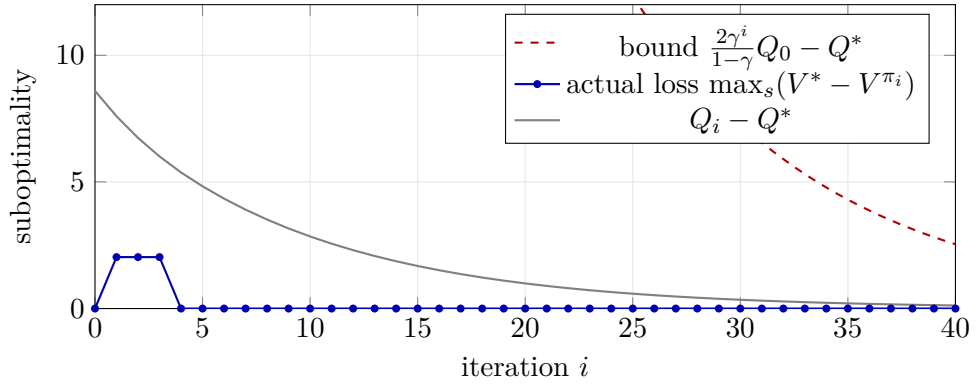


Figure 47: Guaranteed versus actual loss of the greedy policy (Example 13.2; original figure). The bound is vacuous (above the plotted range) until $i \approx 26$, yet the greedy policy is already optimal from $i = 4$. The theorem is a worst-case guarantee, not a prediction.

14 Number of Iterations of Value Iteration *(slides 66–74)*

Note: as with exact policy evaluation, the companion note lists this derivation as "on the slides only".

If we want an ϵ error on the quality of the policy, we can determine the number of iterations i by solving for it in

$$\frac{2\gamma^i}{1-\gamma} Q_0 - Q^* \leq \epsilon.$$

Proposition 14.1 (Number of iterations for an ϵ-optimal greedy policy)**.** *Let* $Q_0 \in [0, 1/(1-\gamma)]^{|\mathcal{S}||\mathcal{A}|}$ *(e.g.* $Q_0 = 0$*) and rewards in* $[0, 1]$*. If*

$$i \;\geq\; \frac{1}{1-\gamma} \ln\Big(\frac{2}{\epsilon(1-\gamma)^2}\Big),$$

then $V^{\pi_i}(s) \geq V^*(s) - \epsilon$ *for every state* s*.*

Proof. Step 1 (slides 67–69). Q^* also lies in $[0, 1/(1-\gamma)]$ (Fact 1.3), so by the infinity norm $Q_0 - Q^* \le 1/(1-\gamma)$ (for $Q_0 = 0$ it is just $Q^* \le 1/(1-\gamma)$). Hence by Theorem 13.1 it suffices that $\dfrac{2\gamma^i}{(1-\gamma)^2} \le \epsilon$.

Step 2 (slides 70–71). Write $\gamma = 1-(1-\gamma)$ ("add and subtract 1") and use $1+x \le e^x$ for all real x with $x = -(1-\gamma)$:

$$\gamma^i = (1-(1-\gamma))^i \le e^{-(1-\gamma)i}.$$

So it suffices that $\dfrac{2e^{-(1-\gamma)i}}{(1-\gamma)^2} \le \epsilon$, i.e. $e^{-(1-\gamma)i} \le \dfrac{\epsilon(1-\gamma)^2}{2}$.

Step 3 (slides 72–74). Take logarithms and use $\ln(1/x) = -\ln x$: $-(1-\gamma)i \le -\ln(\frac{2}{\epsilon(1-\gamma)^2})$. Isolating i gives the claimed condition. □

Intuition

Worked numbers ($\epsilon = 0.01$): for $\gamma = 0.9$ the condition is $i \ge 10\ln(20\,000) = 99.03$, so **100 iterations**; for $\gamma = 0.99$ it is $i \ge 100\ln(2\cdot 10^6) = 1450.9$, so **1451 iterations**. Making γ closer to one multiplies the work by about $10\times$ per extra "nine", while making ϵ ten times smaller adds only $\ln 10/(1-\gamma) \approx 23$ iterations at $\gamma = 0.9$. In words: accuracy is cheap (logarithmic in $1/\epsilon$), the planning horizon $1/(1-\gamma)$ is what costs. Multiplying by the per-iteration cost, an ϵ-optimal policy takes $O(|\mathcal{S}|^2|\mathcal{A}| \cdot \frac{1}{1-\gamma}\ln\frac{1}{\epsilon(1-\gamma)^2})$ arithmetic operations. (Replacing $\ln(1/\gamma)$ by its lower bound $1-\gamma$ is a safe simplification: it can only make the required i larger, e.g. 99.0 instead of 94.0 for $\gamma = 0.9$.)

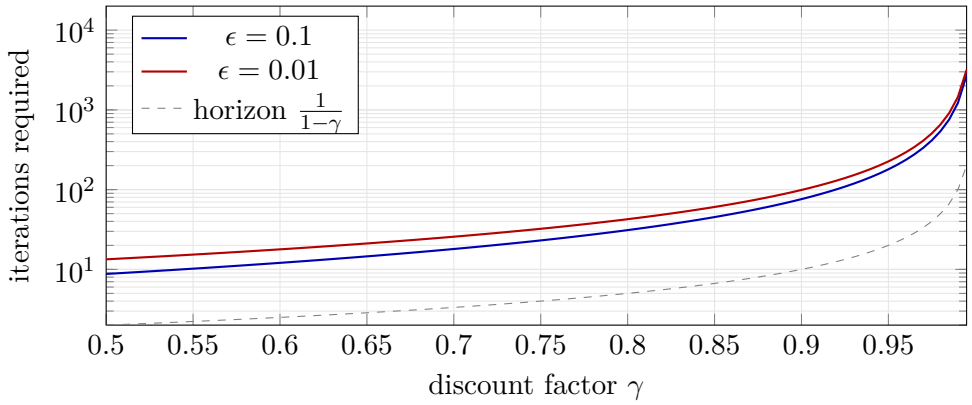


Figure 48: Iterations sufficient for an ϵ-optimal greedy policy (Proposition 14.1; original figure, slides 66–74 only give the formula). The curves track the horizon $1/(1-\gamma)$ up to a logarithmic factor.

15 Another Note on Value Iteration 75

Q_t is approximating Q^*, and from Q_t we compute a policy π_t. *However*, Q_t is generally different from Q^{π_t} until we converge to (approximately) Q^*. For example, Q_0 is just a random initial guess, which may not correspond to any policy's Q-value.

Intuition

Value iteration is *not* "evaluate a policy, then improve it". The intermediate tables Q_i are only approximations of Q^*; there may not exist any policy whose Q-function equals Q_i. In the two-state MDP, $Q_1 = r$, but every policy has strictly positive values, so $Q^\pi(s,a) = r(s,a) + \gamma\mathbb{E}V^\pi(s') > r(s,a)$ for all π: Q_1 is the Q-function of *no* policy. Only in the limit does Q_i become $Q^* = Q^{\pi^*}$. This is why Theorem 13.1 is needed at all: it tells us what the

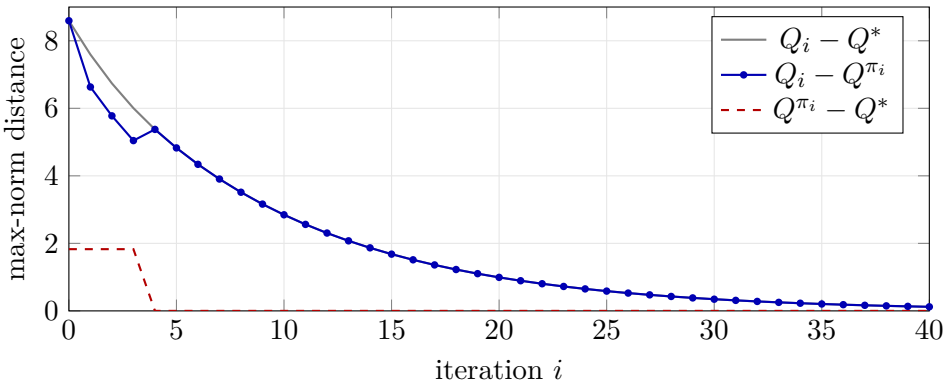


Figure 49: On Example 3.2: the iterate Q_i is far from the Q-function Q^{π_i} of its own greedy policy for many iterations, even after π_i has become optimal ($i \geq 4$, where $Q^{\pi_i} - Q^* = 0$). Ties at $i = 0$ are broken towards a. (Original figure; slide 75 is text only.)

greedy policy of an imperfect Q_i is worth.

16 Summary

16.1 Comparison of the methods and results

Method	Computes / operator	Cost	Guarantee	Slides
Exact policy evaluation	V^π; solve $(I - \gamma P^\pi)V = r^\pi$	$O(\lvert\mathcal{S}\rvert^3)$	exact (Prop. 3.1)	16–22
Iterative policy evaluation	V^π; (affine, γ-contraction)	$O(\lvert\mathcal{S}\rvert^2)$ per sweep; $O(\lvert\mathcal{S}\rvert^2 \ln \frac{1}{\epsilon})$ in total	$V_i - V^\pi \leq \gamma^i V_0 - V^\pi$ (Cor. 8.1)	29–38
Naive policy search	π^*; evaluate every policy	$O(\lvert\mathcal{A}\rvert^{\lvert\mathcal{S}\rvert}\lvert\mathcal{S}\rvert^3)$	exact, but exponential	39–42
Value iteration	Q^*; (non-linear, γ-contraction)	$O(\lvert\mathcal{S}\rvert^2\lvert\mathcal{A}\rvert)$ per iteration	$Q_i - Q^* \leq \gamma^i Q_0 - Q^*$ (Thm. 12.3)	49–53
Greedy policy from VI	$\pi_i = \arg\max_a Q_i$	free (an arg max)	$V^{\pi_i} \geq V^* - \frac{2\gamma^i}{1-\gamma} Q_0 - Q^*$ (Thm. 13.1)	54–65
ϵ-optimal policy	number of iterations	$i \geq \frac{1}{1-\gamma} \ln \frac{2}{\epsilon(1-\gamma)^2}$	$V^{\pi_i} \geq V^* - \epsilon$ (Prop. 14.1)	66–74

16.2 Cost scaling

16.3 Key take-aways

- **Policy evaluation is a linear problem.** $V^\pi = (I - \gamma P^\pi)^{-1} r^\pi$ exactly in $O(\lvert\mathcal{S}\rvert^3)$, or approximately by iterating $V_{i+1} = r^\pi + \gamma P^\pi V_i$ at $O(\lvert\mathcal{S}\rvert^2)$ per sweep and $O(\ln \frac{1}{\epsilon})$ sweeps.
- **Contractions are the engine.** A k-contraction on a complete metric space has a unique fixed point and fixed-point iteration converges to it at rate k^i (Banach). Both and are γ-contractions in the sup-norm, because rewards cancel and averages cannot exceed maxima; hence V^π, V^* and Q^* are their unique fixed points.
- **Value iteration** $Q_{i+1} = Q_i$ converges geometrically, $Q_i - Q^* \leq \gamma^i Q_0 - Q^*$, at $O(\lvert\mathcal{S}\rvert^2\lvert\mathcal{A}\rvert)$ per iteration, avoiding the $\lvert\mathcal{A}\rvert^{\lvert\mathcal{S}\rvert}$ policy enumeration.

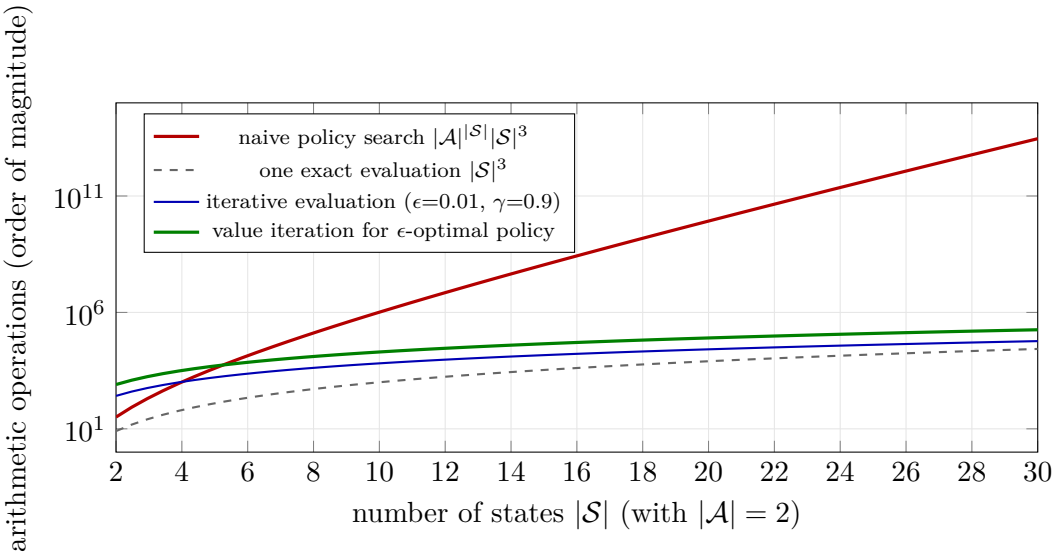


Figure 50: Qualitative cost comparison (original figure; constants dropped, iteration counts from the worked examples: 65 sweeps for evaluation, 100 iterations $\times\, |\mathcal{A}| = 2$ for value iteration, $\gamma = 0.9$, $\epsilon = 0.01$). Enumerating policies explodes exponentially; value iteration stays polynomial.

- **The greedy policy is near-optimal, with a $\frac{2}{1-\gamma}$ penalty:** $V^{\pi_i} \geq V^* - \frac{2\gamma^i}{1-\gamma} Q_0 - Q^*$, so $i \geq \frac{1}{1-\gamma} \ln \frac{2}{\epsilon(1-\gamma)^2}$ iterations give an ϵ-optimal policy: accuracy is logarithmic, the horizon $\frac{1}{1-\gamma}$ is what costs.
- **The intermediate Q_i are not policy values.** Value iteration does not alternate evaluation and improvement; Q_i is generally different from Q^{π_i} until convergence, and the bound in Theorem 13.1 is a worst-case guarantee that is often very pessimistic (fig:greedy).

A Supplementary: Banach's Theorem, Existence and Rate

*This appendix is **not covered on the slides**; it comes from the companion note only. The slides state the uniqueness half of Theorem 1.4 (slide 25) and then use convergence of fixed-point iteration (slides 23–24, 29–38, 49–53) without proving it.*

Proof of Theorem 1.4: existence and rate. Existence. From $x_{i+1} = f(x_i)$ we get $d(x_{i+1}, x_i) = d(f(x_i), f(x_{i-1})) \leq k\, d(x_i, x_{i-1})$, hence $d(x_{i+1}, x_i) \leq k^i\, d(x_1, x_0)$ by induction. For $j > i$ the triangle inequality gives

$$d(x_j, x_i) \leq \sum_{l=i}^{j-1} d(x_{l+1}, x_l) \leq \sum_{l=i}^{j-1} k^l\, d(x_1, x_0) \leq \frac{k^i}{1-k}\, d(x_1, x_0),$$

which tends to 0 as $i \to \infty$: the sequence is Cauchy, and by completeness it has a limit x^*. A contraction is continuous (it is Lipschitz), so $f(x^*) = f(\lim x_i) = \lim f(x_i) = \lim x_{i+1} = x^*$.

Rate. Since $f(x^*) = x^*$, $d(x_i, x^*) = d(f(x_{i-1}), f(x^*)) \leq k\, d(x_{i-1}, x^*)$, and induction gives $d(x_i, x^*) \leq k^i\, d(x_0, x^*)$. □

Intuition

The sequence of iterates is a walk whose steps shrink geometrically ($d(x_{i+1}, x_i) \leq k^i d(x_1, x_0)$), so the *total remaining path length* is a convergent geometric series. A walk with summable steps cannot wander forever: it settles somewhere (completeness says that "somewhere" is a

point of the space), and by continuity that limit must be unmoved by f. The rate is then just the contraction inequality applied to the pair (x_{i-1}, x^*).

B Supplementary: Self-Check List from the Companion Note

Study aid reproduced from the companion note (not a slide topic). "State" means: write the statement with assumptions and say in one sentence what it claims (for an algorithm: write its steps). "Prove" means: write the proof without looking. The last column gives the proof video of the same lecture on Classroom and where the item lives in these notes.

Item	state	prove	Video / location here
On the slides only: exact policy evaluation (slides 16–22)	☐	☐	Prop. 3.1, §3
Definition 1 (contraction)	☐		§5
Theorem 2 (Banach)	☐	☐	`ch05-banach.mp4`; §5, App. A
Definition 3 (Bellman operators)	☐		§6
Theorem 4 (both γ-contractions)	☐	☐	`ch05-contraction.mp4`; §7, §10
Corollary 5 (what the fixed points are)	☐	☐	`ch05-vi.mp4`; §12
Algorithm A (iterative policy evaluation)	☐		§6
Corollary 6 (convergence and cost)	☐	☐	`ch05-vi.mp4`; §8
Algorithm B (value iteration)	☐		§12
Theorem 7 (VI converges geometrically)	☐	☐	`ch05-vi.mp4`; §12
Theorem 8 (quality of the greedy policy)	☐	☐	`ch05-greedyquality.mp4`; §13
On the slides only: iterations of VI for an ϵ-optimal policy (slides 66–74)	☐	☐	Prop. 14.1, §14

Part IV

Lecture 3 – Part 2

Setting, notation and imported results

Throughout, $(, , p, r, \gamma)$ is a finite discounted MDP with $\gamma \in [0, 1)$, transition kernel $p(s' \mid s, a)$ and rewards $r(s, a) \in [0, 1]$. A policy $\pi(a \mid s)$ may be stochastic; deterministic stationary policies are the special case in which $\pi(\cdot \mid s)$ is a point mass, written $\pi(s)$. The value functions are

$$V^\pi(s) = \mathbb{E}^\pi\Big[\sum_{t\geq 0} \gamma^t r(s_t, a_t) \,\Big|\, s_0 = s\Big], \qquad Q^\pi(s, a) = r(s, a) + \gamma\, \mathbb{E}_{s'\sim p(\cdot|s,a)}\big[V^\pi(s')\big],$$

so that $V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)} Q^\pi(s, a)$ and, because $r \in [0, 1]$, $0 \leq Q^\pi, V^\pi \leq 1/(1 - \gamma)$. In vector form (functions on are vectors in $^{||}$) let

$$r^\pi(s) = \sum_a \pi(a \mid s)\, r(s, a), \qquad P^\pi(s, s') = \sum_a \pi(a \mid s)\, p(s' \mid s, a),$$

so P^π is a row-stochastic matrix. Two operators on value functions will be used:

$$(B^\pi V) = r^\pi + \gamma P^\pi V, \qquad (V)(s) = \max_a \Big\{r(s, a) + \gamma\, \mathbb{E}_{s'\sim p(\cdot|s,a)} V(s')\Big\}.$$

The sup-norm is $\cdot$. Inequalities between functions hold pointwise.

Results imported from earlier lectures (used, not re-proved here).

Label	Content as used below
L03, Lemma 3	Value functions and discounted sums are bounded (by $1/(1-\gamma)$ here), so all series below converge absolutely.
L03, Lemma 5	A discounted sum along trajectories equals $\frac{1}{1-\gamma}$ times an expectation under the discounted occupancy measure (defined in Section 5).
L03, Theorem 6	Bellman expectation equations: $Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} V^\pi(s')$ and $V^\pi = B^\pi V^\pi$.
L03, Theorem 8	The optimal value function satisfies $V^* = V^*$.
L03, Theorem 10	Any fixed point of is V^*.
L04, Theorem 2	Value iteration converges: $^k V \to V^*$ for every bounded V.
L04, Theorem 4	is a γ-contraction in sup-norm: $U - V \leq \gamma U - V$.

1 Recap2–11

Policy iteration is built from two ingredients seen earlier: *evaluating* a fixed policy and *improving* it. This section collects exactly what is needed.

1.1 Bellman Equation3

The value of a state is the current reward plus the discounted value of where we land next, with the next state drawn according to the policy.

Theorem 1.1 (Bellman expectation equations)**.** *For every policy π,*

$$V^\pi(s) = r(s, \pi(s)) + \gamma\, \mathbb{E}_{s'\sim p(\cdot|s,\pi(s))}\big[V^\pi(s')\big], \qquad Q^\pi(s, a) = r(s, a) + \gamma\, \mathbb{E}_{s'\sim p(\cdot|s,a)}\big[V^\pi(s')\big],$$

and consequently $V^\pi(s) = Q^\pi(s, \pi(s))$ *(for deterministic* π*; in general* $V^\pi(s) = \mathbb{E}_{a\sim\pi} Q^\pi(s,a)$*).*

Intuition

In $V^\pi(s)$ the action is dictated by π; in $Q^\pi(s,a)$ the *first* action a is forced and only afterwards does π take over. Both share the same one-step structure: "reward now + $\gamma\times$ value of the next state". The only difference is who picks the first action. This one-step view is what lets us compare policies by changing a single decision.

1.2 Bellman Optimality Example4

Take a state s with two actions a' and a'' leading to s' and s'', and assume V^* is known at the successors. Trying each action gives $Q^*(s,a') = r(s,a') + \gamma V^*(s')$ and $Q^*(s,a'') = r(s,a'') + \gamma V^*(s'')$, and

$$V^*(s) = \max_a \left[r(s,a) + \gamma\, \mathbb{E}_{s'\sim p(\cdot|s,a)} V^*(s') \right] = \max\left\{ Q^*(s,a'),\, Q^*(s,a'') \right\}.$$

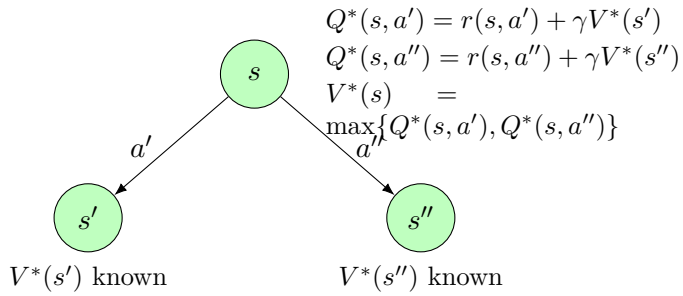


Figure 51: One-step lookahead behind the Bellman optimality equation (redrawn).

Intuition

Worked numbers ($\gamma = 0.9$, deterministic moves): if $r(s,a') = 1$ and $V^*(s') = 10$, then $Q^*(s,a') = 1 + 9 = 10$; if $r(s,a'') = 0$ and $V^*(s'') = 12$, then $Q^*(s,a'') = 10.8$. So $V^*(s) = 10.8$ and the best action is a'' even though its immediate reward is smaller: a good destination can outweigh a good first reward.

1.3 Exact Policy Evaluation5

The Bellman equation holds for *all* states at once, so the $|\mathcal{S}|$ scalar equations stack into one linear system

$$V^\pi = r^\pi + \gamma P^\pi V^\pi \quad\Longleftrightarrow\quad (I - \gamma P^\pi) V^\pi = r^\pi \quad\Longleftrightarrow\quad V^\pi = (I - \gamma P^\pi)^{-1} r^\pi.$$

Lemma 1.2. *$I - \gamma P^\pi$ is invertible and $(I - \gamma P^\pi)^{-1} = \sum_{k\geq 0} \gamma^k (P^\pi)^k$.*

Proof. P^π is row-stochastic, so $\|P^\pi\| = 1$ in the induced norm and $\|\gamma P^\pi\| = \gamma < 1$. The Neumann series converges, and multiplying it by $I - \gamma P^\pi$ telescopes to I. □

Intuition

It is a "nice but expensive" fix: one linear solve gives the exact value of a policy, but inverting (or factorising) a $|\mathcal{S}| \times |\mathcal{S}|$ matrix costs $O(|\mathcal{S}|^3)$. This is the price paid for exactness, and it dominates the cost of each policy iteration step later on.

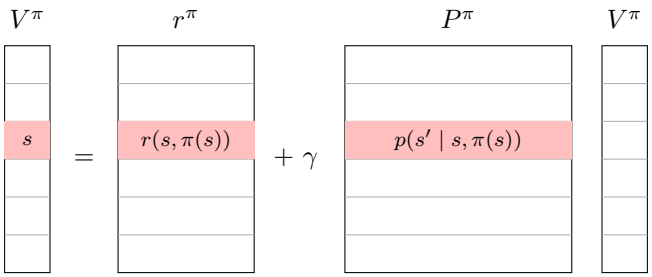


Figure 52: Stacking the Bellman equations: row s of the matrix product is exactly the scalar equation for state s.

1.4 Fixed-Point Iteration & Contractions6

A *fixed point* of f is a point with $x = f(x)$. To find one: initialise x_0, repeat $x_{k+1} = f(x_k)$, and stop when x no longer changes.

Definition 1.3 (Contraction mapping)**.** A map $f : M \to M$ on a metric space (M, d) is a contraction if $d(f(x), f(x')) \leq \kappa\, d(x, x')$ for some $\kappa \in [0, 1)$ and all x, x'.

Theorem 1.4 (Banach fixed-point theorem (standard))**.** *If (M, d) is complete and f is a κ-contraction, then f has a unique fixed point $x^\star$ and, for every x_0, $d(x_k, x^\star) \leq \kappa^k d(x_0, x^\star)$.*

Proof. Uniqueness: two fixed points x, x' satisfy $d(x, x') = d(fx, fx') \leq \kappa d(x, x')$, forcing $d(x, x') = 0$. Existence: $d(x_{k+1}, x_k) \leq \kappa^k d(x_1, x_0)$, so (x_k) is Cauchy (geometric tails), and its limit $x^\star$ is a fixed point by continuity of f. Finally $d(x_k, x^\star) = d(fx_{k-1}, fx^\star) \leq \kappa\, d(x_{k-1}, x^\star)$; iterate. □

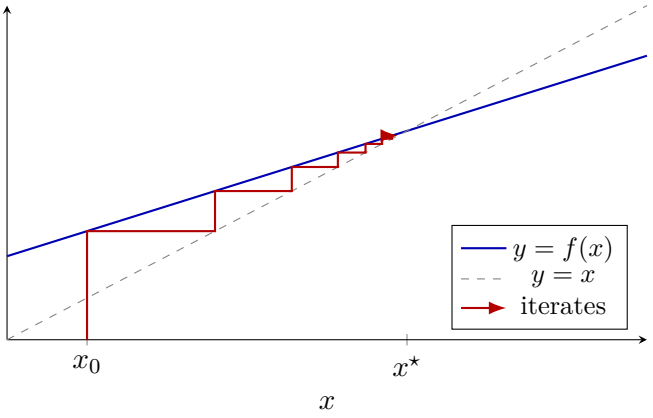


Figure 53: Cobweb view of fixed-point iteration for the contraction $f(x) = 0.6x + 2$ (original illustration): each step jumps vertically to the graph, then horizontally to the diagonal, and spirals into $x^\star = 5$.

Intuition

A contraction pulls any two points closer by a constant factor at every application, so repeated application squeezes everything onto a single point, no matter where we start. Every method below (iterative policy evaluation, value iteration) is "apply a contraction until it stops moving".

1.5 Iterative Policy Evaluation7

To avoid the cubic cost, initialise $V_0 \in [0, 1/(1-\gamma)]$ (typically 0) and iterate, in matrix form because all states are updated at once,

$$V_{i+1} = r^\pi + \gamma P^\pi V_i \qquad \text{until convergence.}$$

Lemma 1.5. *B^π is a γ-contraction in sup-norm with fixed point V^π; hence $V_{i+1} - V^\pi \le \gamma V_i - V^\pi \le \gamma^{i+1} V_0 - V^\pi$.*

Proof. $B^\pi U - B^\pi V = \gamma P^\pi (U - V)$ and each row of P^π is a probability vector, so $P^\pi(U-V) \le U - V$. Theorem 1.4 gives the rate, and $V^\pi = B^\pi V^\pi$ by Theorem 1.1. □

Intuition

Each sweep costs only $O(|\cdot|^2)$ (one matrix–vector product) and the error shrinks by γ per sweep. With $\gamma = 0.9$ the error falls below 1% of its start after about 44 sweeps. A discount close to 1 makes the future matter more and the iteration slower.

1.6 How to Find the Optimal Policy?8

What we really want is π^*. For that use Bellman *optimality* and its operator, which acts on action-value functions $Q : \times \to$ and returns another such function:

$$Q(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)} \Big[\max_{a'} Q(s', a') \Big].$$

This operator is a γ-contraction in sup-norm (L04, Theorem 4), with Q^* as unique fixed point.

1.7 Value Iteration & Optimal Policy9

Since $Q^* = Q^*$, initialise $Q_0 \le 1/(1-\gamma)$ (typically $Q_0 = 0$) and iterate, for all states and actions, $Q_{i+1} = Q_i$. Then

$$Q_{i+1} - Q^* = Q_i - Q^* \le \gamma Q_i - Q^* \le \gamma^{i+1} Q_0 - Q^*.$$

Because $\pi^*(s) = \arg\max_a Q^*(s,a)$ and $Q_i \approx Q^*$, we read off the policy $\pi_i(s) = \arg\max_a Q_i(s,a)$.

1.8 Another Note on Value Iteration10

Q_t approximates Q^* and the policy π_t is computed from Q_t. However, Q_t is in general *different from* Q^{π_t} until we are close to Q^*: for instance Q_0 is just an initial guess, which may correspond to no policy at all.

Intuition

Value iteration outputs numbers that need not be the value of anything real. Concretely, since $Q^\pi(s,a) \ge r(s,a)$ for every π, the all-zero guess $Q_0 \equiv 0$ is the Q-function of no policy as soon as one reward is positive. Policy iteration, introduced next, fixes this: its intermediate objects *are* values of actual policies.

2 Policy Iteration12–20

Policy iteration (PI) outputs a *policy* at every iteration, $\{\pi_0, \pi_1, \ldots, \pi_T\}$, whereas value iteration outputs value estimates. We start from an arbitrary guess π_0 (deterministic or stochastic) and

alternate two steps. For simplicity, and to forget about approximation errors, the evaluation is assumed exact.

[Policy iteration] Start from any policy π_0. For $t = 0, 1, 2, \ldots$:

1. **Policy evaluation.** Compute $Q^{\pi_t}(s, a)$ for all s, a, e.g. $V^{\pi_t} = (I - \gamma P^{\pi_t})^{-1} r^{\pi_t}$ and then $Q^{\pi_t}(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[V^{\pi_t}(s')]$.
2. **Policy improvement.** $\pi_{t+1}(s) \in \arg\max_a Q^{\pi_t}(s, a)$ for every s, *keeping* $\pi_t(s)$ *whenever it is among the maximisers.*

Stop when $\pi_{t+1} = \pi_t$.

The slides' reminder $Q^{\pi}(s_t, a) = r_t + \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^{\pi}(s')]$ explains the remark in step 1: it is convenient to compute V first and obtain Q from it. Unlike value iteration, the quantity produced is the Q-function of an actual policy.

Intuition

The tie-breaking clause in step 2 is not an implementation detail. If two actions have equal value, an arbitrary choice may swap them back and forth forever, producing policies that are worth the same (take one state with two actions, both with the same reward and a self-loop: every swap is allowed, none helps). Keeping the current action whenever it is a maximiser guarantees that any change of policy is a *strict* improvement somewhere, which is what termination (Section 3.5) relies on.

Example 2.1 (A two-state MDP, solved by hand)**.** States A, B with actions *stay* and *go*, deterministic moves, $\gamma = 0.9$. Staying in A gives reward 0.5, staying in B gives reward 1; *go* gives reward 0 and switches state.

	$V^{\pi_t}(A)$	$V^{\pi_t}(B)$	Q^{π_t} at A (stay, go)	Q^{π_t} at B (stay, go)
$\pi_0 = (\text{stay}, \text{stay})$	5	10	(5, **9**)	(**10**, 4.5)
$\pi_1 = (\text{go}, \text{stay})$	9	10	(8.6, **9**)	(**10**, 8.1)

For π_0: $V(A) = 0.5/(1 - 0.9) = 5$, $V(B) = 1/(1 - 0.9) = 10$. Improvement switches A to *go* ($9 > 5$) and keeps B on *stay* ($10 = V(B)$). For π_1: $V(B) = 10$, $V(A) = 0 + 0.9 \cdot 10 = 9$, and no action beats the current one, so PI stops with $\pi^* = (\text{go}, \text{stay})$ and $V^* = (9, 10)$ after a single improvement.

Intuition

Read the table as "what would I get if I deviated once and then behaved as before?". At A under π_0, deviating to *go* is worth 9 against the current 5, so we take it; the new policy then makes B's stay even more attractive than before. The algorithm only ever moves to better policies, which is the content of the next section.

3 Properties of Policy Iteration21–35

Two properties are claimed: *monotonic improvement*, $Q^{\pi_{t+1}} \geq Q^{\pi_t}$ for all s, a, and *convergence*, $V^{\pi_t} - V^* \leq \gamma^t V^{\pi_0} - V^*$. (A third, finite termination, appears in Section 3.4.)

Intuition

Two requirements pull against each other: a value function should be consistent with the policy, and the policy should be consistent with the value function. Evaluation enforces the first, greedy improvement the second. They are satisfied simultaneously only at the optimum, and each alternation moves closer.

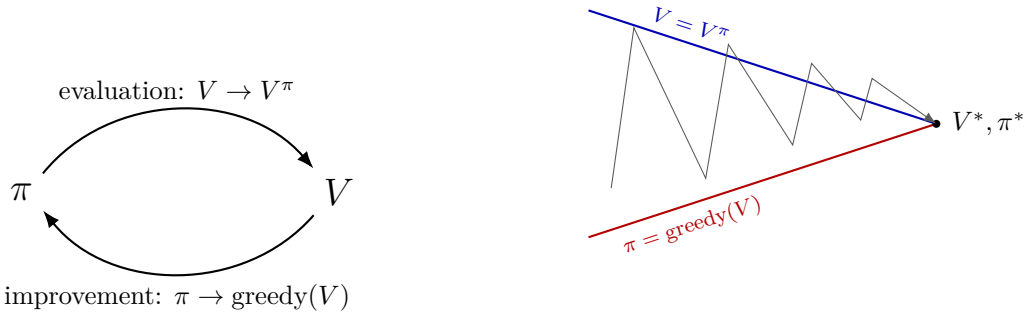


Figure 54: Evaluation and improvement alternate; the pair of iterates zig-zags between "V is the value of π" and "π is greedy for V" and meets only at (V^*, π^*) (original redrawing of the standard picture).

3.1 Monotonic Improvement23–30

Theorem 3.1 (Monotonic improvement)**.** *Let* $\pi_{t+1}(s) \in \arg\max_a Q^{\pi_t}(s,a)$. *Then* $Q^{\pi_{t+1}}(s,a) \geq Q^{\pi_t}(s,a)$ *for all* s, a, *and hence* $V^{\pi_{t+1}} \geq V^{\pi_t}$ *pointwise.*

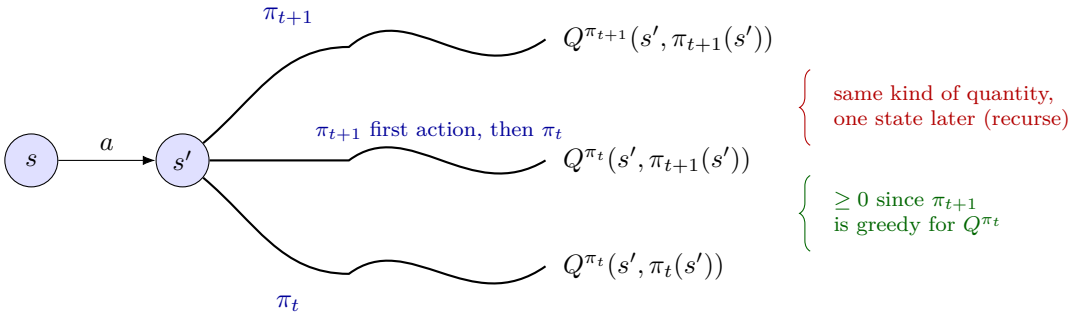


Figure 55: The proof of Theorem 3.1 inserts an intermediate policy that plays the new action once and then reverts to the old policy.

Proof. Both Q-functions share the same immediate reward, so by Theorem 1.1 the difference $D := Q^{\pi_{t+1}} - Q^{\pi_t}$ satisfies

$$D(s,a) = \gamma\, \mathbb{E}_{s' \sim p(\cdot|s,a)} \Big[Q^{\pi_{t+1}}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_t(s')) \Big].$$

Inside the bracket the two terms differ in two ways at once (which policy is valued, which action is taken). Add and subtract the value of the new action under the old policy, $Q^{\pi_t}(s', \pi_{t+1}(s'))$:

$$\underbrace{Q^{\pi_{t+1}}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_{t+1}(s'))}_{\text{same action, new policy} = D(s', \pi_{t+1}(s'))} + \underbrace{Q^{\pi_t}(s', \pi_{t+1}(s')) - Q^{\pi_t}(s', \pi_t(s'))}_{\text{same policy, new action} \geq 0}.$$

The second term is non-negative because π_{t+1} maximises Q^{π_t}. Therefore $D(s,a) \geq \gamma \mathbb{E}_{s'}[D(s', \pi_{t+1}(s'))]$. This is the same inequality one state later, so we may recurse: since expectations preserve inequalities and $\gamma \geq 0$, after k unrollings

$$D(s,a) \;\geq\; \gamma^k\, \mathbb{E}\big[D(s_k, \pi_{t+1}(s_k))\big] \;\geq\; -\gamma^k D \;\geq\; -\frac{\gamma^k}{1-\gamma},$$

where the last step uses boundedness of Q (L03, Lemma 3: $|D| \leq 1/(1-\gamma)$). Letting $k \to \infty$ gives $D \geq 0$. For the values, use greediness once more:

$$V^{\pi_{t+1}}(s) = Q^{\pi_{t+1}}(s, \pi_{t+1}(s)) \;\geq\; Q^{\pi_t}(s, \pi_{t+1}(s)) \;\geq\; Q^{\pi_t}(s, \pi_t(s)) = V^{\pi_t}(s). \qquad \square$$

Intuition

Greedy improvement says: "if I deviate *once* to the new action and then go back to the old behaviour, I do at least as well." The theorem upgrades this to: "if I deviate *every time*, I still do at least as well." The bridge is the recursion: the gap between the new and old policy after the first step has the same shape as the gap we started with, only discounted by γ, and after many steps the discount makes the leftover negligible. Hence PI "only makes progress": no iteration can make any state worse.

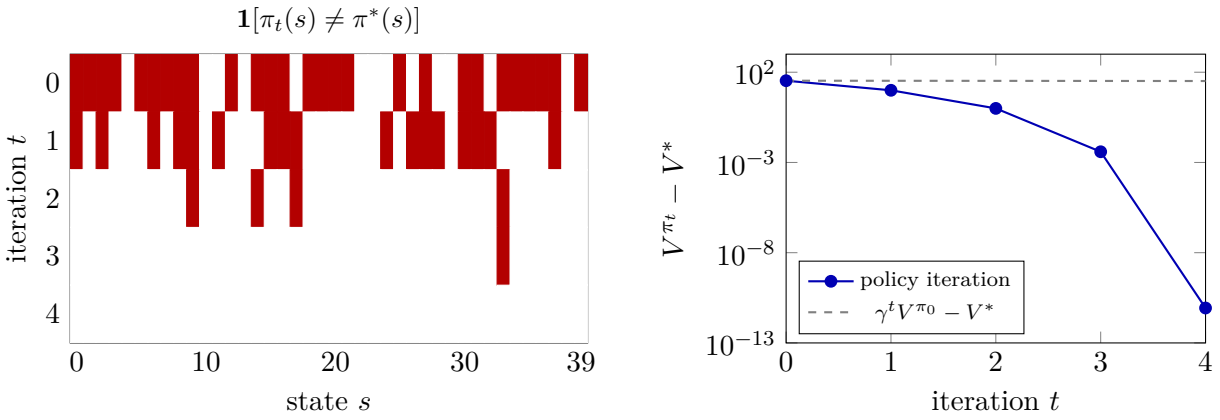


Figure 56: Policy iteration on a randomly generated MDP with $|| = 40$, $|| = 3$, $\gamma = 0.99$ (original run, exact evaluation, keep-current tie-breaking). Left: states whose action still differs from π^* (27, 17, 4, 1, 0 of 40 over $t = 0, \ldots, 4$). Right: the sup-norm error falls far faster than the guaranteed rate; the last value ($\approx 10^{-11}$) is numerical zero.

3.2 Convergence31

Corollary 3.2 (Convergence, termination, optimality)**.** *(a)* $V^{\pi_t} - V^* \leq \gamma^t V^{\pi_0} - V^*$*;*

(b) with finite , and the tie-breaking rule of Algorithm 2, PI terminates after at most $||^{||}$ *iterations;*

(c) when it stops, the policy is optimal.

Proof of (a). Since π_{t+1} is greedy with respect to V^{π_t}, applying the optimality operator to V^{π_t} is the same as applying the expectation operator of π_{t+1}: $V^{\pi_t} = B^{\pi_{t+1}} V^{\pi_t}$. Using $V^* = V^*$ and $B^{\pi_{t+1}} V^{\pi_{t+1}} = V^{\pi_{t+1}}$,

$$0 \leq V^* - V^{\pi_{t+1}} = (V^* - V^{\pi_t}) + (B^{\pi_{t+1}} V^{\pi_t} - B^{\pi_{t+1}} V^{\pi_{t+1}}).$$

The first difference is at most $\gamma V^* - V^{\pi_t}$ by the contraction property of (L04, Theorem 4). The second equals $\gamma P^{\pi_{t+1}}(V^{\pi_t} - V^{\pi_{t+1}}) \leq 0$ componentwise, because $V^{\pi_{t+1}} \geq V^{\pi_t}$ (Theorem 3.1) and $P^{\pi_{t+1}}$ has non-negative entries. Hence $V^* - V^{\pi_{t+1}} \leq \gamma V^* - V^{\pi_t}$; iterate. Parts (b) and (c) are proved in Sections 3.4 and 3.5. □

Intuition

Each PI step contracts the distance to V^* at least as strongly as one value-iteration step. Two facts make this work: greedy improvement means that the policy's own Bellman operator coincides with the optimality operator at V^{π_t}, and monotonicity tells us the new policy's value sits *above* the old one, so the leftover term has a favourable sign. The bound is a worst case; Figure 62 in the summary shows that in practice PI is much faster in iterations.

3.3 Complexity32

One iteration costs $O(|\mathcal{|}^3 + |\mathcal{|}^2|\mathcal{|})$.

Intuition

Evaluation is a linear solve, $O(|\mathcal{|}^3)$. Improvement needs $Q^{\pi_t}(s,a)$ for all $|\mathcal{||}\mathcal{|}$ pairs, each a sum over $|\mathcal{|}$ successor states, hence $O(|\mathcal{|}^2|\mathcal{|})$, followed by an $O(|\mathcal{||}\mathcal{|})$ argmax. PI therefore trades expensive iterations for very few of them.

3.4 Is there a max number of iterations of policy iteration?33–34

Yes: $|\mathcal{|}^{|\mathcal{|}}$, since that is the number of deterministic stationary policies and no policy can appear twice, provided ties in the argmax are broken in favour of the action currently taken.

Proof of Corollary 3.2(b). Suppose $\pi_{t+1} \neq \pi_t$, so $\pi_{t+1}(s) \neq \pi_t(s)$ at some state s. The tie-breaking rule kept $\pi_t(s)$ whenever it was a maximiser, so at that s it was not one:

$$Q^{\pi_t}(s, \pi_{t+1}(s)) > Q^{\pi_t}(s, \pi_t(s)) = V^{\pi_t}(s).$$

By the first chain of inequalities in the proof of Theorem 3.1, $V^{\pi_{t+1}}(s) \geq Q^{\pi_t}(s, \pi_{t+1}(s))$, hence $V^{\pi_{t+1}}(s) > V^{\pi_t}(s)$, while $V^{\pi_{t+1}} \geq V^{\pi_t}$ everywhere. So every change of policy raises the value *strictly* at some state and never lowers it anywhere: no policy can reappear. There are at most $|\mathcal{|}^{|\mathcal{|}}$ deterministic stationary policies, so the loop stops. □

Intuition

Think of the vector V^{π_t} as a staircase that never goes down and, whenever the policy changes, goes strictly up somewhere. A staircase over a finite set of policies cannot be climbed forever. The tie-breaking rule is what rules out "flat" changes. The count $|\mathcal{|}^{|\mathcal{|}}$ is astronomically pessimistic; Figure 56 (40 states, 3 actions, 3^{40} policies) shows termination after four improvements.

3.5 When do we stop?35

When the policy does not change anymore for any state.

Proof of Corollary 3.2(c). $\pi_{t+1} = \pi_t$ means the improvement step leaves the policy unchanged, so π_t is greedy for V^{π_t} and $V^{\pi_t} = B^{\pi_t} V^{\pi_t} = V^{\pi_t}$. Thus V^{π_t} is a fixed point of , and by L03, Theorem 10, $V^{\pi_t} = V^*$. □

Intuition

The stopping rule is a certificate: "no single-state deviation helps" is the same as the Bellman optimality equation, and that equation has only one solution. So a policy that is a fixed point of the evaluate–improve loop cannot be merely locally good; it is globally optimal.

4 Advantage Function36–38

Definition 4.1 (Advantage)**.** $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$: the value of taking a in s minus the average value of the state under π. It satisfies $\mathbb{E}_{a\sim\pi(\cdot|s)}[A^\pi(s,a)] = 0$.

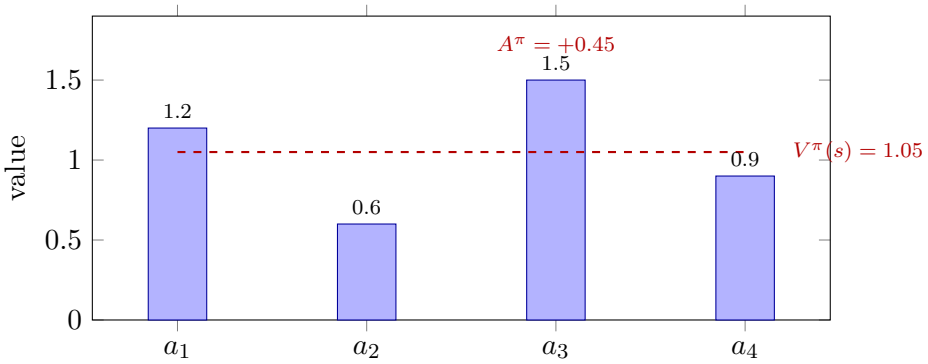


Figure 57: Bars are $Q^\pi(s,a)$ at a fixed state for a uniform policy over four actions; the dashed line is $V^\pi(s)$, the average. The advantage is the signed height above the line (here $+0.15, -0.45, +0.45, -0.15$, summing to zero). Numbers are an original illustration.

Intuition

The advantage is a "better or worse than usual" score. $Q^\pi(s,a)$ alone mixes how good the state is with how good the action is; subtracting $V^\pi(s)$, the average of the actions π actually takes, leaves only the part due to the action. In the picture a_3 and a_1 beat the average, a_2 and a_4 do not, and by construction the advantages average to 0 under π: a policy cannot be better than itself on average.

4.1 Policy Iteration with Advantage Function38

The improvement step can be written as $\pi_{t+1}(s) \in \arg\max_a A^{\pi_t}(s,a)$. It selects the same actions as $\arg\max_a Q^{\pi_t}(s,a)$ because $V^{\pi_t}(s)$ does not depend on a.

Intuition

Subtracting a constant that does not depend on the action never changes which action is largest. The advantage form is useful because it makes the sign of the improvement visible: a state can be improved exactly when some action has strictly positive advantage, and a policy is optimal when all advantages are ≤ 0. The next section uses it to *quantify* the improvement.

5 Performance Difference Lemma39–59

We know the new policy from PI is better than the old one, but by how much? Write $V^\pi(s_0) - V^{\pi'}(s_0) = ??$ for a start state s_0 and two policies.

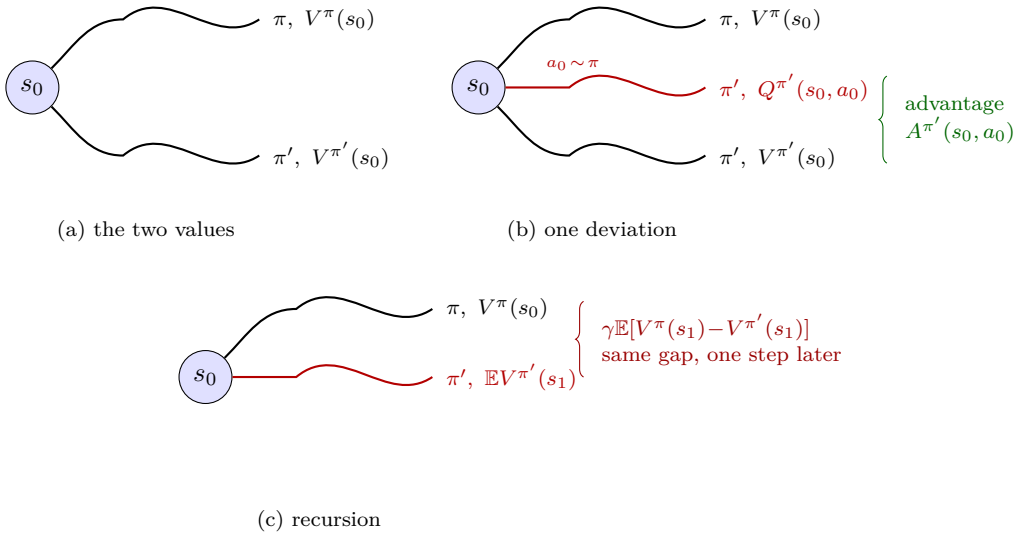
Occupancy measure. For a start state s_0 and policy π let $\pi s_0(s,a) = (1-\gamma)\sum_{t\ge 0}\gamma^t \Pr^\pi(s_t = s, a_t = a \mid s_0)$ and $\pi s_0(s) = \sum_a \pi s_0(s,a)$: the discounted fraction of time spent in each state (and state–action pair). It is a probability distribution. By L03, Lemma 5, $\mathbb{E}^\pi\big[\sum_t \gamma^t f(s_t,a_t)\big] = \frac{1}{1-\gamma}\mathbb{E}_{(s,a)\sim\pi s_0}[f(s,a)]$.

Theorem 5.1 (Performance difference lemma)**.** *For any two policies π, π' and any start state s_0,*

$$V^\pi(s_0) - V^{\pi'}(s_0) = \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim\pi s_0}\Big[\mathbb{E}_{a\sim\pi(\cdot|s)}\big[Q^{\pi'}(s,a) - V^{\pi'}(s)\big]\Big] = \frac{1}{1-\gamma}\,\mathbb{E}_{(s,a)\sim\pi s_0}\big[A^{\pi'}(s,a)\big].$$

In words: the advantage against the old *policy π', averaged over the distribution induced by the* new *policy π.*

Remark. The statement holds for *any* ordered pair, so π is just the name of the policy whose value comes first. The slides call π the new and π' the old policy; the book chapter writes the same identity with the two names exchanged. Appendix A applies it with the roles reversed.



(a) the two values

(b) one deviation

(c) recursion

Figure 58: The structure of the proof: the total gap splits into a one-step advantage term and the same gap one step later.

Proof (recursive, as in the slides). Write $\Delta(s) = V^{\pi}(s) - V^{\pi'}(s)$. Add and subtract the value of "play π's first action, then continue with π'":

$$\Delta(s_0) = \Big(V^{\pi}(s_0) - \mathbb{E}_{a_0\sim\pi}\big[r(s_0,a_0) + \gamma\mathbb{E}_{s_1}V^{\pi'}(s_1)\big]\Big) + \Big(\mathbb{E}_{a_0\sim\pi}\big[r(s_0,a_0) + \gamma\mathbb{E}_{s_1}V^{\pi'}(s_1)\big] - V^{\pi'}(s_0)\Big).$$

Expanding $V^{\pi}(s_0) = \mathbb{E}_{a_0\sim\pi}[r(s_0,a_0) + \gamma\mathbb{E}_{s_1}V^{\pi}(s_1)]$, the rewards cancel in the first bracket, leaving $\gamma\mathbb{E}_{a_0\sim\pi}\mathbb{E}_{s_1}[V^{\pi}(s_1) - V^{\pi'}(s_1)] = \gamma\mathbb{E}[\Delta(s_1)]$. In the second bracket, $r(s_0,a_0) + \gamma\mathbb{E}_{s_1}V^{\pi'}(s_1) = Q^{\pi'}(s_0,a_0)$ by definition, so it equals $\mathbb{E}_{a_0\sim\pi}[Q^{\pi'}(s_0,a_0) - V^{\pi'}(s_0)] = \mathbb{E}_{a_0\sim\pi}[A^{\pi'}(s_0,a_0)]$. Hence

$$\Delta(s_0) = \mathbb{E}_{a_0\sim\pi}\big[A^{\pi'}(s_0,a_0)\big] + \gamma\,\mathbb{E}_{a_0\sim\pi,\,s_1}\big[\Delta(s_1)\big].$$

This is the same relation one state later, so recursing T times gives $\Delta(s_0) = \mathbb{E}^{\pi}\big[\sum_{t<T}\gamma^t A^{\pi'}(s_t,a_t)\big] + \gamma^T\mathbb{E}[\Delta(s_T)]$. The remainder is at most $\gamma^T/(1-\gamma) \to 0$ (boundedness, L03, Lemma 3), and by the occupancy identity the infinite discounted sum equals $\frac{1}{1-\gamma}\mathbb{E}_{(s,a)\sim\pi s_0}[A^{\pi'}(s,a)]$. □

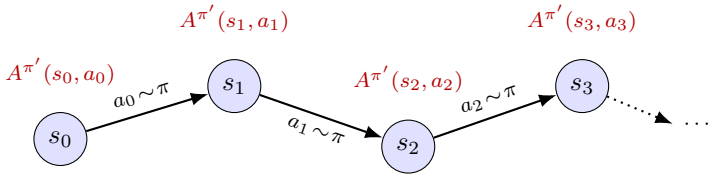
Second proof (telescoping). For any bounded V and any trajectory, $\sum_{t\ge0}\gamma^t(\gamma V(s_{t+1}) - V(s_t)) = -V(s_0)$ pathwise: consecutive terms cancel and the tail vanishes. Apply it with $V = V^{\pi'}$ along trajectories of π:

$$V^{\pi}(s_0) - V^{\pi'}(s_0) = \mathbb{E}^{\pi}\Big[\sum_{t\ge0}\gamma^t\big(r(s_t,a_t) + \gamma V^{\pi'}(s_{t+1}) - V^{\pi'}(s_t)\big)\Big].$$

Conditioning each term on (s_t,a_t), $\mathbb{E}[r + \gamma V^{\pi'}(s_{t+1}) \mid s_t,a_t] = Q^{\pi'}(s_t,a_t)$ (Theorem 1.1), so the bracket has conditional expectation $A^{\pi'}(s_t,a_t)$; the occupancy identity finishes. □

Intuition

Imagine π and π' as two routes and the discounted value as the money collected. Switch from π' to π one step at a time: at step t you compare "take π's action here, then follow π'" with "just follow π'". That difference is exactly the advantage $A^{\pi'}(s_t, a_t)$. Summing these one-step gains, discounted, along the route that π really takes gives the total gap. This is why the weighting πs_0 belongs to the *new* policy: the gains must be collected where π goes, not where π' used to go.



gaps $\gamma^t A^{\pi'}(s_t, a_t)$ collected along π's trajectory; their expectation, rescaled by $1/(1-\gamma)$, is $\mathbb{E}_{(s,a)\sim\pi s_0} A^{\pi'}(s, a)$

Figure 59: "Average advantage value": the lemma weights the old policy's advantage by how often the new policy visits each state–action pair.

Remark (Policy iteration as a special case). Take $\pi = \pi_{t+1}$ and $\pi' = \pi_t$. Because $\pi_{t+1}(s)$ maximises $Q^{\pi_t}(s, \cdot)$, $A^{\pi_t}(s, \pi_{t+1}(s)) = \max_a A^{\pi_t}(s, a) \geq \mathbb{E}_{a\sim\pi_t} A^{\pi_t}(s, a) = 0$ at every s. Theorem 5.1 then gives $V^{\pi_{t+1}}(s_0) - V^{\pi_t}(s_0) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim\pi_{t+1}s_0}[\max_a A^{\pi_t}(s, a)] \geq 0$: a second proof of $V^{\pi_{t+1}} \geq V^{\pi_t}$ that also says *how much* the value rises, namely by the average best-action advantage under the new policy's own state distribution.

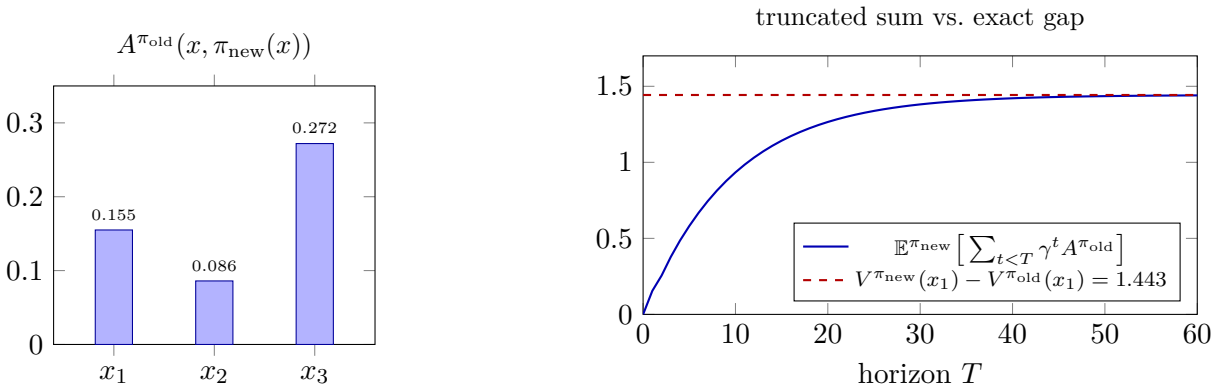


Figure 60: Numerical check of Theorem 5.1 on an original 3-state, 2-action MDP ($\gamma = 0.9$; π_{old} uniform, π_{new} greedy for $Q^{\pi_{\text{old}}}$, start x_1). Left: the advantage of the new action at each state (all ≥ 0, as in Remark 5). Right: the discounted sum of these advantages along π_{new}'s trajectories, computed exactly with powers of $P^{\pi_{\text{new}}}$, converges to the true value gap $5.295 - 3.852 = 1.443$.

6 We Did Dynamic Programming![60–62]

Dynamic programming (DP) solves a complex problem by breaking it into subproblems: solve the subproblems, then combine their solutions. It applies when two properties hold:

- *Optimal substructure*: optimality exists and the optimal solution decomposes into subproblems;

- *Overlapping subproblems*: subproblems recur many times, so their solutions can be cached and reused.

MDPs satisfy both, thanks to the Bellman equation.

Intuition

The Bellman equation *is* the decomposition: the value at s is built from the values at the successor states (optimal substructure), and the same successor values are reused by every predecessor and every iteration (overlapping subproblems; the value table is the cache). We applied DP for **planning**, because we assumed the transition probabilities p are known.

Problem	**Bellman equation used**	**Algorithm**
Prediction	Bellman expectation equation	Iterative policy evaluation
Control	Bellman expectation equation + greedy policy improvement	Policy iteration
Control	Bellman optimality equation	Value iteration

7 Primal Linear Program63–66

As an alternative to VI and PI, write the Bellman optimality equation $V(s) = \max_a\{r(s,a) + \gamma\mathbb{E}_{s'}V(s')\}$ as a linear program.

Proposition 7.1 (Primal linear program)**.** *V^* is the solution of*

$$\min_V \sum_s V(s) \quad \textit{subject to} \quad V(s) \geq r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)V(s') \quad \textit{for all } s,a.$$

Proof. The constraints say exactly $V \geq V$ componentwise. The operator is monotone (if $U \geq V$ then $U \geq V$, since both the expectation and the maximum preserve order), so any feasible V satisfies

$$V \;\geq\; V \;\geq\; {}^2V \;\geq\; \cdots \;\geq\; \lim_k {}^kV = V^*,$$

the limit being V^* by L04, Theorem 2. Hence every feasible point dominates V^* componentwise. Moreover V^* is itself feasible (L03, Theorem 8: $V^* = V^*$). Minimising the sum therefore selects exactly V^*. □

Intuition

The feasible points are exactly the "over-estimates" of the optimal value ($V \geq V$ forces $V \geq V^*$), and V^* is the *smallest* of them in every coordinate simultaneously. So any objective that rewards smaller values, such as the sum, picks V^*. In the example, $V(B)$ cannot be below 10 (it can always stay for reward 1 forever), and then $V(A)$ cannot be below $0.9 \cdot 10 = 9$ (it can always go to B). Using an LP solver we get V^*, but this approach is not used a lot in practice.

Dual linear program66. There is also a dual linear program that finds the solution directly in policy space. The slides only announce it; as standard background (not developed in the slides), its variables can be read as discounted state–action occupancy measures, and maximising expected reward over those measures that satisfy the flow constraints recovers an optimal policy.

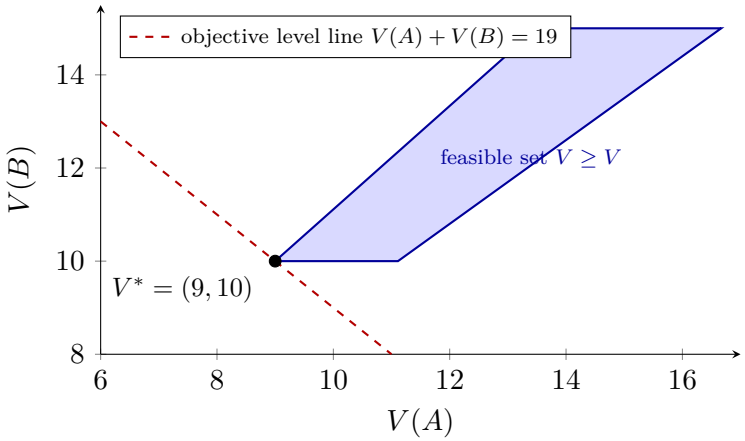


Figure 61: The LP of Proposition 7.1 for the two-state MDP of Example 2.1. Constraints: $V(A) \geq 5$, $V(A) \geq 0.9V(B)$, $V(B) \geq 10$, $V(B) \geq 0.9V(A)$. The feasible set is the shaded wedge (unbounded upwards); its lowest corner, $V^* = (9, 10)$, is where $V(B) \geq 10$ and $V(A) \geq 0.9V(B)$ are both tight.

8 Summary

Comparison of the methods

Method	**Equation used**	**Output**	**Cost / iteration**	**Guarantee**
Exact policy evaluation	$V^\pi = (I - \gamma P^\pi)^{-1} r^\pi$	V^π exactly	$O(\|\|^3)$ (once)	exact
Iterative policy evaluation	$V_{i+1} = r^\pi + \gamma P^\pi V_i$	$\approx V^\pi$	$O(\|\|^2)$	error $\leq \gamma^i V_0 - V^\pi$
Value iteration	$Q_{i+1} = Q_i$	$Q_i \approx Q^*$, policy read off by argmax (not the Q of a real policy)	$O(\|\|^2\|\|)$ (derived)	error $\leq \gamma^i Q_0 - Q^*$
Policy iteration	evaluate Q^{π_t}, then greedy improvement	a policy π_t and its true Q^{π_t}	$O(\|\|^3 + \|\|^2\|\|)$	monotone; error $\leq \gamma^t V^{\pi_0} - V^*$; stops after $\leq \|\|^{\|\|}$ steps at π^*
Primal LP	$\min \sum_s V(s)$ s.t. $V \geq V$	V^*	one LP solve	exact; rarely used in practice

Key take-aways

1. **Policy iteration alternates exact evaluation and greedy improvement**, and unlike value iteration its iterates Q^{π_t} are values of real policies.
2. **Greedy improvement never hurts**: "deviate once, then revert" being no worse implies "deviate always" is no worse (Theorem 3.1); the performance difference lemma makes the gain explicit as an average advantage under the new policy's distribution.
3. **Convergence is both geometric and finite**: $V^{\pi_t} - V^* \leq \gamma^t V^{\pi_0} - V^*$, and with keep-current tie-breaking at most $\|\|^{\|\|}$ iterations; stopping means $V^\pi = V^\pi$, hence optimality.
4. **Few but expensive iterations**: $O(\|\|^3 + \|\|^2\|\|)$ each, versus cheap but numerous value-iteration sweeps.

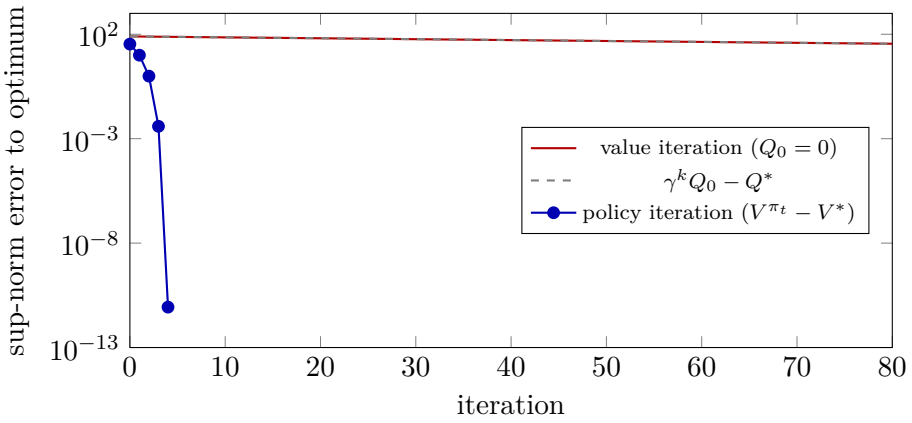


Figure 62: Value iteration versus policy iteration on the same random MDP as before ($|| = 40$, $|| = 3$, $\gamma = 0.99$). VI follows the geometric rate γ^k (about 80 iterations only halve the error), whereas PI needs four improvements. Per-iteration costs differ (cubic for PI, quadratic for VI), so iteration counts alone do not decide which is cheaper overall. Original numerical experiment.

5. **All of this is dynamic programming for planning** (the model is known); the primal LP over $V \geq V$ is a valid but rarely used alternative.

A Supplementary material (not on the slides)

This appendix contains formal material from the book chapter for which there is no corresponding slide.

Proposition A.1 (Optimal in a box)**.** *Let $_0 \subset$ be closed under a set of actions $_0$, meaning that no action in $_0$ leads from a state of $_0$ to a state outside it, and let $\pi^\dagger$ be the policy returned by policy iteration on the restricted MDP $(_{0,0}, p, r, \gamma)$. Then $\pi^\dagger$ is optimal among policies that use only actions in $_0$ while in $_0$, and none of those policies ever leaves $_0$.*
Moreover, $\pi^\dagger$ is a policy on $_0$ and nothing more: to compare it with a policy that leaves $_0$ we must first say what it does outside. Extend it by any rule whatever and call the result $\pi^\dagger$ as well. By Theorem 5.1 (applied with $\pi := \pi'$ the leaving policy and $\pi' := \pi^\dagger$), its shortfall against a policy π' that leaves with positive probability is

$$V^{\pi'}(s_0) - V^{\pi^\dagger}(s_0) = \frac{1}{1-\gamma}\,\mathbb{E}_{(s,a)\sim\pi' s_0}\left[A^{\pi^\dagger}(s,a)\right],$$

an expectation whose support, for $\gamma > 0$, reaches outside $_0$, where the value of $\pi^\dagger$ is whatever the extension made it and policy iteration chose nothing.

Proof. The restricted problem is an MDP, so Corollary 3.2 applies to it and $\pi^\dagger$ is optimal within it. Since $_0$ is closed under $_0$, a trajectory that starts in $_0$ and follows $\pi^\dagger$ stays there, so the extension changes neither $V^{\pi^\dagger}$ on $_0$ nor the optimality claim on $_0$. It does fix $V^{\pi^\dagger}$ and $A^{\pi^\dagger}$ elsewhere, which is what the identity needs. That identity is Theorem 5.1 applied to the pair $(\pi', \pi^\dagger)$, and for $\gamma > 0$ the support of $\pi' s_0$ includes states outside $_0$ whenever π' leaves with positive probability (a state visited at time t carries weight proportional to $\gamma^t > 0$). □

Intuition

Running policy iteration inside a closed region gives the best possible behaviour *for that region's rules*. It would be wrong to read this as "best overall": a policy that steps out of

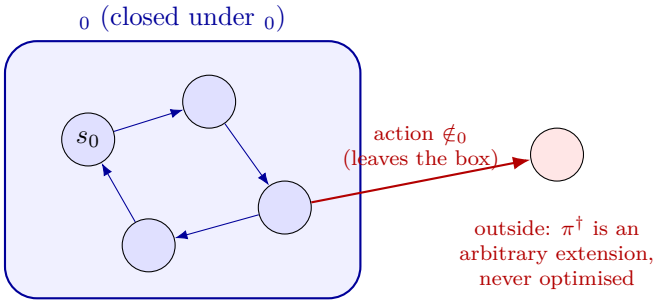


Figure 63: Optimality inside the box says nothing about policies that leave it: the performance difference lemma weights the advantage of $\pi^\dagger$ by the leaving policy's occupancy, which reaches the outside.

the box visits states where $\pi^\dagger$ was never improved, and the performance difference lemma shows precisely that those states contribute to the comparison through $\pi' s_0$. Restricting the problem restricts the guarantee.

Part V

Lecture 4

Abstract

These notes follow the *order of the slides*: each section/subsection carries the (near-verbatim) slide title and, in parentheses, the slide range. The formal statements and proofs of the two companion PDFs are merged at the point where the slides introduce the topic. After each formal result there is a blue *Intuition* box. Material that appears only in the PDFs and has no slide is collected in Appendix A. Where a figure of the slides could not be redrawn (animated GIFs), a red dashed placeholder says which image to paste and where it comes from.

1 Recap *(slides 2–10)*

The deck opens with a recap of the dynamic-programming algorithms for (infinite-horizon, discounted) MDPs, to set the stage for the control setting.

1.1 Policy Iteration *(slides 3–6)*

Policy iteration outputs a *policy* at every iteration, $\{\pi_0, \pi_1, \pi_2, \ldots, \pi_T\}$, whereas value iteration outputs *values*. The procedure is:

1. start with a random guess π_0 (deterministic or stochastic);
2. for $t = 0, \ldots, T$: do *policy evaluation*, i.e. compute $Q^{\pi_t}(s, a)$ for all (s, a); then *policy improvement*, $\pi_{t+1}(s) \in \arg\max_a Q^{\pi_t}(s, a)$.

The algorithm only makes progress: the performance of the policy is monotonically non-decreasing.

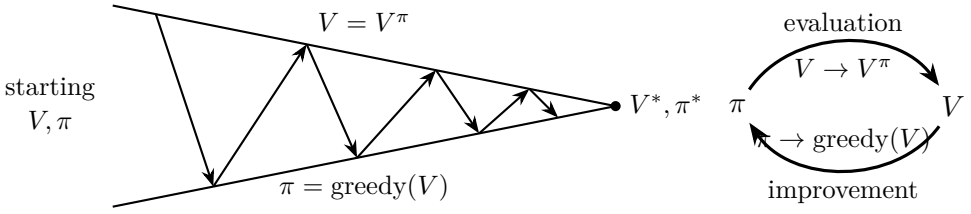


Figure 64: Redrawn from slide 4 (credits on the slide: David Silver). Left: policy iteration alternates two projections and spirals into (V^*, π^*). Right: the evaluation/improvement cycle.

Properties (slides 5–6). Is there a maximum number of iterations? Yes: $|A|^{|S|}$, the number of deterministic policies. Since each improvement step is monotone, a policy can appear in at most one round unless it is already optimal. *Stopping rule:* stop when the policy does not change any more in any state.

Intuition

Policy iteration is a "coordinate ascent" between two objects: given a policy, you can compute exactly how good it is (evaluation); given values, you can read off a policy that is at least as good (greedy improvement). Because each round can only help, and there are finitely many policies, you cannot cycle: you must land on the optimum.

1.2 We Did Dynamic Programming! *(slides 7–8)*

Dynamic programming applies when there is (i) **optimal substructure** (the optimal solution decomposes into subproblems) and (ii) **overlapping subproblems** (subproblems recur and their solutions can be cached). MDPs have both, thanks to the Bellman equation. We used DP for *planning*: we assumed the transition probabilities of the MDP are known.

Problem	Bellman equation	Algorithm
Prediction	Bellman expectation equation	Iterative policy evaluation
Control	Bellman expectation eq. + greedy improvement	Policy iteration
Control	Bellman optimality equation	Value iteration

Table 2: Table of slide 8 (credits: David Silver), retyped.

1.3 Primal Linear Program *(slides 9)*

As an alternative to VI and PI, the Bellman optimality equation can be written as a linear program: minimise $\sum_s V(s)$ subject to the Bellman inequalities $V(s) \geq r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) V(s')$ for all (s,a). The optimal V^* is the pointwise smallest feasible vector, hence the unique minimiser. (The slide shows the objective "$\min V(s)$" and the constraints as images; this is the standard form.)

Intuition

Any V that satisfies all Bellman inequalities is an *upper bound* on what any policy can achieve; the best upper bound that is still valid is V^* itself. A linear program finds it without iterating Bellman updates.

2 Finite-Horizon MDPs *(slides 11–25)*

2.1 Definition and time-dependent policies *(slides 11–15)*

The deck uses a slightly different formulation: (S, A, R, T, H, μ_0), with horizon H and initial-state distribution $s_0 \sim \mu_0$. (The companion notes write the same object as (S, A, p, r, H, μ), $H \geq 1$.)

Definition 2.1 (Finite-horizon MDP)**.** A finite-horizon MDP is a tuple (S, A, p, r, H, μ) with horizon $H \geq 1$ and initial distribution $s_0 \sim \mu$. An *episode* is $s_0, a_0, s_1, \ldots, s_{H-1}, a_{H-1}$, and there is *no discount factor*. Policies are *time-dependent*: $\pi = (\pi_0, \pi_1, \ldots, \pi_{H-1})$ with $a_h = \pi_h(s_h)$.

The slides stress (slides 13–15): the action may differ for the same state depending on t (e.g. "I could explore more at the beginning"), and this is *very common in control*.

Intuition

With a deadline, the *time remaining* is part of the situation. Think of an exam: with two hours left you explore a hard problem; with two minutes left you write down what you have. The same state ("this question is open") calls for different actions. A stationary policy cannot express that; a time-dependent one can. A concrete numerical example where this is *necessary* is in Appendix A.

2.2 Finite-Horizon MDPs — Example *(slides 16)*

The slide shows a wheeled robot that must follow a reference trajectory: state $(X(t), Y(t))$ (plus orientation), controls v (forward speed) and ω (turn rate), reference point $(X_r(t), Y_r(t))$. The cost penalises the distance to the moving reference over a finite horizon, which is a natural finite-horizon problem.

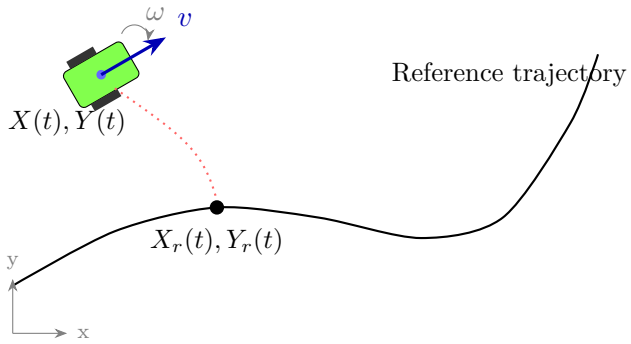
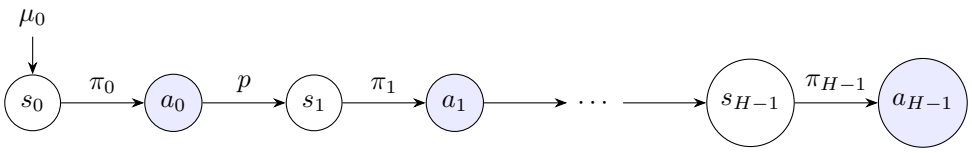


Figure 65: Redrawn from the example on slide 16: a robot with state (X, Y) and controls (v, ω) tracking a moving reference point.

2.3 Finite-Horizon MDP & Policy Interaction *(slides 17–21)*

The slides build the interaction step by step: $s_0 \sim \mu_0$; the policy picks a_0; the environment draws $s_1 \sim P(\cdot \mid s_0, a_0)$; the policy picks a_1; …; until the episode $(s_0, a_0, s_1, a_1, \ldots, s_{H-1}, a_{H-1})$ is complete.



2.4 Finite-Horizon MDP: V & Q *(slides 22–24)*

With no discount factor, for $s_h = s$, $a_\tau = \pi_\tau(s_\tau)$ and $s_{\tau+1} \sim P(\cdot \mid s_\tau, a_\tau)$:

$$V_h^\pi(s) = \mathbb{E}_\pi\Big[\sum_{\tau=h}^{H-1} r(s_\tau, a_\tau)\Big], \qquad Q_h^\pi(s,a) = r(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}\big[V_{h+1}^\pi(s')\big],$$

where in Q_h^π the first action is $a_h = a$ and afterwards π is followed. The optimal policy is the collection $\pi^* = \{\pi_0^*, \pi_1^*, \ldots, \pi_{H-1}^*\}$ (slide 26).

2.5 Finite-Horizon MDP: Bellman Equation *(slides 25)*

The Bellman equation links consecutive levels: $V_h^\pi(s) = Q_h^\pi(s, \pi_h(s))$ and $Q_h^\pi(s,a) = r(s,a) + \mathbb{E}_{s'}[V_{h+1}^\pi(s')]$, i.e. level h depends on level $h+1$ *only*.

Intuition

V_h is the "reward-to-go" with $H - h$ steps left. The Bellman equation says: reward-to-go = reward now + average reward-to-go from wherever you land. Level $h+1$ has already been paid for, so you only have to think one step ahead. This is precisely the structure that lets you compute everything by *walking backwards from the deadline*.

3 Finding the Optimal Policy *(slides 26–32)*

The slides note: this is an *easier problem than the infinite-horizon one*. We reason backwards in time and apply dynamic programming. At the last step $H-1$ there is no future, so $Q_{H-1}^*(s,a) = r(s,a)$; then we ask what happens at $H-2$, and so on (slides 27–32).

Algorithm 3.1 (Backward induction / finite-horizon value iteration)**.** 1. At the last step: $Q^*_{H-1}(s,a) = r(s,a)$, $\pi^*_{H-1}(s) \in \arg\max_a Q^*_{H-1}(s,a)$, $V^*_{H-1}(s) = \max_a Q^*_{H-1}(s,a)$.
2. For $h = H-2, H-3, \dots, 0$:

$$Q^*_h(s,a) = r(s,a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}\left[V^*_{h+1}(s')\right],$$
$$\pi^*_h(s) \in \arg\max_a Q^*_h(s,a), \qquad V^*_h(s) = \max_a Q^*_h(s,a).$$

Theorem 3.2 (Backward induction is exact (SKIPPED IN CLASS))**.** *Algorithm 3.1 returns $V^*_h = \max_\pi V^\pi_h$ for every h and every state, and $\pi^* = (\pi^*_0, \dots, \pi^*_{H-1})$ is optimal. The cost is $O(H|S|^2|A|)$ and there is no approximation error at any step.*

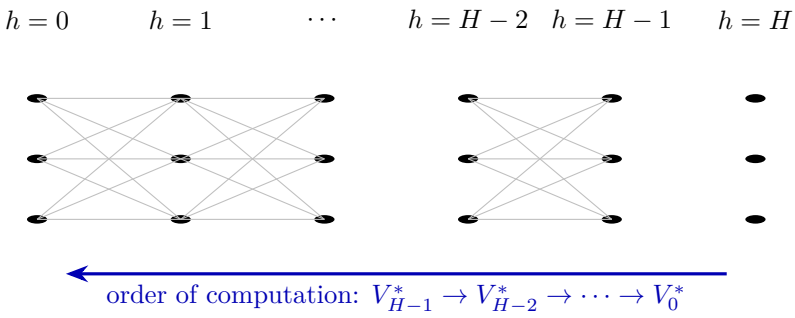
Proof. Backward induction on h. At $h = H-1$ no future remains, so $V^\pi_{H-1}(s) = r(s, \pi_{H-1}(s))$ for any π, and taking the maximum over a is optimal by inspection; the maximum is attained because A is finite.

Suppose V^*_{h+1} is the optimal value at level $h+1$. For any policy π, by the Bellman equation of the previous section,

$$V^\pi_h(s) = r(s,\pi_h(s)) + \mathbb{E}_{s'}[V^\pi_{h+1}(s')] \le r(s,\pi_h(s)) + \mathbb{E}_{s'}[V^*_{h+1}(s')] \le \max_a \left[r(s,a) + \mathbb{E}_{s'}[V^*_{h+1}(s')]\right],$$

the first inequality by the induction hypothesis and the second because the maximum dominates any particular action. The right-hand side is exactly $V^*_h(s)$ as computed by the algorithm, and it is attained by the policy that plays π^*_h now and behaves optimally afterwards, which exists by the induction hypothesis. Hence V^*_h is the optimal value and π^*_h attains it.

Cost. Each of the H levels evaluates $|S||A|$ pairs, each requiring an expectation over $|S|$ successors. □



Intuition

It is like computing the cheapest route backwards from the destination. At the end there is nothing to plan: pick the best immediate reward. One step earlier, the only thing you need to know about the future is the value table you just built — not the many futures that lead to it. Each stage costs one sweep, so the total is linear in the horizon, and nothing is approximated. That is why the finite-horizon problem is *easier*: no fixed-point equation, just one pass.

4 Control Problems and Optimal Control *(slides 33–39)*

4.1 Control Problems *(slides 33–36)*

So far the state and action spaces were discrete. What about a cartpole? State: angular position & velocity, linear position & velocity; action/control: the force on the cart; goal: find the control

policy that minimises the long-term cost. More generally in control $x \in^d$ and $u \in^k$; we write x **for the state and** u **for the action**, and we talk about **cost** rather than reward: we *minimise* cost.

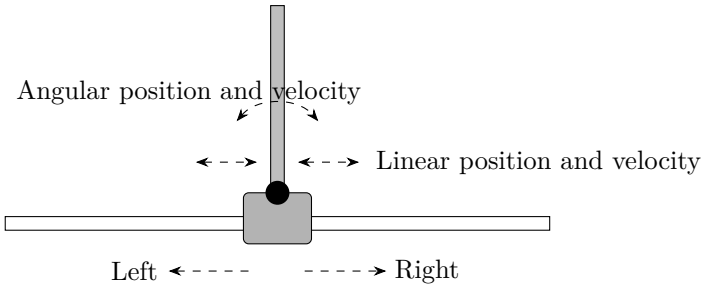


Figure 66: Cartpole, redrawn from slides 33–34: four state variables, one scalar control (horizontal force).

4.2 Optimal Control *(slides 37–39)*

Definition 4.1 (Optimal control problem)**.** Given a dynamical system with non-linear transition function f, state $x \in^d$ and control $u \in^k$, find a control policy π such that

$$\min_{\pi} \ \mathbb{E}_{\pi}\Big[c_H(x_H) + \sum_{h=0}^{H-1} c_h(x_h, u_h)\Big], \qquad u_h = \pi(x_h),\ x_0 \sim \mu_0.$$

This looks very familiar: can we treat it as a finite-horizon MDP and use value iteration? **Approach 1: discretise.**

5 Discretization and Bellman's Curse of Dimensionality *(slides 40–42)* (SKIPPED IN CLASS)

Discretise each coordinate of $x \in^d$ and $u \in^k$ at resolution ϵ. The number of grid points grows exponentially with the dimension (slide 40): $(1/\epsilon)^d$ states and $(1/\epsilon)^k$ actions. In practice discretisation is only computationally feasible up to **5 or 6 dimensional** state spaces (slides 41–42). **Approach 2:** work directly in continuous space, starting from simplified problems.

Proposition 5.1 (Bellman's curse of dimensionality)**.** *Discretising each dimension of x and u at resolution ϵ gives $|S| = \Theta((1/\epsilon)^d)$ and $|A| = \Theta((1/\epsilon)^k)$, so one level of backward induction costs $\Theta((1/\epsilon)^{2d+k})$ operations with a dense transition table, and $\Theta((1/\epsilon)^{d+k})$ when the dynamics are deterministic and each state-action pair has a single successor cell.*

Proof. A hypercube of side 1 discretised at resolution ϵ has $1/\epsilon$ points per dimension. The sweep cost of Theorem 3.2 is $|S|^2|A|$ per level, because the expectation over s' has $|S|$ terms; with a single successor it has one, and the cost is $|S||A|$. □

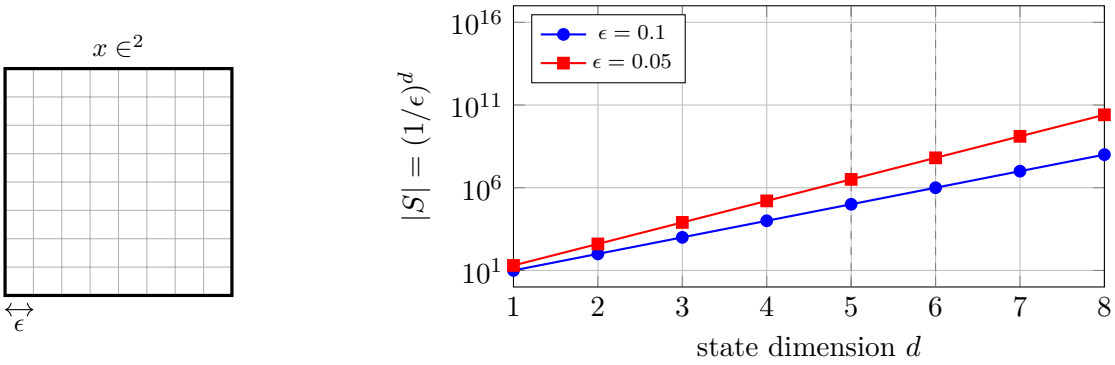


Figure 67: Left: grid at resolution ϵ (slide 40). Right: original illustration of the exponential growth; the dashed lines mark the 5–6 dimensions that the slides give as the practical limit.

Intuition

Halving ϵ multiplies the number of cells by 2^d. A 6-dimensional state at $\epsilon = 0.05$ already has 6.4×10^7 cells, and with a dense transition table the sweep costs the *square* of that times $|A|$. (The slide writes the grid size as a sum $(1/\epsilon)^d + (1/\epsilon)^k$; the number of state-action *pairs* that the sweep must touch is the product $(1/\epsilon)^{d+k}$, which is what Proposition 5.1 counts.) Discretisation is not wrong — it is just *exponentially expensive*, which motivates looking for problems where the continuous solution has a closed form.

6 Linear Systems and Quadratic Cost *(slides 43–46)*

6.1 Linear Systems *(slides 43–44)*

Consider the system

$$x_{t+1} = F_x x_t + F_u u_t, \qquad F_x \in^{d\times d},\ F_u \in^{d\times k},$$

where x_t is the state and u_t the control (action) at time t. This *is* our transition function.

6.2 Quadratic Cost Function *(slides 45–46)*

Definition 6.1 (Linear-quadratic regulator (LQR))**.** $x_{t+1} = F_x x_t + F_u u_t$, $c(x,u) = x^\top C_x x + u^\top C_u u$, with $C_x \succeq 0$ symmetric and $C_u \succ 0$ symmetric. The problem is to minimise $\mathbb{E}[x_H^\top C_x x_H + \sum_{h=0}^{H-1} c(x_h, u_h)]$.

(The slides ask for C_x, C_u both positive definite, so that any non-zero state costs something even with all-zero control; the companion notes only require $C_x \succeq 0$, which is what the proof needs.) The slides also use the alternative notation $g(x_t, u_t)$ for the cost.

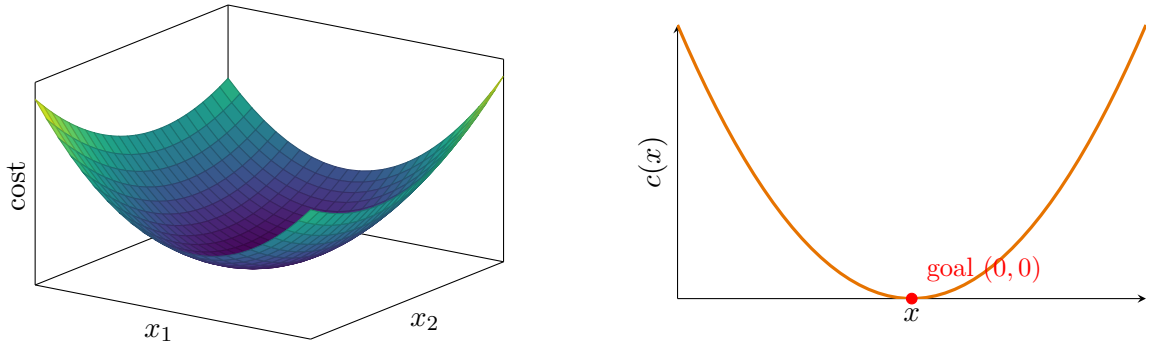


Figure 68: A quadratic cost with $C_x \succ 0$ is a bowl (two-dimensional case, left; cross-section, right), as drawn on slide 46. Its unique minimum is the goal $x = 0$.

Intuition

The cost is a bowl whose bottom is the origin: "be at the goal, and don't waste control effort". C_x says how much you dislike each kind of deviation, C_u how much you dislike each kind of effort. Note the problem is a *regulator*: the goal is to drive x to 0. All other cases (non-zero targets, trajectories) are reduced to this later by changing variables.

7 Q Functions for Linear Systems and Quadratic Cost *(slides 47–54)*

(Credits on the slides: Pieter Abbeel.) We do value iteration backwards in time, as in the previous section, but now in costs. The Q function at level h is the immediate cost plus the expected optimal cost-to-go of the next state (slides 47–49):

$$Q_h^*(x_t, u_t) = x_t^\top C_x x_t + u_t^\top C_u u_t + \mathbb{E}_{x' \sim p(\cdot|x_t,u_t)}\left[V_{h+1}^*(x')\right].$$

Deterministic for now (slide 50), the expectation disappears (slide 51):

$$Q_h^*(x_t, u_t) = x_t^\top C_x x_t + u_t^\top C_u u_t + V_{h+1}^*(F_x x_t + F_u u_t).$$

Inductive step: assumption (slide 52). Assume $V_{h+1}^*(x_t) = x_t^\top P_{h+1} x_t$ with $P \in^{d\times d}$. **Question (slide 53):** can we show that also $V_h^*(x_t) = x_t^\top P_h x_t$? **Implementing the assumption (slide 54):**

$$Q_h^*(x_t, u_t) = x_t^\top C_x x_t + u_t^\top C_u u_t + (F_x x_t + F_u u_t)^\top P_{h+1}(F_x x_t + F_u u_t).$$

Intuition

The hope is a *closed under iteration* family: if the cost-to-go is a quadratic form at level $h+1$ and the dynamics are linear, then plugging a linear map into a quadratic gives a quadratic; adding the quadratic stage cost keeps it quadratic. All that remains is the minimisation over u, and the minimum of a quadratic is easy.

8 Finding the Best Action and the Inductive Step *(slides 55–63)*

8.1 Finding the Best Action *(slides 55–57)*

We want $\pi_h^*(x) = \arg\min_u Q_h^*(x, u)$. How do you find minima? Set $\nabla_u Q_h^*(x, u) = 0$ and solve for u (slide 57).

8.2 Inductive Step: Proof *(slides 58–63)*

Everything is assembled in the following theorem, with the proof of the companion notes laid out along the slide steps.

Theorem 8.1 (LQR: exact solution by backward induction)**.** *Define* $P_H = C_x$ *and, for* $h = H-1, \dots, 0$*,*

$$K_h = -(C_u + F_u^\top P_{h+1} F_u)^{-1} F_u^\top P_{h+1} F_x,$$
$$P_h = C_x + K_h^\top C_u K_h + (F_x + F_u K_h)^\top P_{h+1}(F_x + F_u K_h).$$

Then $V_h^*(x) = x^\top P_h x$ *and the optimal control is the linear feedback* $\pi_h^*(x) = K_h x$*.*

Proof. Backward induction on h, with the statement "V_h^* is a quadratic form $x^\top P_h x$" as hypothesis.
Base (slides 64–65). At $h = H$ no control remains and the terminal cost is $x^\top C_x x$, so $P_H = C_x$. (The slides also allow $P_H = 0$ or $C_{x,\text{final}}$, i.e. a different terminal cost.)
Step (slides 58–63). Assume $V_{h+1}^*(x) = x^\top P_{h+1} x$. Then

$$Q_h^*(x,u) = x^\top C_x x + u^\top C_u u + (F_x x + F_u u)^\top P_{h+1}(F_x x + F_u u),$$

a quadratic in (x,u) jointly (slides 59–60). Its gradient in u is

$$\nabla_u Q_h^*(x,u) = 2C_u u + 2F_u^\top P_{h+1}(F_x x + F_u u) = 2(C_u + F_u^\top P_{h+1} F_u)u + 2F_u^\top P_{h+1} F_x x.$$

The matrix $C_u + F_u^\top P_{h+1} F_u$ is positive definite: $C_u \succ 0$ by hypothesis and the second term is positive semi-definite provided $P_{h+1} \succeq 0$, which holds by induction since P_h is a sum of positive semi-definite terms. Hence Q_h^* is strictly convex in u, the stationary point is the unique minimum, and setting the gradient to zero gives $u = K_h x$ with K_h as stated (slide 61).
Substituting $u = K_h x$ back, $V_h^*(x) = Q_h^*(x, K_h x)$ (slides 62–63), i.e.

$$V_h^*(x) = x^\top C_x x + x^\top K_h^\top C_u K_h x + \big((F_x + F_u K_h)x\big)^\top P_{h+1}\big((F_x + F_u K_h)x\big) = x^\top P_h x$$

with P_h as stated, which is again a quadratic form, closing the induction. □

Intuition

The two facts that make everything work: (i) *quadratic in, quadratic out* — the cost-to-go stays a bowl at every stage, so you never leave a finite-dimensional family; (ii) the best action is a *linear* function of the state because setting a gradient of a quadratic to zero is a linear equation. The solution is a feedback gain matrix K_h ("if you are this far from the goal, push this hard in the opposite direction"). Numerical sanity check in one dimension: $x_{t+1} = x_t + u_t$, $C_x = C_u = 1$, $P_{h+1} = 1$. Then $K_h = -(1+1)^{-1} \cdot 1 \cdot 1 = -\frac{1}{2}$ and $P_h = 1 + \frac{1}{4} + (1 - \frac{1}{2})^2 \cdot 1 = 1.5$: the controller cancels half the state per step and the value goes from 1 to 1.5.

9 The LQR Algorithm *(slides 64–70)*

9.1 Cost at H *(slides 64–65)*

Recall the goal $\min \mathbb{E}_\pi[c_H(x_H) + \sum_{h=0}^{H-1} c_h(x_h, u_h)]$. At H only $c_H(x_H)$ remains, typically 0 or $x_H^\top C_x x_H$. This is the base case: $P_H = 0$ or C_x (or $C_{x,\text{final}}$).

9.2 LQR algorithm *(slides 66–70)*

Algorithm 9.1 (LQR)**.** Initialise P_H (at 0 or C_x). Starting from $h = H-1$, *backwards*:

1. $K_h^* = -(C_u + F_u^\top P_{h+1} F_u)^{-1} F_u^\top P_{h+1} F_x$;
2. control $u = \pi_h^*(x_t) = K_h^* x_t$;
3. $P_h = C_x + K_h^{*\top} C_u K_h^* + (F_x + F_u K_h^*)^\top P_{h+1}(F_x + F_u K_h^*)$;
4. $V_h^* = x_t^\top P_h x_t$.

Slide 67: this is the *value iteration update done in closed form*: it is always the same and solves this particular continuous-state system with quadratic cost. Slide 68: the recursion for P_h is the **Riccati equation**. Slide 69: for this kind of problem, V is renamed J, the "cost-to-go", so $J_h^* = x_t^\top P_h x_t$.

Remark. Expanding the square gives the more familiar form $P_h = C_x + F_x^\top P_{h+1} F_x - F_x^\top P_{h+1} F_u (C_u + F_u^\top P_{h+1} F_u)^{-1} F_u^\top P_{h+1} F_x$.

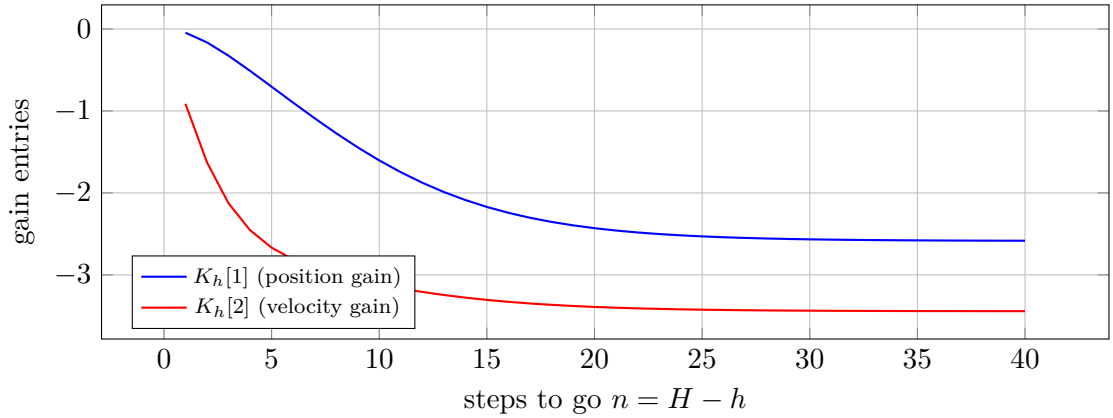


Figure 69: Original illustration (not on the slides). Riccati iteration for a discretised double integrator, $F_x = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}$, $F_u = \begin{bmatrix} 0.005 \\ 0.1 \end{bmatrix}$, $C_x = I$, $C_u = 0.1$, $P_H = C_x$, $H = 40$. Far from the end of the episode the gains change very slowly, near the end (n small) they shrink to zero: there is little time left to pay off.

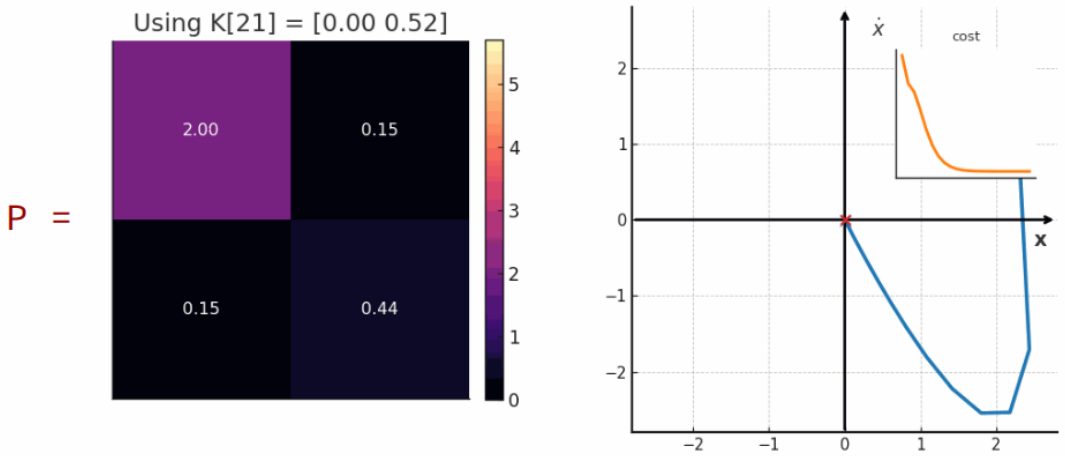


Figure 70: $P[h]$ as a heat map, the current gain $K[h]$ and the state-space trajectory

Intuition

LQR is value iteration where the whole value table is replaced by a single matrix P_h. One backward pass of cheap matrix algebra gives the *optimal controller for every time and every state*, with no grid. It is a controller, not just a plan: $u = K_h x$ tells you what to do from *wherever you end up*, which makes it robust to disturbances. Note also that K_h does not depend on the initial state at all.

10 LQR Extensions *(slides 71–76)*

The extensions make LQR applicable to: affine systems, systems with stochasticity, regulation around a non-zero fixed point for non-linear systems, trajectory following for non-linear systems, ...

10.1 LQR for Linear Time Varying (LTV) Systems *(slides 72–73)*

LTV system: $x_{t+1} = F_{x,t}x_t + F_{u,t}u_t$, cost $c(x_t, u_t) = x_t^\top Q_t x_t + u_t^\top R_t u_t$ (so F_x, F_u, C_x, C_u all carry the index t). Going backwards ($i = 1, 2, \dots$) the same recursion holds with time-indexed matrices:

$$K^*_{H-i} = -(C_{u,H-i} + F^\top_{u,H-i} P_{H-i+1} F_{u,H-i})^{-1} F^\top_{u,H-i} P_{H-i+1} F_{x,H-i},$$
$$P_{H-i} = C_{x,H-i} + K^{*\top}_{H-i} C_{u,H-i} K^*_{H-i} + (F_{x,H-i} + F_{u,H-i} K^*_{H-i})^\top P_{H-i+1}(\cdots).$$

Intuition

The proof of Theorem 8.1 never used that the matrices were the same at every step; it only used one step at a time. So the same argument goes through with time-indexed matrices. This is the engine for trajectory following and iLQR below.

10.2 LQR for Affine Systems *(slides 74)*

Affine system $x_{t+1} = F_x x_t + F_u u_t + c$ with cost $x_t^\top C_x x_t + u_t^\top C_u u_t$. Re-define the state and transition by appending a constant coordinate, $z_t = [x_t; 1]$:

$$z_{t+1} = \begin{bmatrix} x_{t+1} \\ 1 \end{bmatrix} = \begin{bmatrix} F_x & c \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_t \\ 1 \end{bmatrix} + \begin{bmatrix} F_u \\ 0 \end{bmatrix} u_t = F'_x z_t + F'_u u_t,$$

and the cost on z is $z^\top \mathrm{diag}(C_x, 0) z + u^\top C_u u$ (positive *semi*-definite, which Theorem 8.1 allows — this is why the companion notes only ask $C_x \succeq 0$). The resulting policy is affine in x: $u = K_{x,h} x + k_h$.

Intuition

The constant offset c is a "bias" in the dynamics. By pretending the number 1 is just another state that never changes, the bias becomes an ordinary linear term, and the linear machinery applies unchanged.

10.3 LQR for Stochastic Systems *(slides 75)*

Stochastic system $x_{t+1} = F_x x_t + F_u u_t + w_t$, $w_t \sim \mathcal{N}(0, \sigma^2 I)$, cost as before. The optimal control policy is the *same*; the cost-to-go has an extra term that depends on the variance of the system and cannot be controlled: $J^*_h = x_t^\top P_h x_t + p_h$, with base case $p_H = 0$ and $p_h = (\sigma^2 P_{h+1}) + p_{h+1}$.

Proposition 10.1 (Certainty equivalence)**.** *The optimal gains K_h are exactly those of Theorem 8.1, since the noise does not change the control law, and the value acquires an*

additive constant, $V_h^*(x) = x^\top P_h x + \sum_{\tau>h}(P_\tau \Sigma)$.

Proof. In the induction step the only change is the expectation over w:

$$\mathbb{E}[(F_x x + F_u u + w)^\top P_{h+1}(F_x x + F_u u + w)] = (F_x x + F_u u)^\top P_{h+1}(F_x x + F_u u) + (P_{h+1}\Sigma),$$

the cross terms vanishing because $\mathbb{E}[w] = 0$ and w is independent of (x, u). The added term does not depend on u, so the minimiser is unchanged; it accumulates through the recursion as stated. □

Intuition

Zero-mean noise pushes you around but, on average, it has no preference for any direction, so the best *response* to the state you see is unchanged: act as if there were no noise ("certainty equivalence"). The noise only costs you a fixed "tax" ($P\Sigma$) at every step that you cannot avoid. Example: with $\Sigma = \sigma^2 I$ and $P_{h+1} = 1.5$ in one dimension, each remaining step adds $1.5\sigma^2$ to the expected cost.

10.4 Penalize for Change in Controls *(slides 76)*

To penalise changes in the control rather than its magnitude, augment the state with the previous control, $x'_t = [x_t;\, u_{t-1}]$, and take $\Delta u_t = u_t - u_{t-1}$ as the new control:

$$\begin{bmatrix} x_{t+1} \\ u_t \end{bmatrix} = \begin{bmatrix} F_x & F_u \\ 0 & I \end{bmatrix} \begin{bmatrix} x_t \\ u_{t-1} \end{bmatrix} + \begin{bmatrix} F_u \\ I \end{bmatrix} \Delta u_t, \qquad \text{cost} = x'^\top C'_x x' + \Delta u^\top C'_u \Delta u, \quad C'_x = \begin{bmatrix} C_x & 0 \\ 0 & C_u \end{bmatrix}.$$

(Check: $x_{t+1} = F_x x_t + F_u(u_{t-1} + \Delta u_t) = F_x x_t + F_u u_t$.)

Intuition

Real actuators cannot jump: a car cannot change its steering angle instantly without wearing the hardware and unsettling the passengers. By making "the control I applied last time" part of the state, the *change* in control becomes the thing you choose and pay for, and you again have a standard LQR problem.

11 LQR for Non-Linear Systems *(slides 77–82)*

Non-linear system $x_{t+1} = f(x_t, u_t)$, cost $x_t^\top C_x x_t + u_t^\top C_u u_t$. We have a goal (x^*, u^*) that is a fixed point, $x^* = f(x^*, u^*)$, and want to stabilise the system around it.

Linearisation (slides 78–80). First-order Taylor expansion around (x^*, u^*):

$$x_{t+1} \approx f(x^*, u^*) + \frac{\partial f}{\partial x}(x^*, u^*)(x_t - x^*) + \frac{\partial f}{\partial u}(x^*, u^*)(u_t - u^*).$$

Let $z_t = x_t - x^*$ and $v_t = u_t - u^*$. Since $f(x^*, u^*) = x^*$,

$$z_{t+1} = A\, z_t + B\, v_t, \qquad A = \frac{\partial f}{\partial x}(x^*, u^*),\ B = \frac{\partial f}{\partial u}(x^*, u^*),$$

a linear system with cost $z^\top C_x z + v^\top C_u v$. **Solve it the usual way (slide 81):** $v_t = K z_t \Rightarrow u_t = u^* + K(x_t - x^*)$.

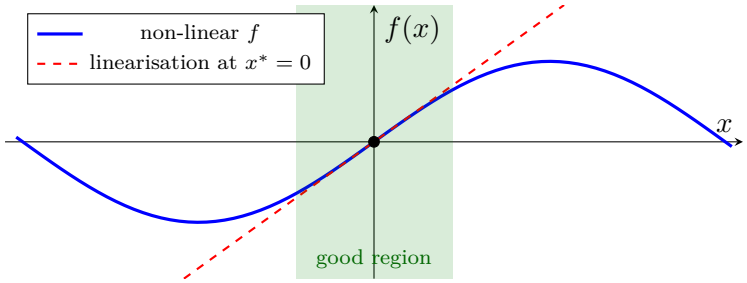


Figure 71: Original illustration (not on slides): **Near the fixed point the tangent is a good model of f (shaded), far from it it is not.**

Linearising the cost (slide 82). We can also linearise the cost with a 2nd-order Taylor expansion ("do it at home"), which gives something like

$$x_t^\top C_x x_t + u_t^\top C_u u_t + u_t^\top M x_t + x_t^\top q + u_t^\top r + c,$$

i.e. a quadratic with a *cross term* M, linear terms q, r and a constant c. The linear terms are handled by the constant-coordinate trick, the cross term by the cross-term form of Theorem 8.1.

Intuition

Zoom in enough on any smooth curve and it looks like a straight line. LQR on the linearised system is optimal for the *linear* approximation; it is a good controller as long as the state stays close to the point you linearised around — which is what a regulator tries to do. Think of balancing a pole: you only care about small deviations from upright.

12 LQR for Trajectory Following in Non-Linear Systems *(slides 83–85)*

Non-linear system $x_{t+1} = f(x_t, u_t)$; given a trajectory $x_0^*, \dots, x_T^*$ we want controls such that $x_{t+1}^* = f(x_t^*, u_t^*)$ along the trajectory; in other words we solve (slide 84)

$$\min_{u_0,\dots,u_{H-1}} \sum_{t=0}^{H-1} (x_t - x_t^*)^\top C_x (x_t - x_t^*) + (u_t - u_t^*)^\top C_u (u_t - u_t^*) \quad \text{s.t. } x_{t+1} = f(x_t, u_t).$$

We use the same trick, but linearise around (x_t^*, u_t^*) — the linearisation point *changes with* t, so we get an LTV system (slide 85): with $z_t = x_t - x_t^*$ and $v_t = u_t - u_t^*$,

$$z_{t+1} \approx A_t z_t + B_t v_t, \quad A_t = \frac{\partial f}{\partial x}(x_t^*, u_t^*), \; B_t = \frac{\partial f}{\partial u}(x_t^*, u_t^*),$$

which is solved with LQR for LTV systems, giving $u_t = u_t^* + K_t(x_t - x_t^*)$.

Intuition

Regulating around a point is the special case where the target never moves. Following a trajectory is regulating around a *moving* point: at each time you linearise the world around where you *should* be, and compute the gain that pulls you back onto the path. This is the idea you need for iLQR: if the trajectory you are tracking is the one you are currently trying to improve, you get a sequence of better and better LQR problems.

13 Iterative LQR (iLQR) *(slides 86–92)*

13.1 Iterative LQR (iLQR) *(slides 86)*

iLQR attempts to solve the *most general case* $\min \sum_{h=0}^{H} g_h(x_h, u_h)$ subject to $x_{t+1} = f(x_t, u_t)$, by iteratively approximating it and leveraging the LQR formulation.

13.2 Iterative LQR — Algorithm *(slides 87)*

Algorithm 13.1 (Iterative LQR)**.** *Initialisation:* pick either a control policy $\pi^{(0)}$ or a sequence $\{\bar{x}_t^{(0)}, \bar{u}_t^{(0)}\}_{t=0}^{H}$ (by simulating any policy or by guessing). For $i = 0, \dots, I$:

1. execute (or simulate) $\pi^{(i)}$ and collect $\{\bar{x}_t^{(i)}, \bar{u}_t^{(i)}\}_{t=0}^{H}$;
2. obtain the LQ approximation of the problem, linearising around $\{\bar{x}_t^{(i)}, \bar{u}_t^{(i)}\}$ with a 1st-order Taylor expansion for the dynamics and a 2nd-order one for the cost, giving a time-varying linear-quadratic problem in the deviations;
3. compute $\pi^{(i+1)}$ with the LQR-LTV formulation, in its cross-term form and augmenting the state with a constant coordinate as for affine dynamics: the expanded cost has a linear term wherever its gradient along the reference is not zero, the constant coordinate turns its part in δu_t into a mixed term, and its Hessian may couple δx_t and δu_t as well. The solution is an *affine* policy $\delta u_t = k_t^{(i)} + K_t^{(i)} \delta x_t$;
4. roll the true system forward under $u_t = \bar{u}_t^{(i)} + k_t^{(i)} + K_t^{(i)}(x_t - \bar{x}_t^{(i)})$ to obtain the next trajectory, and iterate.

13.3 Iterative LQR — LTV *(slides 88–89)*

With $z_t = [x_t - \bar{x}_t^{(i)};\ 1]^\top$ and $v_t = u_t - \bar{u}_t^{(i)}$ the dynamics are

$$F_{x,t} = \begin{bmatrix} \frac{\partial f}{\partial x}(\bar{x}_t^{(i)}, \bar{u}_t^{(i)}) & f(\bar{x}_t^{(i)}, \bar{u}_t^{(i)}) - \bar{x}_{t+1}^{(i)} \\ 0 & 1 \end{bmatrix}, \qquad F_{u,t} = \begin{bmatrix} \frac{\partial f}{\partial u}(\bar{x}_t^{(i)}, \bar{u}_t^{(i)}) \\ 0 \end{bmatrix},$$

(the top-right block is zero when the reference is a true rollout), and, assuming $g(x, u) = g(x) + g(u)$ (separable), the 2nd-order expansion of the cost $g(x_t, u_t) \approx z_t^\top C_{x,t} z_t + v_t^\top C_{u,t} v_t + 2 z_t^\top N_t v_t$ has

$$C_{x,t} = \begin{bmatrix} \frac{1}{2}\frac{\partial^2 g}{\partial x^2} & \frac{1}{2}\frac{\partial g}{\partial x} \\ \frac{1}{2}\left(\frac{\partial g}{\partial x}\right)^\top & g(x) + g(u) \end{bmatrix}, \quad C_{u,t} = \frac{1}{2}\frac{\partial^2 g}{\partial u^2}, \quad N_t = \begin{bmatrix} 0 \\ \frac{1}{2}\left(\frac{\partial g}{\partial u}\right)^\top \end{bmatrix},$$

all evaluated at $(\bar{x}_t^{(i)}, \bar{u}_t^{(i)})$. This is exactly the *quadratic approximation behind iLQR*: its $2z^\top N v$ term is the mixed term mentioned in step 3 of Algorithm 13.1. Check: expanding, $z^\top C_x z = \frac{1}{2}\delta x^\top g_{xx} \delta x + g_x^\top \delta x + g$, $v^\top C_u v = \frac{1}{2} v^\top g_{uu} v$, $2 z^\top N v = g_u^\top v$, which is the second-order Taylor expansion of g.

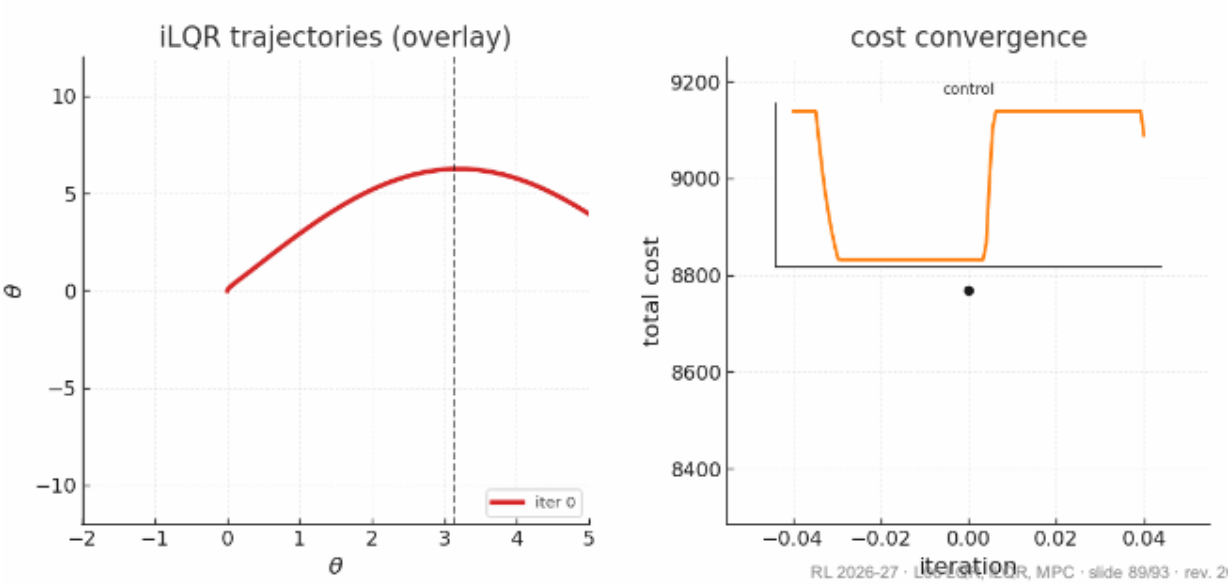


Figure 72: left panel "iLQR trajectories (overlay)" in the $(\theta, \dot{\theta})$ plane, right panel "cost convergence"

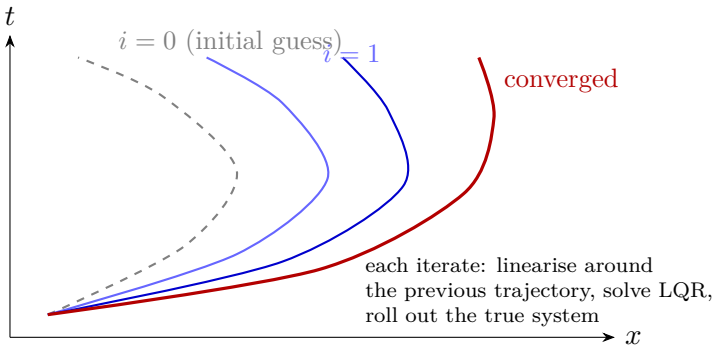


Figure 73: Original schematic (not on the slides) of iLQR: each iteration replaces the nominal trajectory with the roll-out of the LQR policy of the local LQ model.

Intuition

iLQR is Newton-like optimisation done by dynamic programming. At each iteration you build a *local* linear-quadratic model of the world around your current best trajectory, solve it exactly with LQR, and apply the result to the real system to get a new (better) trajectory. The affine policy $\delta u = k + K\delta x$ has two parts: k is the feed-forward correction to the plan, K is the feedback that keeps you on it.

13.4 Iterative LQR — Convergence *(slides 90–91)*

As it is, iLQR might not converge: the optimal policy of the LQ approximation might end up *not staying close* to the linearisation points. **Solution:** change the cost to

$$(1-\alpha)\, g(x_t, u_t) + \alpha(\|x_t - \bar{x}_t^{(i)}\|_2^2 + \|u_t - \bar{u}_t^{(i)}\|_2^2),$$

and if g is bounded and α is close enough to 1, the second term dominates and ensures the linearisations are good approximations of the solution trajectory.

Intuition

It is a leash. The LQ model is only trustworthy near the nominal trajectory, so you add a penalty for straying from it. With $\alpha = 0.9$, one unit of deviation costs 0.9 while a unit of true cost counts 0.1: the update becomes timid but safe, and you take many small, reliable steps instead of one big wrong leap. This is the same idea as a trust region.

13.5 Iterative LQR — Tricks *(slides 92)*

- **Initialisation matters**: you can get stuck in local optima. Alternatives: simulated annealing (if there are no computation constraints) or line search.
- If g is non-convex, the LQ approximation may give non positive-definite C_x, C_u. **Fix:** add more cost for deviating from the linearisation points until they become positive-definite.

14 Receding Horizon Control, i.e. Model Predictive Control (MPC) *(slides 93)*

At iLQR convergence, the system could execute the policy and still not be at the expected trajectory (inaccurate dynamics, noise, ...). **Idea:** at every time step t replan from t to H. **In practice:** replanning until H at every step is too costly, so replan over a *shorter horizon* h and use a cost-to-go $J(t+h)$ at its end. This is a Model Predictive Controller with an iLQR solver.

Algorithm 14.1 (Receding-horizon control (MPC))**.** At each time t: solve the control problem from the current *measured* state over a short horizon $[t, t+h]$, with a terminal cost approximating the cost-to-go beyond h; apply only u_t; discard the rest; measure; repeat.

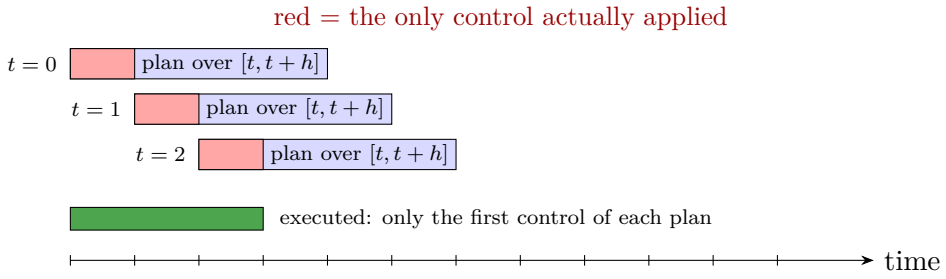


Figure 74: Original schematic (not on the slides): the planning window slides forward at every step.

Intuition

It is the way you navigate with GPS: you do not drive the original route blindly, you recompute from where you *are* after every turn. Replanning gives feedback through the optimiser, so errors due to noise or a wrong model are corrected at the next step. The finite window keeps the computation affordable, and the terminal cost-to-go $J(t+h)$ stops the controller from being short-sighted.

Method	Setting	Output	Cost / scaling	Main limitation
Backward induction (Thm. 3.2)	finite-horizon MDP, discrete S, A	V_h^*, π_h^* (tables), exact	$O(H\|S\|^2\|A\|)$; with a grid $\Theta((1/\epsilon)^{2d+k})$	curse of dimensionality: only $d \lesssim 5$–6
LQR (Thm. 8.1)	linear dynamics, quadratic cost	K_h, P_h, optimal linear feedback, exact	closed-form Riccati recursion, polynomial in d, k (matrix products/inverse)	needs linear model and quadratic cost
LQR extensions	LTV, affine, stochastic (Prop. 10.1), Δu penalty	same recursion on augmented/time-indexed matrices	same as LQR	still linear-quadratic
Linearised LQR	non-linear f near a fixed point or a reference trajectory	$u = u^* + K(x - x^*)$	LQR-LTV along reference	valid only near the reference
iLQR (Alg. 13.1)	general non-linear f, general g	locally optimal affine policy / trajectory	$I\times$ (linearise + LQR-LTV)	may diverge; local optima; needs trust-region/penalty
MPC (Alg. 14.1)	any of the above, online	u_t re-computed at every step	one short-horizon solve per step	model error, compute budget per step; needs terminal cost

Table 3: Summary of the methods of lecture L06.

15 Summary

15.1 Comparison of the methods

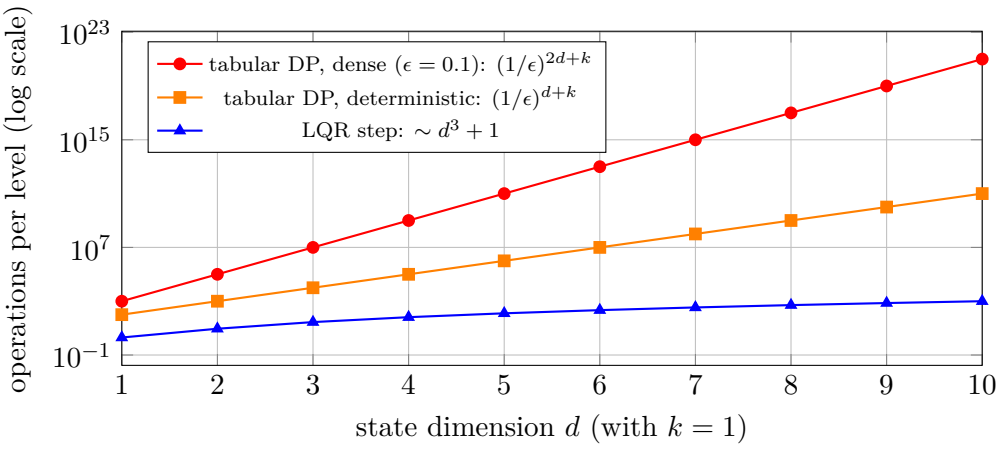


Figure 75: Qualitative comparison (original, order-of-magnitude, constants ignored): tabular DP grows exponentially in d, a Riccati step grows like a polynomial. LQR escapes the curse of dimensionality by *structure* (linear dynamics, quadratic cost), not by approximation.

15.2 Key take-aways

- **Finite horizon makes DP exact and one-pass**: backward induction from the deadline gives time-dependent optimal policies, but its cost explodes with dimension when the state

is discretised (Bellman's curse).

- **LQR is value iteration in closed form**: for linear dynamics and quadratic cost the value is $x^\top P_h x$ and the policy is the linear feedback $u = K_h x$, computed by the Riccati recursion with no discretisation.
- **Extensions reduce to the same recursion**: affine terms (constant coordinate), LTV matrices, penalties on Δu (augmented state); additive zero-mean noise leaves the gains unchanged (certainty equivalence) and only adds $(P\Sigma)$ to the cost.
- **Non-linear problems are handled by repeated local LQR**: linearise around a point or a trajectory (regulation, tracking), and iterate in iLQR with 1st-order dynamics and 2nd-order cost; keep the iterates close to the linearisation points (penalty weight α) and watch for local optima and non positive-definite Hessians.
- **MPC adds feedback through replanning**: solve over a short horizon from the measured state, apply the first control only, repeat; this makes the system robust to model error and noise at the price of one solve per time step.

A Supplementary material (in the companion PDFs, not on the slides)

The following result is in the companion notes "Finite-horizon MDPs and optimal control" but has **no corresponding slide**: the slides only state (slides 13–15) that policies are time-dependent, without an example proving it is necessary.

Proposition A.1 (The optimal policy depends on the time remaining)**.** *There are finite-horizon MDPs in which every optimal policy satisfies $\pi_h^*(s) \neq \pi_{h'}^*(s)$ for some state s and some $h \neq h'$.*

Proof. Two actions at a single state s. Action *safe* pays 1 and returns to s. Action *gamble* pays 0 now and moves to an absorbing state g paying 3 per step with probability $1/2$, or to an absorbing state b paying 0 per step with probability $1/2$. Write $n = H - h$ for the number of steps remaining and V_n for the optimal value at s with n steps left, $V_0 = 0$. Gambling now is worth $Q_n(\text{gamble}) = \frac{1}{2} \cdot 3(n-1) = 1.5(n-1)$, since the first step pays nothing and the remaining $n-1$ pay 3 half the time; playing safe now is worth $Q_n(\text{safe}) = 1 + V_{n-1}$, one now and the best play afterwards.

For $n \leq 3$ safe is optimal throughout: by induction $V_{n-1} = n - 1$, so $Q_n(\text{safe}) = n$, which beats $1.5(n-1)$ at $n \leq 2$ and ties with it at $n = 3$, giving $V_3 = 3 = 1.5 \cdot 2$. From there $V_{n-1} = 1.5(n-2)$ by induction, and

$$Q_n(\text{gamble}) - Q_n(\text{safe}) = 1.5(n-1) - 1 - 1.5(n-2) = \tfrac{1}{2} > 0 \qquad (n \geq 4),$$

so the gamble is optimal and $V_n = 1.5(n-1)$, which carries the induction. So safe is strictly better for $n \leq 2$, the two tie at $n = 3$, and the gamble is strictly better for $n \geq 4$. Any optimal policy therefore gambles early in a long episode and plays safe in the last two steps. □

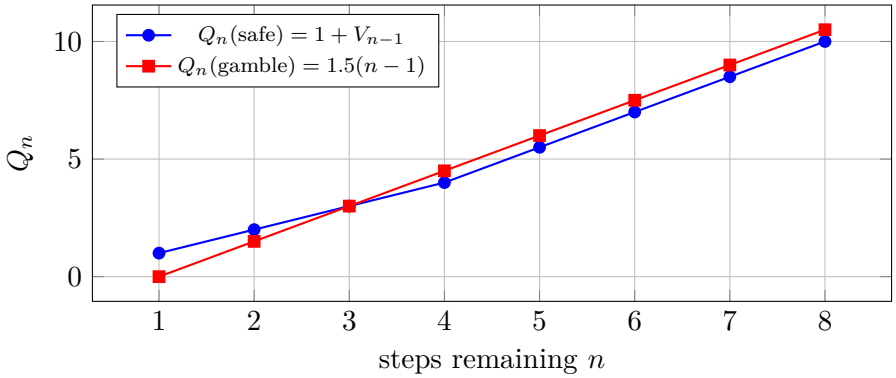


Figure 76: Values of the two actions in the proof of Proposition A.1 (original figure). The curves cross at $n = 3$.

Intuition

Gambling pays off only if there is enough time to harvest the jackpot: the expected gain per remaining step (1.5) beats the safe payoff (1) only if enough steps remain to compensate for the one step you give up. Concretely, with $n = 2$ you get 2 safe versus 1.5 from gambling, with $n = 4$ you get 4 versus 4.5. The identical state calls for different actions at different times, so no single stationary policy can be optimal; this justifies the time-dependent policies $\pi = (\pi_0, \ldots, \pi_{H-1})$ of Definition 2.1.

A note on the companion notes' "slides only" items. The self-check of the LQR PDF lists as "on the slides only" the derivations for affine dynamics, penalising changes in control, LQR around a fixed point, trajectory following, and the quadratic approximation behind iLQR. They are in the main text at their slide position (Sections on LQR Extensions, Non-Linear Systems, Trajectory Following, iLQR).